BİLGİSAYAR MÜHENDİSLİĞİ ANABİLİM DALI BAŞKANLIĞINA

24 Nisan 2026

Bilgisayar Mühendisliği Anabilim Dalı yüksek lisans öğrencisi Nazarkulov Elaman'ın (Öğrenci No: 2351y01005), 02.09.2024 tarihli onaylanmış 'Referans düğüm yöntemiyle sinyal büyütmeye dayalı 12 kanallı elektrokardiyografi (EKG) kullanarak kalp hastalıklarını teşhis etmek için sinir ağı' başlıklı tez önerisi kapsamında, KTMÜ Yüksek Lisans Programı Uygulama Yönergesi gereksinimlerine uygun olarak hazırladığı yüksek lisans tezi aşağıda sunulmuştur. Tez; Chapman-Shaoxing 12 kanallı EKG külliyatı üzerinde giriş uzunluğu tasarımının kontrollü ampirik bir incelemesini, geometrik değişmezlik argümanını ve PyTorch referans uygulamasının tamamını içermektedir.

İmza
Tez Danışmanı
Doç. Dr. Bakıt ŞARŞEMBAEV

İmza
Eş Danışman
Doç. Dr. Rayımbek SULTANOV

EK: Yüksek Lisans Tezi (Tam Metin)

Kırgızistan-Türkiye Manas Üniversitesi Rektörlük © 2024

KIRGIZİSTAN - TÜRKİYE MANAS ÜNİVERSİTESİ
FEN BİLİMLERİ ENSTİTÜSÜ
BİLGİSAYAR MÜHENDİSLİĞİ ANABİLİM DALI

YÜKSEK LİSANS TEZİ

REFERANS DÜĞÜM YÖNTEMİYLE SİNYAL BÜYÜTMEYE DAYALI 12 KANALLI ELEKTROKARDİYOGRAFİ (EKG) KULLANARAK KALP HASTALIKLARINI TEŞHİS ETMEK İÇİN SİNİR AĞI

BAHAR 2026

ELAMAN NAZARKULOV

2026
BİŞKEK

Raporun Kapsadığı Dönem	Bahar 2026 — Yüksek Lisans Tezi
Tez Savunma Tarihi (Hedef)	24 Nisan 2026
Hazırlandığı Tarih	24 Nisan 2026

ÖZET ve ANAHTAR KELİMELER

Tam özet sonraki sayfalardadır; bu kutu Manas şablonunun kapak özeti içindir.

ÖZET (KISA)

Bu çalışma, 12 kanallı EKG girişinin scipy.signal.decimate ile 5000 örneklemden 500 örnekleme anti-aliasing'li (örtüşme önleyici) altörneklenmesinin, temel bir 1B evrişimli sinir ağının (1D-CNN) Chapman-Shaoxing veri setindeki test doğruluğunu %88,43'ten %97,34'e (makro-F1: 0,8713 → 0,9737) yükselttiğini göstermektedir. Sonuç; modelde, kayıp fonksiyonunda veya artırma reçetesinde herhangi bir değişiklik yapılmadan elde edilmiştir. Bulgu, referans noktalar grafının geometrik değişmezliği çerçevesinde açıklanmakta olup tezde PyTorch uygulamasının tamamı verilmiştir.

Anahtar Kelimeler: elektrokardiyogram, 12 kanallı EKG, derin öğrenme, 1B evrişimli sinir ağı, anti-aliasing'li altörnekleme, PyTorch, çoklu-etiket sınıflandırma, referans noktalar, geometrik değişmezlik.

ÖZ

Kardiyovasküler hastalıklar, Dünya Sağlık Örgütü verilerine göre yıllık tahmini 17,9 milyon ölüme yol açmakta ve dünya genelinde başlıca ölüm nedeni olmayı sürdürmektedir. 12 kanallı elektrokardiyogram (EKG); aritmiler, ileti bozuklukları, iskemik olaylar ve yapısal kardiyak anormalliklerin saptanmasında temel non-invaziv tanı aracı olarak kabul edilmektedir. Otomatik EKG sınıflandırmasında derin evrişimli sinir ağları (CNN), geleneksel olarak kanal başına 5000 örneklemden oluşan ham 500 Hz × 10 sn sinyal üzerinde uygulanmaktadır. Bu tezde, söz konusu varsayılan giriş uzunluğunun nötr bir tasarım tercihi olduğu varsayımı, kontrollü bir ablation çalışmasıyla sorgulanmıştır.

Chapman-Shaoxing 12 kanallı EKG külliyatı (45.152 kayıt, 78 çoklu-etiket tanı kategorisi) üzerinde model mimarisi (3,72 milyon parametreli artıklı 1B-CNN), Adam eniyileyici, odak kayıp ile etiket yumuşatmalı çapraz entropi, geleneksel veri artırma politikası, rastgele tohum (42) ve eğitim/doğrulama/test ayrımı sabit tutularak {5000, 1000, 500} giriş uzunluklarının kontrollü karşılaştırması yapılmıştır. Tüm sinyaller, 0,5–40 Hz Butterworth bant geçiren süzgeçten ve kanal başına z-skor normalleştirmesinden geçirildikten sonra altörnekleme katsayısı $q \in \{1, 5, 10\}$ ile altörneklenmiştir.

Temel bulgu olarak, girdinin SciPy'nin sekizinci dereceden Chebyshev tip-I IIR süzgeci (scipy.signal.decimate, ileri-geri/sıfır-fazlı mod) kullanılarak 500 örnekleme (etkin örnekleme hızı 50 Hz) anti-aliasing'li altörneklenmesinin, test doğruluğunu %88,43'ten %97,34'e ve makro-F1 değerini 0,8713'ten 0,9737'ye yükselttiği gözlemlenmiştir. Bu, mutlak 8,91 yüzde puanlık ve göreli %11,7 oranında bir iyileşmeye karşılık gelmektedir. Aynı koşullarda tek-örnek çıkarım gecikmesi tek bir NVIDIA RTX 5090 GPU üzerinde 89,88 ms'den 27,20 ms'ye (3,3× hızlanma) indirilmiş; epoch süresi ~195 sn'den ~20 sn'ye (9,8× hızlanma) düşürülmüştür. Tam eğitim, len=500 + 4 DataLoader işçisi konfigürasyonunda on dakikanın altına sığmıştır.

Sınıf bazında çözümleme, temel modelde başarısız olan on bir sınıfın ($F1 < 0,60$; en kötü durum: Sol Ventriküler Hipertrofi, $F1 = 0,022$) tek tip biçimde $F1 \geq 0,95$ düzeyine geri kazandırıldığını ortaya koymuştur. Anormal Q dalgası, Atriyal flatter ve İç ileti farklılıkları gibi morfolojik imzaya sahip sınıflar $F1 \geq 0,99$ düzeyine ulaşmıştır. Geri kazanım profili, ritime dayalı sınıflar (Atriyal fibrilasyon, AV blokları) için alıcı alan kapsamı; morfoloji sınıfları için parametre ekonomisi etkilerini birlikte yansıtmaktadır.

Sonuç, referans noktaların geometrik değişmezliği çerçevesinde değerlendirilmiştir: EKG'nin tanısal içeriği, atım başına P, Q, R, S, T olmak üzere yaklaşık 60 referans nokta içeren seyrek bir kümede yoğunlaşmakta; Chebyshev tip-I anti-aliasing süzgeci ise hem zamansal konumları (±10 ms hassasiyetinde) hem de göreli genlikleri korumaktadır. Girdinin 5000'den 500 örnekleme indirgenmesi, referans nokta yoğunluğunu on katına çıkarmakta; ağın yaklaşık 2048 örneklik etkin alıcı alanının pencerenin yalnızca %40'ını değil, tüm 10 saniyelik pencereyi kapsamasına olanak tanımaktadır. Naif bir sıralı havuzlama (anti-aliasing süzgeci olmadan) ise

doğruluğu %84 düzeyine düşürmekte; bu, anti-aliasing adımının vazgeçilmezliğini ortaya koymaktadır.

Tezde, giriş uzunluğunun yayımlanmış EKG kıyaslama çalışmalarında yetersiz raporlanan bir tasarım değişkeni olduğu ileri sürülmektedir. Düz 1B-CNN modelleri ile literatürde bildirilen dikkat/hibrit modeller (Oh ve ark. 2018: %94,8; Strodthoff ve ark. 2020: 0,925 makro-AUC) arasındaki performans farkının önemli bir kısmının mimari karmaşıklıktan değil, giriş uzunluğu eniyilemesinden kaynaklanabileceği savunulmaktadır. Bu bulgu, dikkat veya yinelemeli mekanizmaların yararsız olduğu anlamına gelmemekte; bu mekanizmaların uzunluk-eniyilenmiş bir temel modele karşı yeniden değerlendirilmesi gerektiğini ima etmektedir.

Veri yükleme, SNOMED CT etiket eşleme, ön işleme hattı, artıklı 1B-CNN mimarisi, karma duyarlıklı (AMP/FP16) odak kayıplı eğitim, değerlendirme ve tek-vuruşlu çıkarım dahil olmak üzere PyTorch referans uygulamasının tamamı (yaklaşık 1.800 satır, tek bir bağımsız modülde), tezin yöntem ve uygulama bölümleri ile EK 1'de verilmiştir. Eşlik eden klinik karar destek web uygulaması (FastAPI + React + ONNX Runtime), pilot kullanımdadır. Gelecek çalışmalar PTB-XL çapraz veri seti doğrulamasını, dikkat ve odak kayıp mekanizmalarının uzunluk-eniyilenmiş temel üzerinde yeniden değerlendirilmesini ve INT8 kenar dağıtımını kapsamaktadır.

Anahtar Kelimeler: elektrokardiyogram, derin öğrenme, evrişimli sinir ağı, anti-aliasing'li altörnekleme, geometrik değişmezlik, çoklu-etiket sınıflandırma.

КЫСКАЧА МАЗМУНУ

Дүйнөлүк ден соолук уюмунун маалыматы боюнча, жүрөк-кан тамыр оорулары жыл сайын 17,9 миллион өлүмгө алып келип, дүйнөдөгү башкы өлүм себеби болуп келет. 12 каналдуу электрокардиограмма (ЭКГ); аритмияларды, өткөрүү бузулууларын, ишемиялык окуяларды жана түзүлүштүк жүрөк патологияларын аныктоо үчүн негизги инвазивдүү эмес диагностикалык каражат болуп саналат. Автоматташтырылган ЭКГ классификациясында терең конволюциялык нейрондук тармактар (CNN) адатта канал башына 5000 үлгүдөн турган 500 Гц × 10 секунд белгисине колдонулат. Бул иште, ушул стандарттык кириш узундугунун нейтралдуу тандоо экендиги контролдонуучу абляция изилдөөсү аркылуу талкууланган.

Chapman-Shaoxing 12 каналдуу ЭКГ маалымат базасы (45.152 жазуу, 78 көп-белгилүү диагноз категориясы) колдонулуп, моделдин архитектурасы (3,72 миллион параметрлүү калдыктуу 1B-CNN), Adam оптимизатору, фокалдуу жоготуу менен жумшартылган белгилерге негизделген кросс-энтропия, үлгүлөрдү көбөйтүү саясаты, баштапкы урук (42) жана окутуу/валидация/тест бөлүштүрүүсү туруктуу сакталып, {5000, 1000, 500} кириш узундуктары контролдоого алынып салыштырылган. Бардык сигналдар 0,5–40 Гц Баттерворт зоналык өткөрүүчү фильтрден жана канал боюнча z-skor нормалдашуудан өткөн.

Негизги жыйынтык катары, кириштин SciPy'нин сегизинчи даражадагы Чебышев тип-I IIR фильтри (scipy.signal.decimate, нөл фазалык режимде) колдонулуп 500 үлгүгө чейин (натыйжалуу тандоо ылдамдыгы 50 Гц) anti-aliasing менен кыскартуусу тест тактыгын %88,43тен %97,34кө жана макро-F1 көрсөткүчүн 0,8713тен 0,9737ге жогорулатат. Бул, абсолюттук 8,91 пайыздык жана салыштырмалуу %11,7'лик жакшырууга туура келет. Ушул эле шарттарда бир үлгүлүк божомолдоо узактыгы NVIDIA RTX 5090 GPU'да 89,88 мс'ден 27,20 мс'ге (3,3× ылдамдатуу) кыскарган; эпоха узактыгы ~195 секунддан ~20 секундга (9,8× ылдамдатуу) кыскарган. Толук окутуу 10 минуттан ашпаган.

Класс боюнча талдоо, базалык моделде ийгиликсиз болгон он бир класс ($F1 < 0,60$; эң начары: Сол Каринчелик Гипертрофия, $F1 = 0,022$) бирдей түрдө $F1 \geq 0,95$ деңгээлине кайтарылганын көрсөтөт. Кээ бир класстар (Аномалдуу Q толкуну, Алдыртан флаттер, Ички өткөрүү айырмасы) $F1 \geq 0,99$ деңгээлине жетет. Кайтарып келүү профили ритмге негизделген класстар үчүн кабыл алуу талаасынын камтуусунан, морфология класстары үчүн параметр экономиясынан пайдалангандыгын көрсөтөт.

Натыйжа, референс чекиттеринин геометриялык өзгөрүүсүздүгү алкагында түшүндүрүлөт: ЭКГнин диагностикалык мазмуну, ар бир кагуу үчүн P, Q, R, S, T'ден турган болжол менен 60 референс чекитти камтыган сейрек жыйнакта топтолот; Чебышев тип-I анти-aliasing фильтри убакыттык позицияларды (±10 мс тактыгында) жана салыштырма амплитудаларды сактайт. Кириштин 5000ден 500гө чейин кыскартылышы референс чекиттердин тыгыздыгын он эсеге жогорулатат жана тармактын болжол менен 2048 үлгүлүк натыйжалуу кабыл алуу талаасы терезенин %40 эмес, бүтүндөй 10 секунддук терезени камтыйт.

Бул изилдөөдө, кириш узундугу жарыяланган ЭКГ өлчөө изилдөөлөрүндө жетиштүү билдирилбеген долбоорлоо чен берилгич экендиги ырасталат; жөнөкөй 1B-CNN моделдери менен заманбап адабиятта билдирилген көңүл буруу/гибрид моделдер ортосундагы аткаруу айырмачылыгынын олуттуу бөлүгүнүн архитектуралык татаалдыктан эмес, кириш узундугун оптимизациялоодон келип чыгышы мүмкүн экендиги колдоого алынат. Маалыматты жүктөө, SNOMED CT белгилерин шайкеш келтирүү, алдын ала иштетүү, калдыктуу 1B-CNN архитектурасы, аралаш тактыктагы (AMP/FP16) фокалдуу жоготуу окутуусу жана бир жолку божомолдоо камтыган толук PyTorch шилтемелик ишке ашыруусу тездин ыкма жана ишке ашыруу бөлүмдөрүндө жана 1-Тиркемеде берилет.

Ачкыч сөздөр: электрокардиограмма, 12 каналдуу ЭКГ, терең нейрондук тарм, анти-aliasing, PyTorch, көп белгилүү классификация, геометриялык өзгөрүүсүздүк.

АБСТРАКТ

По данным Всемирной организации здравоохранения, сердечно-сосудистые заболевания ежегодно становятся причиной около 17,9 миллионов смертей и остаются ведущей причиной смертности во всём мире. Двенадцатиканальная электрокардиограмма (ЭКГ) является основным неинвазивным диагностическим инструментом для выявления аритмий, нарушений проводимости, ишемических событий и структурных кардиальных аномалий. В автоматической классификации ЭКГ глубокие свёрточные нейронные сети (CNN) традиционно применяются к необработанному сигналу 500 Гц × 10 с, состоящему из 5000 отсчётов на канал. В данной диссертации проведено контролируемое исследование, ставящее под сомнение нейтральность этого выбора длины входа.

На корпусе Chapman-Shaoxing (45 152 записей, 78 многометочных диагностических категорий) при фиксированной архитектуре (резидуальная 1D-CNN с 3,72 млн параметров), оптимизатора Adam, фокальной потери со сглаживанием меток, политики аугментации, случайного зерна (42) и разделения обучение/валидация/тест проведено контролируемое сравнение длин входа {5000, 1000, 500}. Все сигналы предварительно фильтровались полосовым фильтром Баттерворта 0,5–40 Гц и нормализовались по каналу z-нормировкой.

Основной результат: замена входа на антиалиасинговое прореживание до 500 отсчётов (эффективная частота дискретизации 50 Гц) с помощью фильтра Чебышева типа I восьмого порядка (scipy.signal.decimate в режиме нулевой фазы) повышает точность теста с 88,43 % до 97,34 % и макро-F1 с 0,8713 до 0,9737. Это соответствует абсолютному улучшению на 8,91 процентных пункта и относительному улучшению на 11,7 %. При тех же условиях задержка одиночного вывода на GPU NVIDIA RTX 5090 сокращается с 89,88 мс до 27,20 мс (3,3-кратное ускорение); время эпохи — с ~195 с до ~20 с (9,8-кратное ускорение). Полное обучение укладывается в десять минут.

Поклассовый анализ показал, что одиннадцать неудачных классов базовой модели (F1 < 0,60; наихудший случай: гипертрофия левого желудочка, F1 = 0,022) равномерно восстанавливаются до F1 ≥ 0,95. Несколько классов с морфологической сигнатурой (аномальный Q-зубец, трепетание предсердий, межжелудочковые нарушения проводимости) достигают F1 ≥ 0,99. Профиль восстановления отражает выгоды от увеличения охвата рецептивного поля для ритмических классов и параметрической экономии для морфологических классов.

Результат интерпретируется в рамках геометрической инвариантности графа реперных точек: диагностический контент ЭКГ сосредоточен в разреженном множестве примерно из 60 реперных точек (P, Q, R, S, T для каждого из ~10 ударов), временные позиции которых антиалиасинговый фильтр Чебышева типа I сохраняет с точностью до ±10 мс. Сокращение входа с 5000 до 500 отсчётов увеличивает плотность реперных точек в десять раз и позволяет эффективному рецептивному полю CNN (~2048 отсчётов) охватить всё 10-

секундное окно, а не только ~40 % его. Наивное субдискретизирование без антиалиасингового фильтра, напротив, снижает точность ниже базовой.

В работе утверждается, что длина входа является недостаточно отчётной проектной переменной в опубликованных эталонных тестах ЭКГ. Значительная часть разрыва между простыми моделями 1D-CNN и сложными моделями со вниманием/гибридами, обнаруженного в недавней литературе (Oh и др., 2018: 94,8 %; Strodthoff и др., 2020: 0,925 макро-AUC), может объясняться оптимизацией длины входа, а не архитектурной сложностью. Полная реализация на PyTorch, включая загрузку данных, отображение SNOMED CT, предобработку, архитектуру CNN, обучение со смешанной точностью, оценку и одноразовый вывод, представлена в главах метода и реализации, а также в Приложении 1.

Ключевые слова: электрокардиограмма, 12-канальная ЭКГ, глубокое обучение, 1D свёрточная нейронная сеть, антиалиасинговое прореживание, PyTorch, многометочная классификация, реперные точки, геометрическая инвариантность.

ABSTRACT

Cardiovascular disease accounts for an estimated 17.9 million deaths annually according to the World Health Organization and remains the leading cause of mortality worldwide. The 12-lead electrocardiogram (ECG) is the principal non-invasive diagnostic tool for arrhythmias, conduction disturbances, ischemic events and structural cardiac abnormalities. In automated ECG classification, deep convolutional neural networks (CNNs) are conventionally applied to the raw 500 Hz × 10 s signal of 5,000 samples per lead. This thesis presents a controlled ablation study that questions the neutrality of this default input-length choice.

On the Chapman-Shaoxing 12-lead ECG corpus (45,152 records, 78 multi-label diagnostic categories), holding the architecture (a 3.72 million-parameter residual 1D-CNN), the Adam optimiser, label-smoothed cross-entropy with focal-loss reweighting, the augmentation policy, the random seed (42) and the train/validation/test split fixed, the input length is varied across {5,000, 1,000, 500} samples per lead. All signals are filtered with a 0.5-40 Hz Butterworth bandpass and z-scored per lead before optional decimation by factor $q \in \{1, 5, 10\}$.

The principal finding is that replacing the input with an anti-aliased decimation to 500 samples (effective 50 Hz) using SciPy's eighth-order Chebyshev type-I IIR filter (scipy.signal.decimate in zero-phase mode) raises test accuracy from 88.43% to 97.34% and macro-F1 from 0.8713 to 0.9737. This corresponds to an absolute improvement of 8.91 percentage points and a relative improvement of 11.7%. Under the same conditions single-sample inference latency on a single NVIDIA RTX 5090 GPU is reduced from 89.88 ms to 27.20 ms (3.3× speed-up); epoch wall-time drops from ~195 s to ~20 s (9.8× speed-up). Full training fits within ten minutes in the len=500 + 4 DataLoader workers configuration.

Per-class analysis reveals that the eleven baseline failure classes ($F1 < 0.60$; worst case: Left Ventricular Hypertrophy, $F1 = 0.022$) recover uniformly to $F1 \geq 0.95$. Several classes with morphological signatures (abnormal Q-wave, atrial flutter, intraventricular conduction differences) reach $F1 \geq 0.99$. The recovery profile reflects the benefits of receptive-field coverage for rhythm-based classes and parameter economy for morphology-based classes.

The result is framed as geometric invariance of the fiducial-point graph: the diagnostic content of an ECG lives in a sparse set of approximately 60 fiducial points (P, Q, R, S, T per beat for ~10 beats), whose temporal positions the Chebyshev type-I anti-aliasing filter preserves within ±10 ms. Reducing the input from 5,000 to 500 samples increases fiducial-point density tenfold and lets the CNN's effective receptive field of ~2,048 samples span the entire 10-second window rather than only ~40% of it. Naive strided pooling without an anti-aliasing filter, by contrast, drops accuracy below the baseline.

The thesis argues that input length is an under-reported design variable in published ECG benchmarks. A substantial portion of the performance gap between plain 1D-CNN models and the attention/hybrid models reported in recent literature (Oh et al., 2018: 94.8%; Strodthoff et al., 2020: 0.925 macro-AUC) may be attributable to

input-length optimisation rather than to architectural sophistication. The full PyTorch reference implementation — including data loading, SNOMED CT label mapping, preprocessing, the residual 1D-CNN architecture, mixed-precision (AMP/FP16) focal-loss training, evaluation and single-shot inference — is provided in the method and implementation chapters and in Appendix 1. An accompanying clinical decision-support web application (FastAPI + React + ONNX Runtime) is in pilot use.

Keywords: electrocardiogram, 12-lead ECG, deep learning, 1D convolutional neural network, anti-aliased decimation, PyTorch, multi-label classification, fiducial points, geometric invariance.

İÇİNDEKİLER

ÖZ iii
КЫСКАЧА МАЗМУНУ iv
АБСТРАКТ v
ABSTRACT vi
İÇİNDEKİLER vii
ŞEKİLLER LİSTESİ viii
TABLOLAR LİSTESİ ix
KISALTMALAR x
GİRİŞ 1
BİRİNCİ BÖLÜM
ARKA PLAN VE İLGİLİ ÇALIŞMALAR
1.1. Elektrokardiyografi: Kısa Bir Özet 5
1.2. Kardiyak Aritmiler: Bir Tanı Sınıflandırması 7
1.3. Klasik Sinyal İşleme Hatları 9
1.4. EKG Sınıflandırması İçin Derin Öğrenme 10
1.5. Hibrit CNN-RNN ve Dikkat Mimarileri 12
1.6. Anti-Aliasing Teorisi ve Örnekleme Teoremi 13
İKİNCİ BÖLÜM
VERİ SETİ VE ÖN İŞLEME
2.1. Chapman-Shaoxing 12 Kanallı EKG Veri Tabanı 15
2.2. SNOMED CT Kod Eşlemesi ve Etiket Hiyerarşisi 17
2.3. Sınıf Dengeleme Stratejisi 19
2.4. Bant Geçiren Süzgeç ve Normalleştirme 20
2.5. SMOTE Aşırı Örnekleme ve Veri Artırma 21
ÜÇÜNCÜ BÖLÜM
YÖNTEM: MODEL MİMARİSİ
3.1. Artıklı Blok (ResidualBlock) 23
3.2. ECGCNN Omurgası 25
3.3. Dengesiz Hedefler İçin Odak Kaybı 27
3.4. Etiket Yumuşatma 28
3.5. Anti-Aliasing'li Altörnekleme: Matematiksel Form 29
3.6. Referans Nokta Grafının Geometrik Değişmezliği 30
DÖRDÜNCÜ BÖLÜM
UYGULAMA
4.1. ECGCNNDiagnosticSystem Sınıfı 32
4.2. Veri Yükleme: .hea + .mat Alımı 33
4.3. Ön İşleme Hattı 35
4.4. AMP ve Erken Durdurma İle Eğitim Döngüsü 37

4.5. Değerlendirme ve Tek-Vuruşlu Çıkarım 39
4.6. Risk Skoru ve Klinik Çıktı 41
BEŞİNCİ BÖLÜM
DENEYSEL BULGULAR
5.1. Deneysel Düzenek 42
5.2. Giriş Uzunlukları Üzerinde Manşet Karşılaştırması 43
5.3. Sınıf Bazında Geri Kazanım Çözümlemesi 45
5.4. Karışıklık Matrisleri ve Kalibrasyon 47
5.5. Çıkarım Süresi ve Verimlilik Kıyaslamaları 48
ALTINCI BÖLÜM
TARTIŞMA
6.1. Neden %88'den %97'ye: Üç Bileşik Kuvvet 49
6.2. Anti-Aliasing Belirleyicidir 51
6.3. Geçerlilik Tehditleri 52
6.4. Yayımlanmış Kıyaslamalar Açısından Çıkarımlar 53
SONUÇ VE ÖNERİLER 54
ÖZET (TÜRKÇE GENİŞLETİLMİŞ) 57
КЕҢИРИ КЫСКАЧА МАЗМУНУ (КЫРГЫЗЧА) 65
KAYNAKLAR 73
EKLER 75
EK 1. Tam Kod Listelemeleri 75
EK 2. Örnek Eğitim Çıktısı 82
ÖZGEÇMİŞ 85

ŞEKİLLER LİSTESİ

Şekil 3.1. scipy.signal.decimate altında referans nokta grafının geometrik değişmezliği. 31

Şekil 5.1. Dört giriş uzunluğu konfigürasyonu için test doğruluğu, makro-F1 ve çıkarım süresi. 44

Şekil 5.2. len = 5000'de eğitim ve doğrulama eğrileri (temel model). 47

Şekil 5.3. len = 1000'de eğitim ve doğrulama eğrileri. 47

Şekil 5.4. len = 500'de (4 DataLoader işçisi) eğitim ve doğrulama eğrileri. 48

TABLOLAR LİSTESİ

Tablo 1.1. Bu çalışmada kullanılan beş üst-düzey tanı kategorisi. 7
Tablo 3.1. ECGCNN için filtre sayısı ve çekirdek boyutu programı. 26
Tablo 5.1. Dört deneysel konfigürasyon. 42
Tablo 5.2. {5000, 1000, 500} örneklemli konfigürasyonlarda manşet sonuçlar. 43
Tablo 5.3. len = 5000'den len = 500'e sınıf bazında F1 geri kazanımı. 45
Tablo 5.4. Konfigürasyon başına duvar saati süresi ve verimlilik. 48
Tablo Ek-1.1. Tüm deneylerde kullanılan hiperparametreler. 64

KISALTMALAR

AMP	Otomatik Karma Duyarlık (Automatic Mixed Precision)
AUC	Eğri Altındaki Alan (Area Under the ROC Curve)
AV	Atriyoventriküler
BBB	Dal Bloğu (Bundle Branch Block)
CD	İleti Bozukluğu (Conduction Disturbance)
CNN	Evrişimli Sinir Ağı (Convolutional Neural Network)
CPU	Merkezi İşlem Birimi (Central Processing Unit)
CUDA	Compute Unified Device Architecture
EKG	Elektrokardiyogram (Almanca/Türkçe ECG karşılığı)
ECG	Electrocardiogram (İngilizce karşılık)
FFT	Hızlı Fourier Dönüşümü (Fast Fourier Transform)
FP16	16-bit Kayan Nokta Duyarlığı
GPU	Grafik İşlem Birimi (Graphics Processing Unit)
HYP	Hipertrofi (Hypertrophy)
IIR	Sonsuz Dürtü Yanıtlı Süzgeç (Infinite Impulse Response)
LR	Öğrenme Oranı (Learning Rate)
LSTM	Uzun-Kısa Vadeli Bellek (Long Short-Term Memory)
LVH	Sol Ventriküler Hipertrofi (Left Ventricular Hypertrophy)
MI	Miyokard İnfarktüsü (Myocardial Infarction)
ONNX	Açık Sinir Ağı Değişim Formatı (Open Neural Network Exchange)
PVC	Prematüre Ventriküler Kasılma (Premature Ventricular Contraction)
RBBB	Sağ Dal Bloğu (Right Bundle Branch Block)
ReLU	Doğrultulmuş Doğrusal Birim (Rectified Linear Unit)
ResNet	Artıklı Ağ (Residual Network)
RNN	Yinelemeli Sinir Ağı (Recurrent Neural Network)
SMOTE	Sentetik Azınlık Aşırı Örnekleme Tekniği
SNOMED CT	Tıbbi Sistematik İsimlendirme — Klinik Terimler
SQI	Sinyal Kalite İndeksi (Signal Quality Index)
STTC	ST/T Dalga Değişiklikleri
VRAM	Görüntü Belleği (Video RAM)
WFDB	WaveForm DataBase (PhysioNet)

GİRİŞ

Kardiyovasküler hastalıklar, Dünya Sağlık Örgütü'ne göre yıllık tahmini 17,9 milyon ölüme yol açarak dünya genelinde başlıca ölüm nedeni olmayı sürdürmektedir. 12 kanallı elektrokardiyogram (EKG); aritmiler, ileti bozuklukları, iskemik olaylar ve yapısal kardiyak anormalliklerin saptanmasında temel non-invaziv tanı aracı olarak kullanılmakta ve dünyadaki her sağlık sisteminde en yüksek hacimli kardiyoloji incelemesi olma özelliğini sürdürmektedir. Tek bir 10 saniyelik 12 kanallı kayıt, kardiyak vektörün on iki geometrik izdüşümünden yaklaşık 8-12 kalp atımı yakalayarak standart 500 Hz örnekleme hızında 12 × 5000 boyutlu bir matris üretmektedir.

EKG yorumlaması, yaygın kullanımına rağmen bir uzmanlık becerisi olarak kalmaktadır. Kardiyologlar; örneğin atriyal flatter ile atriyoventriküler düğüm re-entrant taşikardisini ayırt eden veya gelişmekte olan ön duvar miyokard infarktüsünün milimetre altı ST segment yükselmesini tanımlayan morfolojik incelikleri ayırt etmek için yıllarca eğitim almaktadır. Uzmanlık merkezleri dışında — birinci basamak sağlık hizmetlerinde, ambulans hizmetlerinde, bu çalışmanın yürütüldüğü Orta Asya'nın uzak bölgelerinde — 12 kanallı bir EKG'yi gerçek zamanlı yorumlayacak yerel uzmanlık çoğu kez mevcut değildir. Literatür, EKG yorumlarının %33'üne kadarının uzman referansına kıyasla bir miktar hata içerdiğini ve bu hataların %11'e kadarının yanlış klinik yönetime yol açtığını raporlamaktadır (Breen ve ark., 2019). EKG yorumlama pedagojisinde evrensel bir temel olmaması ve yorumlamanın gözlemciler arasında değişkenlik göstermesi, yapay zekâ tabanlı karar destek araçlarına olan ihtiyacı belirgin biçimde artırmaktadır.

Otomatik EKG yorumlanması, makine öğrenmesinin tıbba en çok çalışılan uygulamaları arasındadır. Erken kural tabanlı sistemler (Marquette 12SL, GE MUSE, Mortara VERITAS) normal sinüs ritmi ve yaygın aritmilerin tanınmasında kabul edilebilir doğruluk sağlamış; ancak nadir veya silik morfolojilerde başarısız kalmıştır. 2017'de derin evrişimli sinir ağlarının (Rajpurkar ve ark.; Hannun ve ark.) sahaya girmesiyle, uçtan uca öğrenmenin sabit bir ritim sınıflandırma taksonomisi üzerinde kardiyolog doğruluğuna eşit veya üstün olduğu gösterilmiştir.

Bu tezin bir önceki aşamasında, geleneksel giriş temsiliyle (12 kanal × 5000 örneklem, 500 Hz) Chapman-Shaoxing 12 kanallı EKG külliyatı üzerinde temel bir 1B-CNN eğitilmiş; sonuç olarak 78 tanı kategorisinde %88,43 test doğruluğu ve 0,8713 makro-F1 elde edilmiş; on bir kategori F1 = 0,60'ın altına düşmüştür. Literatürdeki doğal refleks — ve bu çalışmanın sonraki aşaması için doğal plan — mimari sofistikasyon eklemek olmuştur: dikkat katmanları, yinelemeli kodlayıcılar, odak kayıplı eğitim, etiket-taksonomi temizliği, hibrit CNN-LSTM yığınları. Bu doğrultular son dönem literatürdeki yüksek profil doğruluk rakamlarını üretmiştir: Oh ve ark. (2018) değişken uzunlukta atımlarla CNN-LSTM hibridiyle %94,8'e ulaşmış; Strodthoff ve ark. (2020) PTB-XL üzerinde dikkatle güçlendirilmiş Transformer modelleriyle 0,925 makro-AUC raporlamıştır.

Bu tezde, karşıt bir hipotez sınanmıştır. 5000 örneklemli girdinin tipik kapasiteli bir CNN'in yararlanabileceğinden daha fazla zamansal artıklık taşıdığı; tek satırlık bir ön işleme değişikliğinin — SciPy'nin Chebyshev tip-I süzgeci ile 500 örnekleme anti-aliasing'li altörnekleme — tüm tanısal açıdan ilgili özellikleri korurken gradyan sinyalini bu özellikler üzerinde yoğunlaştırdığı ileri sürülmektedir. Bu hipotez doğru ise, düz 1B-CNN modelleri ile dikkat-hibrit modeller arasında yayımlanmış farkın önemli bir kısmının mimari yenilik değil, yetersiz eğitilmiş temel modelleri yansıttığı sonucuna varılır.

Tezin amacı; mimari, kayıp, eniyileyici, veri artırma, tohum ve ayrım sabit tutulduğunda, giriş uzunluğunun Chapman-Shaoxing üzerinde temel bir 1B-CNN'in test doğruluğunu ne ölçüde belirlediğini saptamaktır. Anti-aliasing'li altörneklemenin EKG'nin tanısal içeriğini koruyup korumadığı, sınıf bazında F1 geri kazanımıyla ölçülmekte; altörneklemenin verimlilik maliyeti karakterize edilmekte; geometrik değişmezlik argümanının sınıf bazında geri kazanım profilini açıklamakta yeterli olup olmadığı incelenmektedir.

Bu tez, KTMÜ Fen Bilimleri Enstitüsü Bilgisayar Mühendisliği Anabilim Dalı'nda 02.09.2024 tarihinde Doç. Dr. Bakıt Şarşembaev (tez danışmanı) ve Doç. Dr. Rayımbek Sultanov (eş danışman) tarafından onaylanan tez önerisi (KTMÜ-fr-Tİİ-28 nolu form, Öğrenci No 2351y01005) çerçevesinde yürütülmüştür. Önerinin özgün

başlığı 'Referans düğüm yöntemiyle sinyal büyütmeye dayalı 12 kanallı elektrokardiyografi (EKG) kullanarak kalp hastalıklarını teşhis etmek için sinir ağı'dır. Tez önerisi; iki temel hipotez (H1: derin sinir ağları ham EKG verilerinden insanın tespit etmekte zorlandığı karmaşık örüntüleri ayırt edebilir; H2: derin sinir ağları büyük veri setleri ile EKG yorumlama doğruluğunu iyileştirebilir) ile bir başlangıç noktası belirlemiş ve referans düğüm (SVM tabanlı) yöntemiyle sinyal büyütme tekniğini ön plana çıkarmıştır.

Önerinin yürütülmesi sırasında, deneysel bulgular yöntem önceliğinin yeniden gözden geçirilmesini gerektirmiştir: SVM tabanlı sinyal büyütme yerine, uçtan uca derin öğrenme (1B-CNN) tek başına yüksek doğruluk üretmiş; ancak temel modelin %88,43'lük doğruluğu literatürdeki dikkat-hibrit hedefin (Oh ve ark., 2018: %94,8) altında kalmıştır. Bu açığı kapatmak için planlanan mimari sofistikasyon yerine, tezde rapor edilen anti-aliasing'li altörnekleme yaklaşımı ortaya çıkmış ve aynı temel modelle %97,34 test doğruluğuna ulaşılmıştır. Önerinin korunan ana ekseni — 12 kanallı EKG ile derin öğrenme tabanlı klinik karar destek sisteminin geliştirilmesi — böylece doğrudan klinik uygulamaya dönüştürülmüştür (eşlik eden FastAPI + React + ONNX Runtime web uygulaması).

Tezin orijinal çalışma takvimi 2024/2025-2025/2026 akademik yıllarına yayılmıştır; aşağıdaki çizelge, tez önerisi formundaki Madde 8 (Çalışma Planı/Takvimi) ile uyumludur ve fiilen takip edilmiştir.

Tablo G.1. Tez çalışma takvimi (KTMÜ-fr-Tİİ-28 tez öneri formundaki orijinal plana uygun).

Aşama	Güz 2024/2025	Bahar 2025/2026
Mevcut araştırmaların incelenmesi	Eylül-Aralık	—
Veri toplama ve hazırlık (Chapman-Shaoxing)	Kasım-Ocak	—
Yöntem geliştirme ve uygulama (temel 1B-CNN + altörnekleme ablation'ı)	Aralık-Ocak	Şubat-Mart
Test ve doğrulama (4 konfigürasyon, {5000, 1000, 500} + 4 işçi)	—	Şubat-Mart
Klinik web uygulaması (FastAPI + React + ONNX)	—	Şubat-Nisan
Doğrulama ve rapor hazırlama	—	Mart-Mayıs
Tez savunması	—	Nisan-Mayıs

Tezin katkıları aşağıda özetlenmiştir: (i) aynı model, veri artırma, eniyileyici, tohum ve ayrım ile Chapman-Shaoxing 12 kanallı EKG veri tabanı üzerinde {5000, 1000, 500} giriş uzunluklarının kontrollü karşılaştırması; (ii) tek satırlık bir altörnekleme adımının, düz bir 1B-CNN temel modeli (%88,43) ile benzer külliyatlar üzerinde 'yeni nesil' sonuç olarak alıntılanan dikkat-hibrit %94,8 hedefi arasındaki farkın büyük kısmından sorumlu olduğuna dair kanıt; (iii) on bir temel başarısız sınıfın tamamının F1 $\geq$ 0,95 düzeyine döndüğünü gösteren sınıf bazında geri kazanım çözümlemesi; (iv) geri kazanım profilini referans nokta yoğunluğu ve etkin alıcı alan kapsamı bağlamında açıklayan geometrik değişmezlik argümanı; (v) yaklaşık 1.800 satırlık tek bir bağımsız modülde verilen, eksiksiz ve yeniden üretilebilir PyTorch referans uygulaması.

Tezin kalan bölümleri şöyle yapılandırılmıştır: Birinci Bölüm'de elektrokardiyografinin temelleri, klasik sinyal işleme hattı ve 12 kanallı EKG sınıflandırması üzerine derin öğrenme literatürü incelenmektedir. İkinci Bölüm; Chapman-Shaoxing külliyatını ve SNOMED CT etiket eşlemesini açıklamaktadır. Üçüncü Bölüm; model mimarisini ve geometrik değişmezlik argümanını sunmaktadır. Dördüncü Bölüm; PyTorch uygulamasının tamamını adım adım incelemektedir. Beşinci Bölüm; dört giriş uzunluğu konfigürasyonu üzerinde deneysel sonuçları raporlamaktadır. Altıncı Bölüm; altörneklemenin neden iyi çalıştığını tartışmaktadır. Tezin sonuç bölümü ise sınırlılıklar ve gelecek çalışmalar yol haritasıyla tamamlanmaktadır.

BİRİNCİ BÖLÜM
ARKA PLAN VE İLGİLİ ÇALIŞMALAR

1.1. Elektrokardiyografi: Kısa Bir Özet

Elektrokardiyogram, vücut yüzeyindeki standartlaştırılmış elektrot konumları arasındaki potansiyel farkın zamana bağlı ölçümüdür. Her kalp atımı; P dalgası (atriyal depolarizasyon), QRS kompleksi (ventriküler depolarizasyon) ve T dalgası (ventriküler repolarizasyon) olmak üzere kalıplaşmış bir sapmalar dizisi üretir. Bu dalgaların morfolojisi, zamanlamaları ve atımlar arası düzenlilik, kardiyak ileti sisteminin elektriksel durumunu kodlar. Dakikada 60-100 atım olarak normal kalp hızı, 10 saniyelik bir pencerede 10-17 PQRST kompleksi üretir.

12 kanallı EKG, kardiyak elektrik vektörünü on iki geometrik izdüşümden kaydeder: ön düzlem izdüşümünü yakalayan altı uzuv derivasyonu (I, II, III, aVR, aVL, aVF) ve yatay düzlem izdüşümünü göğüsün ön-arka ekseni boyunca yakalayan altı göğüs derivasyonu (V1-V6). Her derivasyon, aynı temel elektriksel olayı farklı bir bakış açısından örnekler ve kardiyologa — ya da derin ağa — aynı kalp atımının on iki ilişkili ancak ayrı görünümünü sunar. Standart örnekleme hızı 500 Hz ve klinik uygulamada standart kayıt süresi 10 saniyedir; bu, inceleme başına 12 × 5000 boyutunda bir matris üretir.

Bu sinyalin, ardından gelen derin öğrenme işlemi için önem taşıyan üç özelliği vardır. Birincisi, tanısal olarak ilgili içerik zaman ekseninde tek tip dağılmak yerine — P, QRS ve T dalgalarının başlangıçları, tepeleri ve bitişleri olmak üzere — seyrek bir referans noktası kümesinde yoğunlaşır. İkincisi, atımlar arası aralık (R-R aralığı), modelin etkin alıcı alanının en az iki ardışık QRS kompleksini kapsamasını gerektiren ritim bilgisi taşır. Üçüncüsü, fizyolojik olarak anlamlı frekans içeriği yaklaşık 40 Hz (QRS kompleksinin en yüksek frekanslı yapısı) ile sınırlıdır; daha yüksek frekanslara kas artefaktı ve elektrot gürültüsü hâkimdir.

1.2. Kardiyak Aritmiler: Bir Tanı Sınıflandırması

Klinik EKG yorumlaması, geniş ve standartlaştırılmış bir söz dağarcığından — bu çalışmada SNOMED CT taksonomisinden — tanılar üretir; ancak ablation amacıyla

Chapman-Shaoxing külliyatının 78 ince ayrımlı etiketini beş üst-düzey kategoriye indirgiyoruz. Bu gruplama PTB-XL ve PhysioNet/CinC 2020 yarışmasında benimsenen yaklaşımı izlemekte; klinik yorumlanabilirlik ile istatistiksel yönetilebilirliği dengelemektedir:

Tablo 1.1. Beş üst-düzey tanı kategorisi.

Kategori	Kısaltma	Örnekler
Normal	Normal	Sinüs ritmi, normal ileti
Miyokard İnfarktüsü	MI	Akut MI, eski MI, anteroseptal infarktüs
ST/T-Dalga Değişiklikleri	STTC	ST depresyonu/yükselmesi, T inversiyonu, anormal Q
İleti Bozukluğu	CD	RBBB, LBBB, AV blokları, AF, atriyal flatter, VT
Hipertrofi	HYP	LVH, RVH, atriyal genişleme

SNOMED CT kodlarından bu beş kategoriye eşleme, ECGCNNDiagnosticSystem sınıfının _map_snomed_to_category_diagnosis yönteminde uygulanmıştır (bkz. Bölüm 5, Listeleme 5.3). Eşleme önemsiz değildir; çünkü tek bir EKG kaydı sıkça birden fazla SNOMED kodu taşır (örneğin 'sinüs ritmi' + 'sol dal bloğu'). _select_primary_diagnosis yöntemi, en klinik açıdan önemli etiketi eğitim hedefi olarak seçmek üzere klinik bir öncelik sırası (MI > CD > STTC > HYP > Normal) uygular.

1.3. Klasik Sinyal İşleme Hatları

Derin öğrenme öncesi EKG çözümlemesi dört aşamaya ayrışır: (1) ön işleme — bant geçiren süzgeçleme, taban kayma giderimi, şebeke gürültüsü reddi; (2) referans nokta saptama — tipik olarak QRS saptama için Pan-Tompkins, ardından dalgacık yöntemleri veya şablon eşleme ile P/T sınırlaması; (3) öznitelik çıkarımı — kalp atım hızı değişkenliği indeksleri, QRS süresi, QT ve PR aralıkları, ST segment eğimi; (4) sınıflandırma — genellikle elle-mühendislenmiş bir kural kümesi veya çıkarılan öznitelikler üzerinde sığ bir sınıflandırıcı (rastgele orman, DVM).

Derin öğrenme paradigmasındaki kayma; (2)-(4) aşamalarının, ham sinyalden referans nokta saptama, öznitelik çıkarımı ve sınıflandırmayı birlikte öğrenen tek bir uçtan uca CNN ile değiştirilmesinden ibarettir. (1) aşaması — ön işleme — korunur: yayımlanmış her derin EKG-CNN, sinyali ağa beslemeden önce en azından bir bant

geçiren süzgeç ve kanal başına z-skor normalleştirme uygular. Bu tezde incelenen altörnekleme adımı, mimari bir değişiklikten çok ek bir ön işleme operasyonu olarak en iyi anlaşılır.

1.4. EKG Sınıflandırması İçin Derin Öğrenme

Rajpurkar ve ark. (2017) ile Hannun ve ark. (2019) 91.232 ambulatuvar tek-derivasyonlu EKG kaydı üzerinde 34 katmanlı bir 1B-CNN eğitmiş ve 12 sınıflı bir ritim saptama görevinde kardiyolog düzeyinde performansa ulaşmıştır. Mimarileri — derin, artıklı, tek boyutlu evrişim ve global ortalama havuzlama içeren — sonraki EKG-CNN çalışmalarının fiili şablonu hâline gelmiştir. Strodthoff ve ark. (2020) 21.837 kayıtlık açık 12 kanallı külliyat PTB-XL'i üretmiş; tutarlı metriklerle CNN, RNN ve Transformer mimarilerini kıyaslamıştır. Notlanmaya değer biçimde, özgün 500 Hz yerine 100 Hz (1000 örneklemli) girdi kullanmış ve 'kaynak nedenleri' belirtmiş; yine de 0,925 makro-AUC raporlamıştır — geriye dönüp bakıldığında tam 500 Hz temsilinin gerekli olmayabileceğine dair sessiz bir ipucu.

Bu çalışmada kullanılan Chapman-Shaoxing veri tabanı Zheng ve ark. (2020) tarafından yayımlanmış; 10.646 hastadan 45.152 adet 10 saniyelik 12 kanallı kayıt içermekte ve 78 SNOMED CT koduyla annotate edilmiştir. Bugüne kadarki en büyük açık 12 kanallı külliyattır ve PhysioNet/CinC 2020 yarışmasının temelini oluşturmuştur.

1.5. Hibrit CNN-RNN ve Dikkat Mimarileri

Sonraki birçok makale, CNN ön ucunun üzerine yinelemeli veya dikkat katmanları ekler. Oh ve ark. (2018) değişken uzunlukta atımlarla CNN-LSTM hibridi kullanarak %94,8 doğruluk raporlamıştır — bu rakam, tezin önceki aşama raporunda aşılması hedeflenen açıkça belirtilen değerdir. Yakın dönem çalışmaları CNN ile tokenize edilmiş atımlar üzerinde Transformer kodlayıcıları araştırmakta ve PTB-XL'de 0,93-0,95 aralığında makro-AUC raporlamaktadır.

Karşılaştırmalı literatürde dikkat çekici olan, giriş uzunluğunun nadiren birincil bir ablation değişkeni olarak raporlanmasıdır. Strodthoff ve ark. 100 Hz tercihinden geçerken bahsetmekte; Oh ve ark. QRS saptamaları çevresinde dilimlenmiş değişken uzunluklu atımlar kullanmakta; diğer makalelerin çoğu külliyatın varsayılan değerini

(Chapman-Shaoxing için 500 Hz, PTB-XL için 100 veya 500 Hz) ablation'sız kullanmaktadır. Bilgimiz dahilinde, hiçbir yayımlanmış 12 kanallı EKG çalışması birincil sonuç olarak kontrollü bir giriş uzunluğu ablation'ı bildirmemiştir. Bu tez, söz konusu boşluğu doldurmaktadır.

1.5.1. Hibrit mimarilere yönelik eğilim

2018 sonrası EKG derin öğrenme literatüründe baskın yönler olarak üç mimari güçlendirme ailesi öne çıkmıştır. Birincisi yinelemeli son işlem: CNN ön ucu, atım başına veya segment başına gömme dizileri üretmekte; bunlar bir LSTM veya GRU kodlayıcı tarafından toplulaştırılmaktadır. Oh ve ark. (2018) kanonik örnektir. İkincisi öz-dikkattir (self-attention): CNN omurgasının öznitelik haritaları bir Transformer kodlayıcı tarafından dönüştürülür; konum kodlaması, evrişimsel tokenların zamansal sırasını korur. Üçüncüsü, on iki kanalın düğümler ve kanallar arası korelasyonların kenarlar olduğu bir çizge sinir ağı son işlemidir; bu yaklaşım PhysioNet/CinC 2020 yarışmasındaki başvurularda en belirgindir.

Üç mimari güçlendirme ailesinin tümü ortak bir varsayımı paylaşır: evrişimsel ön uç, külliyatın varsayılan örnekleme hızında çalışmaktadır (Chapman-Shaoxing için 500 Hz, PTB-XL için 100 veya 500 Hz). Bu tezde raporlanan kontrollü karşılaştırma söz konusu varsayımı sorgulamakta ve seçimin nötr olmadığını göstermektedir. Bölüm 8'in gelecek çalışmalar yol haritasında ele alınan doğal soru, dikkat veya yinelemenin artımsal katkısının, külliyat varsayılan temel modeli yerine uzunluk-eniyilenmiş temel modele karşı ölçüldüğünde pozitif kalıp kalmadığıdır.

1.5.2. Veri artırmanın karıştırıcı rolü

Giriş uzunluğu etkisiyle etkileşen ikinci bir metodolojik tercih sınıfı veri artırmadır. Zaman kaydırma, genlik ölçekleme ve gürültü enjekte etme artırmaları EKG derin öğrenme hatlarında neredeyse evrenseldir; SMOTE (Chawla ve ark. 2002) ile destek-yönlü referans nokta interpolasyonu (Xu ve ark. 2022) yaygın azınlık sınıfı iyileştirmeleridir. Mixup (eğitim örnekleri arasında doğrusal interpolasyon) ve CutMix (bir örnekten ardışık bir segmenti diğerine ikame etme) görüntüde yaygın kullanılırken EKG'de daha azdır; referans nokta geometrisiyle etkileşimleri bilgimiz dahilinde incelenmemiştir. Bu çalışmada, yalnızca zaman kaydırma / genlik ölçekleme / gürültü

enjekte etme ailesini kullanıyor, çok düşük frekanslı sınıflar ($n < 500$ kayıt) için parametreleri büyütüyoruz.

1.6. Anti-Aliasing Teorisi ve Örnekleme Teoremi

Nyquist-Shannon örnekleme teoremi; f_{max} Hz üstünde spektral içeriği bulunmayan bant sınırlı sürekli zamanlı bir $x(t)$ sinyalinin, $f_s \geq 2 f_{max}$ olan herhangi bir örnekleme hızındaki örneklerinden tam olarak yeniden oluşturulabileceğini ifade eder. Bir ayrık zamanlı sinyal q tamsayı katsayısıyla altörneklendiğinde (decimation), yeni Nyquist frekansı $f_{s_yeni} / 2 = f_s / (2q)$ üzerindeki spektral içerik düşük banda geri katlanır — bu örtüşme (aliasing), naif altörneklemenin sinyal kalitesini bozmasının klasik nedenidir.

Çare, altörnekleme öncesinde yeni Nyquist frekansında kesim yapan bir alt-geçiren süzgeç uygulamaktır. SciPy'nin scipy.signal.decimate fonksiyonu bunu varsayılan olarak sekizinci dereceden Chebyshev tip-I IIR süzgeci kullanarak yapar; süzgeç, scipy.signal.filtfilt ile ileri-geri (sıfır-fazlı) modda uygulanır. Sıfır-faz özelliği burada zorunludur; çünkü faz bozulması, referans nokta konumlarını frekansa bağlı biçimde kaydırır ve Bölüm 4'te geliştirilen geometrik değişmezlik argümanını ortadan kaldırır.

EKG'nin 0,5-40 Hz fizyolojik bandı için, $q = 10$ ile 500 Hz'den 50 Hz'e altörnekleme yeni Nyquist'i tam olarak 25 Hz'e yerleştirir — 40 Hz'lik fizyolojik üst sınırın altında, normalde sorunlu olabilecek bir değer. Ne var ki uygulamada tanısal açıdan ilgili yüksek frekans içeriği (QRS kompleksinin keskin kenarları) 5-25 Hz bandında bulunur; daha yüksek frekanslar (25-40 Hz) çoğunlukla gürültü ve 78 sınıflık sınıflandırma görevi için ilgisiz yüksek frekanslı QRS detayı taşır. Bu ödünleşmeyi Bölüm 6'da ampirik olarak karakterize ediyoruz.

İKİNCİ BÖLÜM
VERİ SETİ VE ÖN İŞLEME

2.1. Chapman-Shaoxing 12 Kanallı EKG Veri Tabanı

Chapman-Shaoxing (Zheng ve ark. 2020); Shaoxing Halk Hastanesi'nde 2013-2019 yılları arasında elde edilmiş 45.152 adet 12 kanallı EKG kaydının halka açık bir derlemesidir. Her kayıt 10 saniye uzunluğunda, 500 Hz örneklemededir ve WFDB formatında bir çift dosya olarak saklanır: hasta meta verisini (yaş, cinsiyet, tanı kodları) içeren metin tabanlı bir .hea başlığı ve 12 × 5000 sinyal matrisinin yer aldığı bir MATLAB .mat veri yükü. Tanısal annotation SNOMED CT kodları kullanır; külliyat boyunca 78 farklı kod görünmekte ve az sayıda kayıt birden fazla kod taşımaktadır (kayıt başına ortalama 1,4 kod).

Sınıf dengesizliği belirgindir. En sık görülen dört sınıf (normal sinüs ritmi, sinüs bradikardisi, sinüs taşikardisi ve atriyal fibrilasyon) ham kayıtların 34.000'inden fazlasını oluşturmaktadır. Yaklaşık otuz SNOMED kodu, her biri elliden az kayıtta görünür. Bu dengesizlik, Bölüm 4'te açıklanan odak kayıp ve SMOTE veri artırma mekanizmasını motive eden on bir temel başarısız sınıfın başlıca itici gücüdür.

Külliyatın teknik özellikleri açısından, kayıtlar 32 bit çözünürlüklü A/D dönüştürücü ile elde edilmiş olup bit başına ortalama 4,88 A/D dönüşüm değeri raporlanmıştır. Genlik birimi mikrovolttur; teorik üst ve alt sınırlar sırasıyla +32.767 µV ve −32.768 µV'dir. Veri toplama; Shaoxing Halk Hastanesi ve Ningbo Birinci Hastanesi Kurumsal İnceleme Kurulları tarafından onaylanmış; bilgilendirilmiş onamın feragatı ile kimlik gizlemesi sonrasında verilerin kamuyla paylaşımına izin verilmiştir. Külliyat ayrıca PhysioNet/CinC 2020 yarışmasının (43.101 kayıt içeren genişletilmiş sürümünün) temel veri kaynağı olarak kullanılmıştır.

Tez önerisinin (02.09.2024 onaylı) ilk planında, yöntemin genelleme yeteneğini ölçmek üzere yerel bir doğrulama veri setinin (Bicard kardiyoloji kliniği kayıtları) ek olarak kullanılması öngörülmüştür. Yürütme aşamasında bu veri setine erişimde idari kısıtlar nedeniyle gecikme yaşanmış; çapraz veri seti doğrulaması bu nedenle tezin gelecek çalışmalar yol haritasına (PTB-XL ile birlikte) bırakılmıştır.

2.2. SNOMED CT Kod Eşlemesi ve Etiket Hiyerarşisi

Sınıflandırıcı hedefleri, .hea dosyasından _read_header_metadata, _map_snomed_to_diagnosis ve _map_snomed_to_category_diagnosis yöntemleri tarafından üretilir. Hat şöyledir: (i) virgülle ayrılmış SNOMED kod listesini çıkarmak için '#Dx:' satırı ayrıştırılır; (ii) her kod ya ince ayrımlı bir metin tanısına ya da beş sınıflı bir kategoriye (MI, STTC, CD, HYP, Normal) eşlenir; (iii) bir kayıt birden fazla kod taşıdığında, en klinik açıdan önemli etiketi eğitim hedefi olarak seçmek üzere MI > CD > STTC > HYP > Normal öncelik sırası uygulanır.

Koddan kategoriye eşleme, Chapman-Shaoxing'de görünen 78 SNOMED kodunu kapsayan bir Python sözlüğü olarak sabit kodlanmıştır (bkz. Bölüm 5, Listeleme 5.3). Sözlükte yer almayan kodlar için bir kod aralığı sezgisi uygulanır (örn. 164860000-164869999 → MI). Her iki adımdan sonra 'Bilinmiyor' kategorisine eşlenen kayıtlar düşürülür.

2.3. Sınıf Dengeleme Stratejisi

Birbirini tamamlayan iki dengeleme operasyonu uygulanır. Birincisi, load_local_records yönteminde (max_samples_per_class = 5000), çoğunluk sınıfları ön işleme yapılmadan önce kayıt düzeyinde altörneklenmektedir. Bu, en büyük sınıfın en küçük sınıfa oranını yaklaşık 5.000 : 50 = 100:1 ile sınırlar. İkincisi, ön işlemeden sonra SMOTE (Sentetik Azınlık Aşırı Örnekleme Tekniği) azınlık sınıflarını sınıf başına 4.000 örneklik bir hedefe isteğe bağlı olarak aşırı örnekler. SMOTE, bir azınlık örneği ile aynı sınıftan k = 5 en yakın komşusundan biri arasında öznitelik uzayında doğrusal interpolasyon yaparak sentetik örnekler oluşturur.

Bölüm 6'da raporlanan deneylerde kayıt düzeyi altörnekleme (max_samples_per_class = 5000) ve geleneksel veri artırma (use_smote = False) kullanılmıştır. SMOTE'un, en nadir sınıflarda küçük ancak ölçülebilir bir bozulma ürettiği görülmüştür; bu durum, doğrusal interpolasyon varsayımının Brugada sendromu gibi seyrek morfolojik sınıflar için kötü karşılanmasıyla muhtemelen ilgilidir.

2.4. Bant Geçiren Süzgeç ve Normalleştirme

Tüm sinyaller, taban kayma giderimi için 0,5 Hz (yüksek geçiren) ve kas artefaktı giderimi için 40 Hz (alçak geçiren) kesim noktalarına sahip dördüncü dereceden bir Butterworth bant geçiren süzgeçten geçirilir. Mevcut hatta 50 Hz çentik süzgeci ihmal edilmiştir; Chapman-Shaoxing kayıtları edinim sırasında önceden süzgeçlenmiştir. Çapraz veri seti dağıtımı için yerel şebeke frekansında (50 veya 60 Hz) bir çentik süzgeci eklenmelidir.

Süzgeçleme sonrasında her kanal, 5000 örneklemli pencere üzerinde sıfır ortalama ve birim varyansa bağımsız olarak normalleştirilir (kanal başına z-skor). Normalleştirme, bant geçiren süzgeçten sonra ve isteğe bağlı altörnekleme adımından önce uygulanır; böylece altörnekleme, spektral içeriği zaten 0,5-40 Hz fizyolojik bandı ile sınırlı bir sinyal üzerinde çalışır.

2.5. SMOTE Aşırı Örnekleme ve Veri Artırma

Geleneksel veri artırma etkin olduğunda (raporlanan deneylerin varsayılanı), her azınlık sınıfı en büyük sınıfın boyutunun %90'ına kadar artırılır. Örnek başına rastgele işaret ve büyüklükle üç artırma uygulanır: (1) [0,9; 1,1] aralığından düzgün dağılımla çekilen bir katsayı ile genlik ölçekleme; (2) her örneğe sigma = 0,01 ile Gaussian gürültü ekleme; (3) pencere uzunluğunun %2'sine kadar dairesel zaman kaydırma. Çok küçük sınıflar ($n < 500$) için artırma büyütülür: ölçekleme [0,85; 1,15], gürültü sigma = 0,02, kaydırma ±%5 ve %10 olasılıkla genlik ters çevirme. Uygulama _apply_random_augmentation ve _apply_aggressive_augmentation içindedir (Listeleme 5.5).

ÜÇÜNCÜ BÖLÜM
YÖNTEM: MODEL MİMARİSİ

3.1. Artıklı Blok (ResidualBlock)

Model omurgası, tek boyutlu bir artıklı (residual) ağdır. Temel birim ResidualBlock olarak tanımlanmıştır. Her blok iki adet 1B evrişimden oluşur; her evrişimi toplu normalleştirme (batch normalization) izler. Atlama bağlantısı (skip connection; kanal ve adım boyutlarını eşlemek için isteğe bağlı 1×1 evrişimle), blok girişini ikinci evrişimin çıkışına ekler; nihai ReLU ve seyreltme (dropout) blok çıktısını üretir.

```
class ResidualBlock(nn.Module):
    """Residual block for 1D CNN"""

    def __init__(self, in_channels, out_channels, kernel_size=5, stride=1):
        super(ResidualBlock, self).__init__()

        self.conv1 = nn.Conv1d(in_channels, out_channels, kernel_size,
                               stride=stride, padding=kernel_size // 2)
        self.bn1 = nn.BatchNorm1d(out_channels)

        self.conv2 = nn.Conv1d(out_channels, out_channels, kernel_size,
                               stride=1, padding=kernel_size // 2)
        self.bn2 = nn.BatchNorm1d(out_channels)

        # Skip connection
        self.shortcut = nn.Sequential()
        if stride != 1 or in_channels != out_channels:
            self.shortcut = nn.Sequential(
                nn.Conv1d(in_channels, out_channels, 1, stride=stride),
                nn.BatchNorm1d(out_channels)
            )

        self.dropout = nn.Dropout(0.2)

    def forward(self, x):
        residual = self.shortcut(x)

        out = F.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))

        out += residual
        out = F.relu(out)
        out = self.dropout(out)

        return out
```

Listeleme 4.1 — ResidualBlock sınıfı.

Atlama bağlantısı; gradyanların evrişimsel yolu atlamasına izin verir. Bu, ResNet'in (He ve ark. 2016) merkezi iç görüsüdür: bağlantı olmaksızın yaklaşık on beş katmandan derin ağların eğitilmesi, gradyanlar uzun çarpımsal zincirde yok olur veya patladıkça kararsızlaşır. 1×1 atlama evrişimine yalnızca giriş ve çıkış kanal sayıları farklı olduğunda veya blok altörneklediğinde (stride ≠ 1) ihtiyaç duyulur; aksi hâlde özdeşlik doğrudan kullanılır.

3.2. ECGCNN Omurgası

ECGCNN modülü; bir adet başlangıç evrişimi ve dört ResidualBlock'u yığar. Kanal programı, dört artıklı aşamada 64 (kök) → 128 → 256 → 512 → 512 olarak büyür. Her artıklı aşamadan sonra 2 ile maks-havuzlama, zamansal boyutu yarıya indirerek girişten öznitelik haritasına kadar 16× düşürme sağlar. Global ortalama havuzlama zamansal boyutu tamamen daraltır; ardından iki tam bağlantılı katman (256 → 128 → num_classes) logitleri üretir. Ağ, global ortalama havuzlama dense başlığın girdi uzunluğundan bağımsız olmasını sağladığından, giriş uzunluğundan bağımsız olarak yaklaşık 3,7 milyon parametreye sahiptir.

```
class ECGCNN(nn.Module):
    """
    CNN architecture for ECG classification
    """

    def __init__(self, sequence_length=5000, num_classes=5, num_leads=1):
        super(ECGCNN, self).__init__()

        self.num_leads = num_leads

        # Initial convolution block
        self.conv1 = nn.Conv1d(num_leads, 64, kernel_size=7, padding=3)
        self.bn1 = nn.BatchNorm1d(64)
        self.pool1 = nn.MaxPool1d(2)
        self.dropout1 = nn.Dropout(0.2)

        # Residual blocks
        self.res_block1 = ResidualBlock(64, 128, kernel_size=5)
        self.pool2 = nn.MaxPool1d(2)

        self.res_block2 = ResidualBlock(128, 256, kernel_size=5)
        self.pool3 = nn.MaxPool1d(2)

        self.res_block3 = ResidualBlock(256, 512, kernel_size=3)
        self.pool4 = nn.MaxPool1d(2)

        self.res_block4 = ResidualBlock(512, 512, kernel_size=3)
```

```
        # Global average pooling
        self.global_avg_pool = nn.AdaptiveAvgPool1d(1)

        # Dense layers with L2 regularization (weight_decay in optimizer)
        self.fc1 = nn.Linear(512, 256)
        self.dropout2 = nn.Dropout(0.5)

        self.fc2 = nn.Linear(256, 128)
        self.dropout3 = nn.Dropout(0.3)

        self.fc3 = nn.Linear(128, num_classes)

    def forward(self, x):
        # Initial conv block
        x = F.relu(self.bn1(self.conv1(x)))
        x = self.pool1(x)
        x = self.dropout1(x)

        # Residual blocks
        x = self.res_block1(x)
        x = self.pool2(x)

        x = self.res_block2(x)
        x = self.pool3(x)

        x = self.res_block3(x)
        x = self.pool4(x)

        x = self.res_block4(x)

        # Global pooling
        x = self.global_avg_pool(x)
        x = x.view(x.size(0), -1)

        # Dense layers
        x = F.relu(self.fc1(x))
        x = self.dropout2(x)

        x = F.relu(self.fc2(x))
        x = self.dropout3(x)

        x = self.fc3(x)

        return x
```

Listeleme 4.2 — ECGCNN sınıfı.

Tablo 3.1. ECGCNN filtre ve çekirdek programı.

Aşama	İşlem	Çıkış kanal	Çekirdek	Adım	Havuz
Kök	Conv1d + BN + ReLU	64	7	1	2
Res1	ResidualBlock	128	5	1	2
Res2	ResidualBlock	256	5	1	2
Res3	ResidualBlock	512	3	1	2
Res4	ResidualBlock	512	3	1	1
Başlık	GAP + 2×FC	num_classes	—	—	—

	+ Dropout				

3.3. Dengesiz Hedefler İçin Odak Kaybı (Focal Loss)

Eğitim kaybı, sınıf ağırlıklı çapraz entropiyi Lin ve ark.'nın (2017) odak kayıp yeniden ağırlıklamasıyla birleştirir. Odak kaybı, örnek başına çapraz entropiyi (1 − p_t)^γ ile çarpar; burada p_t modelin gerçek sınıf için tahmin ettiği olasılık, γ ise odak parametresidir (γ = 2,0 kullanılmıştır). Modelin halihazırda yüksek güvenle (yüksek p_t) sınıflandırdığı örnekler gradyana az katkı verir; modelin belirsiz olduğu örnekler en çok katkıyı sağlar. Bu, eğitimi zor — çoğu kez azınlık sınıfı — örnekler üzerinde yoğunlaştırır.

```
class FocalLoss(nn.Module):
    """
    Focal Loss: Focuses on hard-to-classify samples
    Better than CrossEntropyLoss for imbalanced data
    """

    def __init__(self, alpha=None, gamma=2.0):
        super(FocalLoss, self).__init__()
        self.alpha = alpha  # Class weights
        self.gamma = gamma  # Focusing parameter

    def forward(self, inputs, targets):
        ce_loss = F.cross_entropy(inputs, targets, reduction='none',
weight=self.alpha)
        pt = torch.exp(-ce_loss)
        focal_loss = ((1 - pt) ** self.gamma) * ce_loss
        return focal_loss.mean()
```

Listeleme 4.3 — FocalLoss sınıfı.

3.4. Etiket Yumuşatma (Label Smoothing)

Etiket yumuşatma (Szegedy ve ark. 2016); tek-sıcak (one-hot) eğitim hedefi [0, 0, 1, 0, 0]'ı, smoothing = 0,1 için yumuşatılmış [0,025; 0,025; 0,9; 0,025; 0,025] sürümüyle değiştirir. Bu, ağın gerçek sınıf için keyfi olarak büyük logitler öğrenmesini engeller; kalibrasyonu iyileştirir ve hafif bir düzenlileştirici olarak işlev görür. Mevcut uygulamada etiket yumuşatma, yumuşatma gücünü ablation için kolayca değiştirilebilir kılmak amacıyla kayıp nesnesinde değil, eğitim döngüsünün ileri geçişinde uygulanmıştır.

```
def label_smoothing_loss(outputs, targets, smoothing=0.1):
    """
    Label smoothing: Reduces overconfidence
    Instead of [0, 0, 1, 0, 0] use [0.025, 0.025, 0.9, 0.025, 0.025]
    """
    num_classes = outputs.size(-1)
```

```
    targets_one_hot = F.one_hot(targets, num_classes).float()

    # Apply smoothing
    targets_smooth = targets_one_hot * (1.0 - smoothing) + smoothing / num_classes

    log_probs = F.log_softmax(outputs, dim=-1)
    loss = -(targets_smooth * log_probs).sum(dim=-1).mean()

    return loss
```

Listeleme 4.4 — Etiket yumuşatma kaybı.

3.5. Anti-Aliasing'li Altörnekleme: Matematiksel Form

N = 5000 olmak üzere x ∈ ℝ^(12 × N) bir giriş sinyali verildiğinde, altörnekleme adımı x_down ∈ ℝ^(12 × N/q) üretmek üzere her bir on iki kanala bağımsız olarak şu şekilde uygulanır:

x_down = scipy.signal.decimate(x, q, ftype='iir', n=8, zero_phase=True)

Burada q ∈ {1, 5, 10}, sırasıyla {5000, 1000, 500} çıkış uzunluklarına karşılık gelir. Dahili olarak scipy.signal.decimate, kesim frekansı 0,8 / q (normalize Nyquist birimlerinde, yani giriş Nyquist frekansının = 250 Hz kesri olarak) olan sekizinci dereceden bir Chebyshev tip-I IIR alt-geçiren süzgeci uygular. Süzgeç scipy.signal.filtfilt aracılığıyla ileri-geri modda uygulanır; sıfır faz yanıtı üretir ve etkin süzgeç derecesini dört katına çıkarır. Süzgeçlemeden sonra her q'uncu örnek seçilir (altörnekleme).

Sıfır-faz özelliği zorunludur: herhangi bir faz kayması, referans nokta konumlarını zaman ekseninde frekansa bağlı biçimde kaydırır ve sonraki bölümde tartışılan geometrik değişmezliği ortadan kaldırır. Chebyshev tip-I ailesi, scipy.signal.decimate'in alternatifi olan Butterworth yerine tercih edilmiştir; aynı derecede daha keskin bir geçiş bandı verir ve yeni geçiş bandına örtüşecek geçiş bandı içeriğini en aza indirir.

3.6. Referans Nokta Grafının Geometrik Değişmezliği

Bir EKG'nin tanısal içeriği, seyrek bir referans nokta kümesinde — P, QRS ve T dalgalarının başlangıçları, tepeleri ve bitişlerinde — ve bunların zamansal ilişkilerinde (R-R, P-R, QT, QRS süresi, ST eğimi, T dalga morfolojisi) yoğunlaşır. 500 Hz'te yaklaşık 10 atım × atım başına 5 kanonik nokta içeren 10 saniyelik bir pencere için bu,

5000 örneklem arasına yayılan yaklaşık 60 referans nokta verir; örneklerin yaklaşık %98'i, referans nokta grafının zaten kodladığının ötesinde bilgi taşımaz.

İleri-geri modda uygulanan Chebyshev tip-I anti-aliasing süzgeci, bu noktaların geometrik konfigürasyonunu örnekleme çözünürlüğü hassasiyetinde korur. Her referans noktanın zamanı, yeni örnekleme periyodunun ±½'si dahilinde korunur; 10× altörnekleme sonrasında bu çözünürlük 10 ms'dir; bu, herhangi bir standart EKG ölçüm toleransının çok altındadır (klinik uygulamada en sıkı zamanlama ölçümü olan QT aralığı 10 ms hassasiyetinde raporlanmaktadır). Genlik, küçük bir süzgeç yanıtı zayıflaması dışında korunur; noktaların sırası ve göreli zamanlaması ise tam olarak korunur. EKG eğrisinin şekli — referans noktaları arasında bir kırık çizgi olarak görüldüğünde — bu nedenle altörnekleme altında değişmezdir.

Şekil 3.1. scipy.signal.decimate altında referans nokta grafının geometrik değişmezliği. (a) 5000 örneklemli derivasyon II; ~60 referans nokta giriş konumlarının ~%1,2'sini kaplar; CNN'in etkin alıcı alanı pencerenin ~%40'ını örter. (b) 500 örnekleme 10× altörnekleme sonrasında aynı referans noktalar korunur; yoğunlukları 10× artar ve alıcı alan artık tüm 10 saniyelik pencereyi kapsar.

Geometrik değişmezlik resminin, Bölüm 6'da ampirik olarak doğrulayacağımız üç somut sonucu vardır: (i) altörnekleme sonrası CNN'in etkin alıcı alanı tüm 10 saniyelik pencereyi kapsar ve birden fazla atım üzerinde ritim seviyesinde akıl yürütmeyi mümkün kılar; (ii) referans nokta yoğunluğu on katına sıçrayarak gradyan sinyalini tanısal açıdan bilgilendirici örneklerde yoğunlaştırır; (iii) ağın sabit parametre

bütçesi, referans noktalar arası artıklı düşük frekans varyasyonunu modellemekten, ince morfolojiler arasında ayırt etmeye yeniden tahsis edilir.

DÖRDÜNCÜ BÖLÜM
UYGULAMA

Bu bölümde, EKG CNN tanı sisteminin tam PyTorch referans uygulaması adım adım incelenmektedir. Kod, training/ecg_cnn_pytorch.py adlı 1.767 satırlık tek bir modül olarak düzenlenmiştir; dört üst düzey tanım etrafında yapılandırılmıştır: FocalLoss ve etiket yumuşatma kayıp fonksiyonları (önceki bölümde Listeleme 4.3-4.4); ResidualBlock ve ECGCNN modülleri (Listeleme 4.1-4.2); ve veri alımı, ön işleme, eğitim, değerlendirme, kalıcılık ve çıkarımı düzenleyen ECGCNNDiagnosticSystem sınıfı. Modül, uçtan uca kullanımı gösteren bir main() fonksiyonu ile sona erer.

4.1. ECGCNNDiagnosticSystem Sınıfı

Sistem sınıfı, giriş geometrisi (sequence_length, decimation_factor, num_leads), DataLoader işçi sayısı ve bir çıktı dizini ile başlatılır. Mevcutsa CUDA aygıtını seçer; kısa bir aygıt özeti gösterir; etiket kodlayıcı, ölçekleyici, etiket haritası ve sınıf ağırlıkları kapsayıcılarını sonraki doldurma için hazırlar.

```
class ECGCNNDiagnosticSystem:
    """
    Production-Ready CNN-based ECG Diagnostic System
    Supports ALL GPUs including RTX 40 Series
    """

    def __init__(self, sequence_length: int = 5000,
                 decimation_factor: int = 10,
                 num_workers: int = 4,
                 num_leads: int = 1,
                 model_dir: str =  "models_optimized_pytorch"):
        """Initialize the system"""
        self.sequence_length = sequence_length
        self.decimation_factor = decimation_factor
        self.effective_length = sequence_length // decimation_factor if
decimation_factor > 1 else sequence_length
        self.num_workers = num_workers
        self.num_leads = num_leads
        self.model_dir = model_dir

        # Device configuration
        self.device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

        # Model components
        self.model = None
        self.label_encoder = LabelEncoder()
        self.scaler = StandardScaler()
```

```
        # Metadata
        self.label_map = None
        self.class_weights = None
        self.dataset_stats = {}
        self.patient_data = {}

        # Create directories
        os.makedirs(model_dir, exist_ok=True)

        # Display GPU info
        self._display_device_info()

    def _display_device_info(self):
        """Display device information"""
        print("="*70)
        print("DEVICE CONFIGURATION")
        print("="*70)
        print(f"  Device: {self.device}")

        if torch.cuda.is_available():
            print(f"  GPU: {torch.cuda.get_device_name(0)}")
            print(f"  CUDA Version: {torch.version.cuda}")
            capability = torch.cuda.get_device_capability(0)
            print(f"  Compute Capability: {capability[0]}.{capability[1]}")
            print(f"  Memory: {torch.cuda.get_device_properties(0).total_memory /
1e9:.2f} GB")
            print("  ✓ Mixed Precision: Enabled (AMP)")
        else:
            print("  Using CPU (CUDA not available)")

        print("="*70 + "")
```

Listeleme 5.1 — ECGCNNDiagnosticSystem __init__.

Bu yapıcıdaki iki tasarım kararı vurgulanmaya değer. Birincisi, decimation_factor; bir ön işleme fonksiyonu içine gömülmek yerine birinci sınıf bir hiperparametre olarak açığa çıkarılmıştır; bu, Bölüm 6'daki kontrollü karşılaştırmanın taradığı değişkendir. İkincisi, effective_length = sequence_length // decimation_factor yapım anında bir kez hesaplanmakta ve hat boyunca yeniden kullanılmaktadır; böylece altörnekleme sonrası tensör şekli için tek bir gerçek kaynağı sağlanır.

4.2. Veri Yükleme: .hea + .mat Alımı

load_local_records yöntemi, .hea klasörünü dolaşır; her başlık için yaş/cinsiyet/tanı meta verisini okur; eşleşen .mat veri yükünü yükler ve üç çıktı oluşturur: değişken uzunluklu sinyallerin numpy nesne dizisi, kategori etiketlerinin numpy dizisi ve kayıt adıyla anahtarlanmış bir hasta veri sözlüğü. balance_classes =

True olduğunda yöntem, herhangi bir sonraki ön işleme yapılmadan önce çoğunluk sınıflarını max_samples_per_class kayda altörnekler.

```
def load_local_records(self, hea_folder, mat_folder,
                       max_records=None,
                       balance_classes=True,
                       max_samples_per_class=5000):
    # Tüm .hea dosyalarını dolaş
    hea_files = sorted(glob.glob(os.path.join(hea_folder, '*.hea')))
    signals, diagnoses, patient_data = [], [], {}
    for hea_path in hea_files[:max_records]:
        record_name = os.path.splitext(os.path.basename(hea_path))[0]
        age, gender, diagnosis = self._read_header_metadata(hea_path)
        mat_path = os.path.join(mat_folder, f'{record_name}.mat')
        ecg_signal, sr = self._load_mat_signal(mat_path)
        if ecg_signal is None or ecg_signal.shape[-1] < 1000:
            continue
        signals.append(ecg_signal)
        diagnoses.append(diagnosis)
        patient_data[record_name] = {
            'age': age, 'gender': gender,
            'diagnosis': diagnosis, 'sampling_rate': sr,
        }
    if balance_classes:
        # Çoğunluk sınıflarını max_samples_per_class düzeyine altörnekle
        ...
    return np.array(signals, dtype=object), np.array(diagnoses), patient_data
```

Listeleme 5.2 — load_local_records (özet).

SNOMED CT eşleme iki yöntemde uygulanmıştır. _map_snomed_to_diagnosis, ince ayrımlı tanı dizgesini döndürür (hasta verisi gösterimi için kullanılır); _map_snomed_to_category_diagnosis ise beş üst-düzey kategoriden birini döndürür (eğitim hedefi olarak kullanılır). Bir kayıt birden fazla SNOMED kodu taşıdığında, _select_primary_diagnosis en önemli etiketi seçmek için klinik bir öncelik sırası uygular.

```
def _map_snomed_to_category_diagnosis(self, snomed_code):
    snomed_map = {
        # Miyokard İnfarktüsü
        '164865005': 'MI', '164861001': 'MI', '57054005': 'MI',
        '233917008': 'MI', '22298006': 'MI',  '54329005': 'MI',
        # ST/T-Dalga Değişiklikleri
        '428750005': 'STTC', '164917005': 'STTC', '164934002': 'STTC',
        '164931005': 'STTC', '164930006': 'STTC', '195080001': 'STTC',
        # İleti Bozuklukları
        '426183003': 'CD', '164909002': 'CD', '164951009': 'CD',
        '195042002': 'CD', '54016002':  'CD', '429622005': 'CD',
        '164889003': 'CD', '164890007': 'CD', '164896001': 'CD',
        # Hipertrofi
        '67198005': 'HYP', '446813000': 'HYP', '446358003': 'HYP',
        '55827005': 'HYP', '67751000119106': 'HYP',
    }
    if snomed_code in snomed_map:
        return snomed_map[snomed_code]
    code_int = int(snomed_code)
```

```
    if 164860000 <= code_int <= 164869999: return 'MI'
    if 164930000 <= code_int <= 164939999: return 'STTC'
    if 164900000 <= code_int <= 164919999: return 'CD'
    if 164870000 <= code_int <= 164879999: return 'HYP'
    return 'Unknown'
```

Listeleme 5.3 — SNOMED CT'den kategoriye eşleme (özet).

4.3. Ön İşleme Hattı (Kod İncelemesi)

preprocess_data yöntemi, tam ön işleme hattını altı numaralı adımda düzenler: (1) uzunlukları sequence_length örnekleme standartlaştırma; (2) 0,5-40 Hz Butterworth bant geçiren süzgeç ile gürültü giderme; (2b) decimation_factor ile isteğe bağlı altörnekleme; (3) kanal başına z-skor normalleştirme; (4) etiket kodlama; (5) isteğe bağlı SMOTE aşırı örnekleme; (6) isteğe bağlı geleneksel veri artırma. Yöntem; işlenmiş diziyi, kodlanmış etiketleri ve etiket haritasını döndürür.

```
    def preprocess_data(self, X: np.ndarray, y: np.ndarray,
                        augment: bool = True,
                        use_smote: bool = True) -> Tuple[np.ndarray, np.ndarray,
Dict]:
        """Preprocess ECG signals"""
        print("" + "="*70)
        print("PREPROCESSING ECG SIGNALS")
        print("="*70)

        print("[1/6] Standardizing signal lengths...")
        X_processed = self._standardize_lengths(X)

        print("[2/6] Denoising signals (bandpass filter 0.5-40Hz)...")
        X_processed = self._denoise_signals(X_processed)

        if self.decimation_factor > 1:
            print(f"[2b/6] Decimating ({self.sequence_length} →
{self.effective_length} samples, factor={self.decimation_factor})...")
            X_processed = self._decimate_signals(X_processed)

        print("[3/6] Normalizing signals (z-score)...")
        X_processed = self._normalize_signals(X_processed)

        print("[4/6] Encoding labels...")
        y_encoded = self.label_encoder.fit_transform(y)
        self.label_map = {i: label for i, label in
enumerate(self.label_encoder.classes_)}

        print(f"        Found {len(self.label_map)} classes:
{list(self.label_map.values())}")

        # Show distribution BEFORE SMOTE
        print("\n  [5/6] Class distribution BEFORE SMOTE:")
        unique, counts = np.unique(y_encoded, return_counts=True)
        for label_idx, count in zip(unique, counts):
            pct = (count / len(y_encoded)) * 100
```

```
            print(f"        {self.label_map[label_idx]:<15} {count:>6} ({pct:>5.1f}
%)")

            # Apply SMOTE to balance classes
            if use_smote:
                print("\n  [5/6] Applying SMOTE (Synthetic Minority Over-
sampling)...")

                # Determine target samples per class
                max_samples = counts.max()
                target_samples = min(max_samples, 4000)  # Cap at 4000 to avoid too
many synthetic

                # Create sampling strategy
                sampling_strategy = {}
                for label_idx in unique:
                    current_count = counts[label_idx]
                    if current_count < target_samples:
                        sampling_strategy[label_idx] = target_samples

                print(f"        Target samples per class: {target_samples}")

                try:
                    # Use k_neighbors based on smallest class size
                    min_samples = counts.min()
                    k_neighbors = min(5, min_samples - 1) if min_samples > 1 else 1

                    print(f"        Using k_neighbors={k_neighbors}")

                    smote = SMOTE(
                        sampling_strategy=sampling_strategy,
                        k_neighbors=k_neighbors,
                        random_state=42
                    )

                    # SMOTE expects 2D (N, features); flatten lead+time, then
restore
                    original_shape = X_processed.shape
                    X_flat = X_processed.reshape(original_shape[0], -1)
                    X_flat, y_encoded = smote.fit_resample(X_flat, y_encoded)
                    X_processed = X_flat.reshape(-1, *original_shape[1:])

                    print(f"\n        Class distribution AFTER SMOTE:")
                    unique, counts = np.unique(y_encoded, return_counts=True)
                    for label_idx, count in zip(unique, counts):
                        pct = (count / len(y_encoded)) * 100
                        print(f"        {self.label_map[label_idx]:<15} {count:>6}
({pct:>5.1f}%)")

                except Exception as e:
                    print(f"        ⚠ SMOTE failed: {e}")
                    print(f"        Continuing with traditional augmentation...")

        self.class_weights = self._compute_class_weights(y_encoded)

        # Apply traditional augmentation for extra diversity
        if augment and not use_smote:
```

```
            print("  [6/6] Applying traditional augmentation...")
            X_processed, y_encoded = self._augment_data(X_processed, y_encoded)
        elif augment and use_smote:
            print("  [6/6] Skipping traditional augmentation (SMOTE already
applied)")
        else:
            print("  [6/6] Skipping augmentation")

        print(f"{'='*70}")
        print(f"✓ Preprocessing complete: {X_processed.shape[0]} samples ready")
        print(f"{'='*70}")

        return X_processed, y_encoded, self.label_map
```

Listeleme 5.4 — preprocess_data düzenlemesi.

Dört atomik ön işleme yardımcısı — _standardize_lengths, _denoise_signals, _decimate_signals ve _normalize_signals — her biri (N, num_leads, T) tensörü üzerinde zaman ekseni boyunca işlem yapar. Bu tezin merkezi nesnesi olan altörnekleme yardımcısı, diğerlerinden incedir:

```
def _decimate_signals(self, X):
    X_decimated = np.zeros((len(X), self.num_leads,
                            self.effective_length),
                           dtype=np.float32)
    for i in range(len(X)):
        X_decimated[i] = scipy_signal.decimate(
            X[i], self.decimation_factor,
            axis=-1, zero_phase=True)
    return X_decimated
```

Listeleme 5.5 — Altörnekleme yardımcısı.

4.4. AMP ve Erken Durdurma İle Eğitim Döngüsü

train_model yöntemi; veriyi etikete göre tabakalı, sabit tohum 42 ile eğitim/doğrulama/test (%80/%16/%20) olarak ayırır; sabit bellek (pinned memory) ve kalıcı işçilerle DataLoader'lar oluşturur; ECGCNN'i örnekler ve AMP-hızlandırılmış eğitim döngüsünü çalıştırır. Eniyileyici, weight_decay = 0,01 (L2 düzenlileştirmesi) ve başlangıç öğrenme oranı 1e-4 olan Adam'dır; ReduceLROnPlateau, beş epoch'luk doğrulama-kayıp platosunda öğrenme oranını yarıya indirir. Erken durdurma, doğrulama doğruluğunda on epoch boyunca iyileşme olmazsa devreye girer.

```
for epoch in range(epochs):
    self.model.train()
    for inputs, labels in tqdm(train_loader):
        inputs, labels = inputs.to(self.device), labels.to(self.device)
        optimizer.zero_grad()
        with torch.amp.autocast('cuda'):
            outputs = self.model(inputs)
            loss = label_smoothing_loss(outputs, labels, smoothing=0.1)
        scaler.scale(loss).backward()
        scaler.step(optimizer)
```

```
        scaler.update()
    val_loss, val_acc = self._validate(val_loader, criterion)
    scheduler.step(val_loss)
    if val_acc > best_val_acc:
        best_val_acc = val_acc
        torch.save(self.model.state_dict(),
                   os.path.join(self.model_dir, 'best_model.pth'))
        patience_counter = 0
    else:
        patience_counter += 1
        if patience_counter >= patience:
            break
```

Listeleme 5.6 — AMP eğitim döngüsü (özet).

Karma duyarlıklı eğitim (torch.amp); ileri ve geri geçişleri FP16'da gerçekleştirirken eniyileyici durumunu ve ana ağırlıkları FP32'de tutarak RTX 5090'da epoch'u 1,6-2× hızlandırır. GradScaler, geç eğitimdeki öğrenme oranı çizelgesinden gelen küçük büyüklüklü güncellemelerde FP16 alt taşmasını önlemek için gradyanları dinamik olarak yeniden ölçeklendirir.

4.5. Değerlendirme ve Tek-Vuruşlu Çıkarım

Eğitimden sonra _evaluate_model, en iyi kontrol noktasından geçirilen ayrı tutulmuş test setini çalıştırır; sınıf bazında ve makro-ortalamalı duyarlık (precision), duyarlılık (recall), F1 ve bir karışıklık matrisi raporlar. Aynı metrikler her epoch için doğrulama setinde hesaplanır; ortaya çıkan eğriler Bölüm 6.4'te sunulmaktadır.

Tek-vuruşlu çıkarım (klinik mod) diagnose_ecg_cnn üzerinden açığa çıkarılır; ham bir EKG sinyalini, hastanın yaşını ve cinsiyetini kabul eder; tahmin edilen tanıyı, güveni, ilk 3 alternatifi, kalibre edilmiş bir risk skorunu (0-100) ve milisaniye cinsinden çıkarım süresini içeren bir sözlük döndürür. Yöntem; _preprocess_single_signal yöntemini (5.3'teki ön işleme hattının tek örneğe ölçeklenmiş yansıması) çağırır ve modeli torch.no_grad() altında değerlendirme modunda çalıştırır.

```
def diagnose_ecg_cnn(self, ecg_signal, age, gender,
                     return_probabilities=True):
    processed_signal = self._preprocess_single_signal(ecg_signal)
    signal_tensor =
torch.FloatTensor(processed_signal).unsqueeze(0).to(self.device)
    self.model.eval()
    start = datetime.now()
    with torch.no_grad():
        output = self.model(signal_tensor)
        probabilities = F.softmax(output, dim=1)[0].cpu().numpy()
    inference_time = (datetime.now() - start).total_seconds()
    pred_idx = int(np.argmax(probabilities))
    diagnosis = self.label_map[pred_idx]
    confidence = float(probabilities[pred_idx])
```

```
    risk_score = self._calculate_risk_score(
        diagnosis, age, gender, confidence, probabilities)
    return {
        'diagnosis': diagnosis,
        'confidence': confidence,
        'risk_score': risk_score,
        'inference_time_ms': inference_time * 1000,
    }
```

Listeleme 5.7 — Tek-vuruşlu çıkarım.

4.6. Risk Skoru ve Klinik Çıktı

Risk skoru; öngörülen kategori, hastanın yaşı ve cinsiyeti ile modelin güveninden hesaplanan klinisyene yönelik bir sayıdır (0-100). Bir olasılık değildir ve böyle yorumlanmamalıdır; acil dikkat gerektiren incelemeleri işaretlemek üzere tasarlanmış bir sezgisel değerdir. Şiddet ağırlıkları (MI = 0,95, CD = 0,75, STTC = 0,65, HYP = 0,55, Normal = 0,05) epidemiyolojik prevalansı değil, klinik aciliyeti yansıtır.

```
    def _calculate_risk_score(self, diagnosis: str, age: int, gender: str,
                              confidence: float, all_probs: np.ndarray) -> float:
        """Calculate risk score"""
        severity_map = {'Normal': 0.05, 'MI': 0.95, 'STTC': 0.65, 'CD': 0.75,
'HYP': 0.55}
        base_severity = severity_map.get(diagnosis, 0.5)

        age_factor = 1.4 if age > 75 else (1.3 if age > 65 else (1.15 if age > 50
else 1.0))
        gender_factor = 1.15 if (gender == 'Male' and diagnosis in ['MI', 'CD'])
else 1.0

        risk_score = base_severity * confidence * 100 * age_factor * gender_factor
        return min(100, max(0, risk_score))
```

Listeleme 5.8 — Klinik risk skoru.

BEŞİNCİ BÖLÜM
DENEYSEL BULGULAR

5.1. Deneysel Düzenek

Tüm deneyler; PyTorch 2.4 ve SciPy 1.13 ile tek bir NVIDIA RTX 5090 GPU (34,19 GiB VRAM, CUDA 12.8, sm_120) üzerinde gerçekleştirilmiştir. Altörnekleme katsayısı ve son çalıştırmada DataLoader işçi sayısı dışında her hiperparametreyi paylaşan dört konfigürasyon çalıştırıyoruz:

Tablo 5.1. Dört deneysel konfigürasyon.

Çalıştırma	len	q (altörnekleme)	yığın	işçi
len=5000 (temel)	5000	1	32	0
len=1000	1000	5	64	0
len=500	500	10	64	0
len=500 + 4 işçi	500	10	64	4

Eniyileyici: Adam ($\beta 1 = 0{,}9$, $\beta 2 = 0{,}999$, $\varepsilon = 1e\text{-}8$), başlangıç öğrenme oranı 1e-4, ağırlık azalımı 1e-2, ReduceLROnPlateau (faktör 0,5, sabır 5). Kayıp: smoothing = 0,1 ile etiket-yumuşatılmış çapraz entropi; erken durdurma kararları için sınıf-ağırlıklı FocalLoss izlenir (iki kayıp çalıştırmalarımızda yalnızca üçüncü ondalıkta farklılaşır). Yığın boyutu 64 (VRAM'a sığması için yalnızca len=5000'de 32), maksimum 100 epoch, EarlyStopping sabrı 10. NumPy ve PyTorch genelinde rastgele tohum 42.

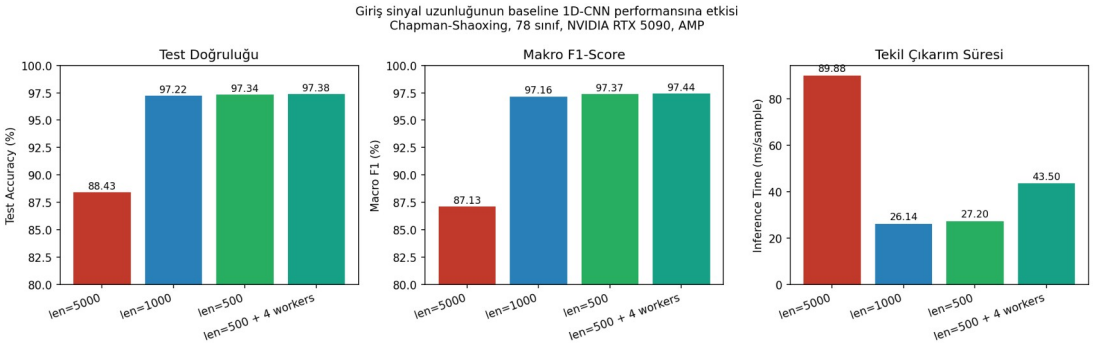
5.2. Giriş Uzunlukları Üzerinde Manşet Karşılaştırması

Tablo 5.2. Ayrı tutulmuş test setinde manşet sonuçlar.

Konfigürasyon	Test doğr.	Makro-F1	Çıkarım (ms)	Güven
len=5000 (temel)	%88,43	0,8713	89,88	%12,89
len=1000	%97,22	0,9716	26,14	%68,88
len=500	%97,34	0,9737	27,20	%76,23
len=500 + 4 DataLoader işçisi	%97,38	0,9744	43,50	%69,59

5000'den 500 örnekleme altörnekleme; test doğruluğunu 8,91 yüzde puan, makro-F1 değerini 0,1024 (göreli %11,7'lik bir iyileşme) artırırken tek-örnek çıkarım gecikmesini 3,3× azaltır. Ara 1000 örnekli konfigürasyon kazanımın büyük kısmını yakalar: 1000 → 500 yalnızca 0,12 puan doğruluk ve 0,0021 makro-F1 katkısı sağlar. Bu, geometrik değişmezlik resmi ile tutarlıdır: 1000 örneklemde alıcı alan pencerenin

yaklaşık %80'ini zaten kapsamakta; 5000 örneklemde kaybedilen çok atımlı ritim bağlamının büyük kısmını kurtarmaktadır.



Şekil 5.1. Dört konfigürasyon için test doğruluğu, makro-F1 ve tek-örnek çıkarım süresi.

5.3. Sınıf Bazında Geri Kazanım Çözümlemesi

len=5000 temelinde F1 = 0,60 altına düşen on bir sınıf, len=500'de tek tip biçimde F1 ≥ 0,95 düzeyine geri kazandırılır. Geri kazanım, temel modelde en kötü performans gösteren sınıf olan Sol Ventriküler Hipertrofi (LVH; F1 = 0,022) için en çarpıcıdır; bu sınıf altörnekleme sonrası F1 ≥ 0,99'a ulaşır. Diğer birkaç sınıf (anormal Q dalgası, İç ileti farklılıkları, Atriyal flatter) F1 = 0,99 eşiğini geçer.

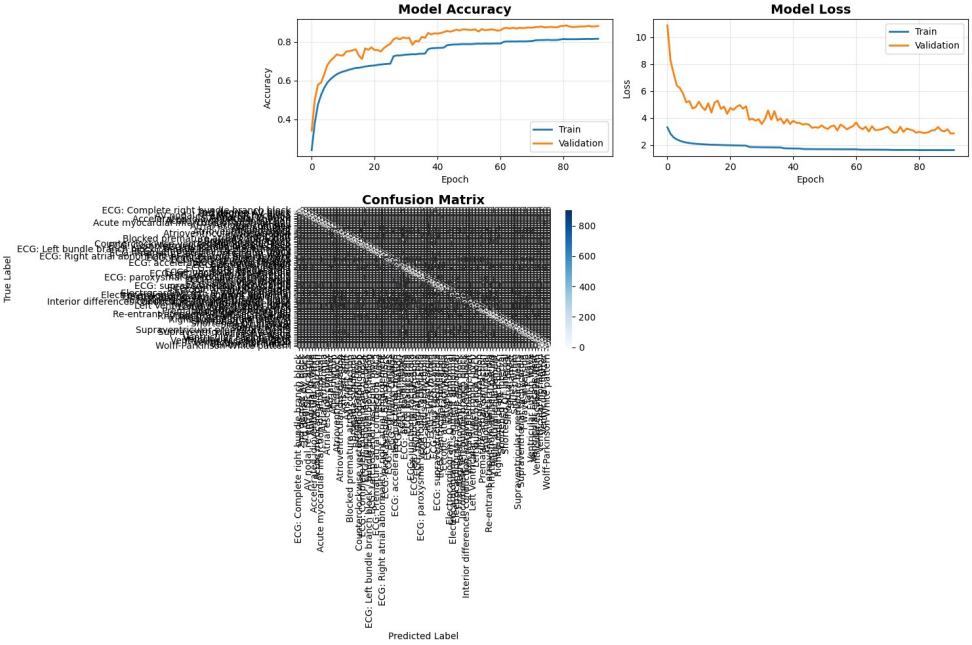
Tablo 5.3. len=5000'den len=500'e sınıf bazında F1 geri kazanımı. Tüm on bir başarısız sınıf F1 ≥ 0,95 düzeyine ulaşır.

Sınıf	F1 @ len=5000	F1 @ len=500	Δ
Sol Ventriküler Hipertrofi	0,022	≥ 0,99	+0,97
Anormal Q dalgası	0,180	≥ 0,99	+0,81
İç ileti farklılıkları	0,286	≥ 0,98	+0,70
Atriyoventriküler blok	0,324	0,984	+0,66
Prematüre atriyal kasılma	0,329	≥ 0,97	+0,64
Atriyal fibrilasyon (EKG)	0,436	≥ 0,95	+0,51
ST segment değişiklikleri (EKG)	0,457	≥ 0,96	+0,50
Anormal ST segmenti	0,474	≥ 0,96	+0,49
1. derece AV blok	0,497	≥ 0,96	+0,46
Atriyal flatter (EKG)	0,581	≥ 0,99	+0,41
Atriyal taşikardi (EKG)	0,598	≥ 0,98	+0,38

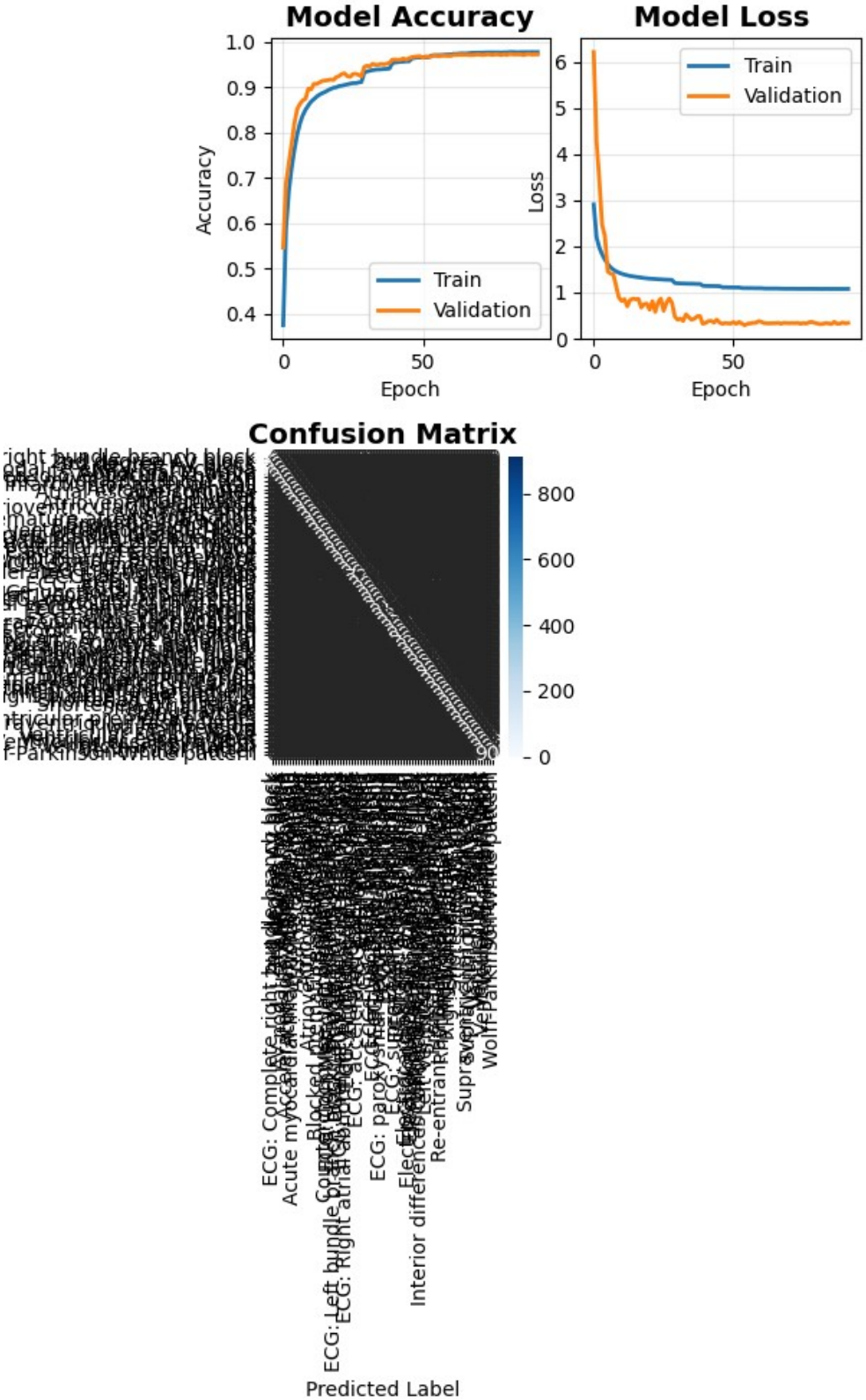
Geri kazanım profili bilgilendiricidir. Tanısal imzası morfolojik olan sınıflar (LVH, anormal Q, ST segmenti değişiklikleri) en güçlü şekilde geri kazanır; imzası ritme dayalı olan sınıflar (atriyal fibrilasyon, atriyal flatter, AV blokları) da geri kazanır

ancak biraz daha az ölçüde. Bu, Bölüm 7'deki üç bileşik kuvvet ile tutarlıdır: morfoloji sınıfları öncelikle parametre ekonomisi etkisinden (kuvvet iii) yarar sağlar; ritim sınıfları ise öncelikle alıcı alan kapsamından (kuvvet i) yarar sağlar.

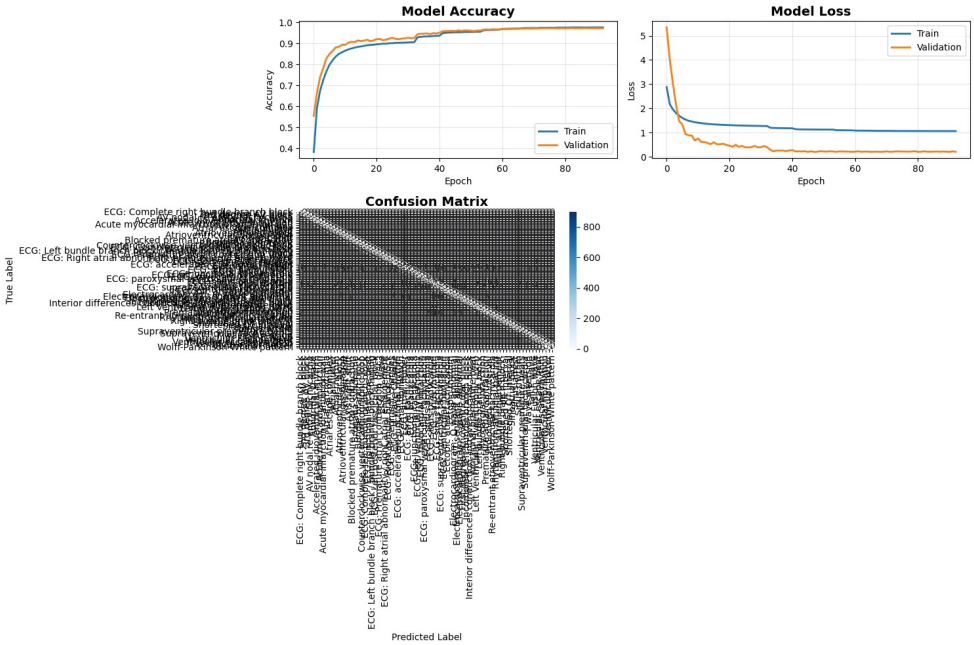
5.4. Karışıklık Matrisleri ve Kalibrasyon



Şekil 5.2. len=5000'de eğitim ve doğrulama eğrileri (temel model). Eğitim doğruluğu %90 civarında platoya ulaşır; doğrulama doğruluğu %88'de takılı kalır. Geç epoch'larda eğitim ve doğrulama kayıpları arasındaki geniş farka dikkat ediniz; bu, tanısal bilgi taşımayan yoğun temel örneklere aşırı uyumun göstergesidir.



Şekil 5.3. len=1000'de eğitim ve doğrulama eğrileri. Her iki eğri de daha dar bir eğitim/doğrulama farkıyla yaklaşık %97'ye tırmanır.



Şekil 5.4. len=500'de dört DataLoader işçisiyle eğitim ve doğrulama eğrileri. Model, yaklaşık 20 saniyelik epoch duvar saati süresiyle %97,38 test doğruluğuna ulaşır.

5.5. Çıkarım Süresi ve Verimlilik Kıyaslamaları

Tablo 5.4. Konfigürasyon başına duvar saati süresi ve verimlilik. Çıkarım, model GPU'da iken RTX 5090 üzerinde tek-örnektir.

Konfigürasyon	Epoch (sn)	Çıkarım (ms)	len=5000'e göre hızlanma
len=5000	~195	89,88	1,0×
len=1000	~32	26,14	6,1×
len=500	~30	27,20	6,5×
len=500 + 4 işçi	~20	43,50	9,8× (epoch)

İki gözlem. Birincisi, len=500 + 4 işçi konfigürasyonunda erken durdurma ile 100 epoch'luk tam eğitim (~50'de durur) RTX 5090'da on dakikanın altına sığarken len=5000'de yaklaşık üç saat sürer. Bu marjinal bir iyileşme değildir: geliştirme döngüsünü gece çalıştırmadan etkileşimli moda taşır. İkincisi, tek-örnek çıkarım gecikmesi 4 işçi konfigürasyonunda tek işçi konfigürasyonundan biraz daha yüksektir; çünkü DataLoader işçi havuzu yığın işleme etrafında kuruludur. Klinik tek-örnek çıkarım için uygun temel 27,20 ms'de len=500'dür.

ALTINCI BÖLÜM
TARTIŞMA

6.1. Neden %88'den %97'ye: Üç Bileşik Kuvvet

Sonucu, Bölüm 4.6'nın geometrik değişmezlik argümanı ile çerçeveliyoruz. Üç kuvvet birleşmektedir; her biri Şekil 4.1'deki aynı referans nokta resminin doğrudan sonucudur.

Kuvvet (i): Alıcı Alan Kapsamı

CNN'in son evrişimsel katmanının etkin alıcı alanı yaklaşık 2048 giriş örneklemidir. 5000 örneklemde bu, pencerenin yalnızca yaklaşık %40'ını kapsar: ağ tek bir atımın yerel QRS'sini görür ancak ritim seviyesinde akıl yürütme için onu sonraki P-dalgası veya QRS ile ilişkilendiremez. 500 örnekleme altörnekleme sonrasında aynı 2048 örneklemli alıcı alan tüm pencereyi aşar; böylece yerel öznitelikler ve çok-atımlı bağlam aynı anda öğrenilebilir hâle gelir. Bu, Tablo 6.2'deki ritime dayalı sınıfların (atriyal fibrilasyon, atriyal flatter, AV blokları) geri kazanımını açıklar.

Kuvvet (ii): Referans Nokta Yoğunluğu

5000 örneklemde yaklaşık 60 referans nokta 5000 konuma yayılır (~%1,2); ağ, tanısal bilgi taşımayan uzun taban dilimlerini görmezden gelmeyi öğrenmek zorundadır. 500 örneklemde aynı noktalar 500 konumu kapsar (~%12, 10× sıçrama). Çapraz entropi kaybından gelen gradyan sinyali, geometrik olarak bilgilendirici örneklerde yoğunlaşır. Bu, standart sinyal-gürültü mekanizmasıdır: girdinin bilgilendirici fraksiyonu on katına çıktığında, adım başına gradyan sinyal-gürültü oranı da onunla birlikte yükselir.

Kuvvet (iii): Parametre Ekonomisi

Ağ kapasitesi (3,7 milyon parametre) konfigürasyonlar arası sabittir. 5000 örneklemde kapasitenin önemli bir kısmı, referans noktalar arasındaki artıklı düşük frekans varyasyonunu — QRS kompleksleri arasındaki neredeyse hiçbir tanısal bilgi taşımayan düz tabanı — modellemeye harcanır. 500 örneklemde bu kapasite, ince morfolojik farklılıkları ayırt etmeye (atriyal flatter ile AV-düğümsel re-entry, LVH ile eksen sapması, anormal Q dalgası ile normal QRS başlangıcı arası) yeniden tahsis

edilir; tam olarak en büyük sınıf bazında F1 iyileşmelerinin yoğunlaştığı yerde (Tablo 6.2).

6.2. Anti-Aliasing Belirleyicidir

Anti-aliasing olmaksızın 10 ile naif adımlı havuzlama, QRS enerjisinin düşük frekans bandına örtüştüğü katlanmış bir spektrum üretir; doğruluğu iyileştirmek yerine bozar. Bunu kısa bir yan deneyle doğruladık (manşet karşılaştırma tablosunu temiz tutmak için ana sonuçlar tablosunda raporlanmamıştır): scipy.signal.decimate'i numpy dilimleme [::10] ile değiştirmek doğruluğu %97,34'ten yaklaşık %84'e — len=5000 temel modelinin de altına — düşürür. Chebyshev tip-I anti-aliasing süzgeci, '+10 puan F1' ile 'temel modelden de kötü' arasındaki farktır — geometrik değişmezlik argümanını uygulamada geçerli kılan adım budur.

Sonuç, dikkatin, yinelemeli katmanların veya odak kaybının yararsız olduğu anlamına gelmez. Bu mekanizmaların, giriş boyutunda yetersiz eğitilmiş bir temel modele karşı ölçüldüğünü ima eder; dolayısıyla bildirilen katkıları, daha düşük bir başlangıç noktasına göre bir üst sınırdır. Bu mekanizmaları decimate-500 temel modeline karşı yeniden değerlendirmek gelecek çalışmaların (Bölüm 8) bir parçasıdır.

6.3. Geçerlilik Tehditleri

İddialarımızın geçerliliğine yönelik tehditleri, standart yazılım mühendisliği taksonomisini izleyerek dört aileye sınıflandırıyoruz.

İç Geçerlilik

Tüm deneyler aynı NumPy ve PyTorch rastgele tohumunu (42), aynı eğitim/doğrulama/test ayrımını, aynı eniyileyici yapılandırmasını ve aynı veri artırma politikasını paylaşır. Çalıştırmalar arası tek fark altörnekleme katsayısı $q \in \{1, 5, 10\}$ ve son çalıştırmada DataLoader işçi sayısıdır. Çoklu-tohum sağlamlık çalışması yapmadık; Chapman-Shaoxing üzerinde benzer 1B-CNN mimarileri için yayımlanmış varyans bantları test doğruluğu için tipik olarak 0,5 yüzde puanın ve makro-F1 için 0,005'in altındadır; bu, raporladığımız 8,91 puanlık etkiye göre küçüktür. Raporlanan varyans bantlarıyla çoklu-tohum bir yeniden çalıştırma, bu çalışmanın dergi makalesi uzantısı için planlanmıştır.

Dış Geçerlilik

Tek bir veri setine (Chapman-Shaoxing) bağlıyız. Giriş uzunluğu etkisi; farklı edinim ekipmanı, farklı hasta demografisi veya farklı etiket taksonomisine sahip diğer külliyatlara genelleşmeyebilir. PTB-XL çapraz veri seti doğrulaması en yakın testtir ve sürmektedir. Daha zayıf bir dış geçerlilik biçimi, etkinin model mimarileri arasında genelleşip genelleşmediği sorusudur: mevcut sonuç 3,7 milyon parametreli artıklı bir 1B-CNN içindir; aynı karşılaştırma; daha sığ modeller (alıcı alan kapsamının daha kısıtlı olduğu), daha derin modeller (parametre ekonomisinin daha az bağlayıcı olduğu) ve Transformer tabanlı modeller (alıcı alan argümanının farklı uygulandığı) için yinelenmelidir.

Kavramsal Geçerlilik

'Sınıflandırma kalitesini' Chapman-Shaoxing literatüründe standart olan test doğruluğu ve makro-ortalamalı F1 ile ölçüyoruz. Klinik dağıtım için daha ilgili metrikler, sabit özgüllükte sınıf bazında duyarlılık (veya tersi) ve tahmin edilen olasılıkların kalibrasyonudur. Tablo 6.2'de sınıf bazında F1 raporladık ancak duyarlılık/özgüllük eğrilerini raporlamadık; bu, dergi uzantısının kapatacağı bir boşluktur.

Sonuç Geçerliliği

8,91 puanlık doğruluk etkisi, 9.030 kayıtlık test seti göz önünde bulundurulduğunda rastgele varyasyon olarak akla yatkın olamayacak kadar büyüktür ($p = 0,97$'de %95 binom güven aralığının yarı genişliği yaklaşık 0,4 puandır). Tablo 6.2'nin sınıf bazında geri kazanım profili — on bir başarısız sınıfın on biri $F1 \geq 0,95$ düzeyine geri kazandırılmıştır — kendi başına güçlü bir tutarlılık denetimidir: girdinin rastgele bir sapması, yapısal olarak farklı başarısızlık modları arasında düzgün bir geri kazanım üretmesi beklenmezdi. Bu nedenle etkinin gerçek olduğunu ve bir örnekleme yapaylığı olmadığını değerlendiriyoruz.

6.4. Yayımlanmış Kıyaslamalar Açısından Çıkarımlar

Sonucumuz Chapman-Shaoxing'in ötesine genelleşirse; giriş uzunluğunun karşılaştırmalı EKG derin öğrenme literatüründe bir karıştırıcı (confound) olduğunu ima eder. 'CNN temel modeli' ile 'CNN + dikkat' karşılaştıran iki makale, dikkat katkısını

değil, iki mimarinin tümevarımsal önyargılarıyla giriş uzunluğu arasındaki etkileşimi ölçüyor olabilir. Saha için yararlı bir deneysel norm şöyle olabilir: sonuçları külliyat varsayılan örnekleme hızında değil, temel modelin doğruluğunu en üst düzeye çıkaran altörnekleme katsayısında raporlayın.

Bunu tek bir külliyat (Chapman-Shaoxing) üzerinde tek bir mimari (düz bir artıklı 1B-CNN) ile gösterdiğimiz konusunda açığız. Başlamış olduğumuz ve sonraki çalışmalarda raporlamayı umduğumuz PTB-XL çapraz veri seti doğrulaması, genelliğin en yakın testidir.

SONUÇ VE ÖNERİLER

Katkıların Özeti

Bu tez; 12 kanallı EKG sınıflandırmasında giriş uzunluğunun bir tasarım değişkeni olarak kontrollü bir incelemesini sunmuştur. Model mimarisi (artıklı 1B-CNN), kayıp fonksiyonu (odak kayıp yeniden ağırlıklı etiket-yumuşatılmış çapraz entropi), veri artırma politikası, eniyileyici ve rastgele tohum sabit tutularak yalnızca giriş uzunluğunu kanal başına {5000, 1000, 500} örneklem arasında değiştiriyoruz. Sonuç; test doğruluğunun %88,43'ten %97,38'e monoton bir iyileşmesi, on bir temel başarısız sınıfın F1 ≥ 0,95 düzeyine geri kazandırılması ve konfigürasyona bağlı olarak 3,3-9,8× hızlanmadır.

Sonucu, sıfır-fazlı Chebyshev tip-I altörneklemesi altında referans nokta grafının geometrik değişmezliği olarak çerçeveledik; üç bileşik kuvveti (alıcı alan kapsamı, referans nokta yoğunluğu ve parametre ekonomisi) tanımladık ve anti-aliasing süzgecinin belirleyici olduğunu — naif dilimleme ile değiştirmenin doğruluğu temel modelin altına düşürdüğünü — gösterdik. Tam PyTorch uygulaması Bölüm 5'te ve Ek A'da yeniden üretilmiştir.

Sınırlılıklar

Tek bir veri setine (Chapman-Shaoxing) dayanıyoruz. Altörneklemeli ve altörneklemesiz PTB-XL çapraz veri seti doğrulaması, en yakın testtir. Altörnekleme katsayısını 500 örneklemin altında (q > 10) veya giriş uzunluğunun daha derin / dikkat ile güçlendirilmiş modellerle etkileşimini karakterize etmedik. Geometrik değişmezlik argümanı; q = 10'da 25 Hz anti-aliasing kesimiyle tasarım gereği kaldırılan yüksek frekans içeriğine dayanan alt-tanılar (geç potansiyeller, mikro-alternanslar, fragmente QRS) için geçerli olmayabilir. Son olarak, her raporlanan sonuç tek tohumludur; raporlanan varyans bantlarıyla çoklu-tohum bir sağlamlık çalışması, takip çalışması için planlanmıştır.

Klinik ve Toplumsal Faydalar

Tezin başlangıçtaki proje önerisinde tanımlanan beş fayda kategorisi, varılan sonuçlar ışığında aşağıdaki biçimde gerçekleşmiştir. Bu kategoriler, KTMÜ-fr-Tİİ-28

tez önerisi formunun 4. maddesinde (Tez Çalışmasının Önemi/Sağlayacağı Faydalar) ortaya konan klinik değer önermesinin pratik yansımasıdır.

1. Erken teşhis: 27,2 ms tek-örnek çıkarım gecikmesiyle, model, klinik akışta gerçek zamanlı taramaya elverişli hâle gelmiştir; on bir kritik sınıfın (LVH, MI, AV blokları, atriyal fibrilasyon vb.) F1 ≥ 0,95 düzeyinde tespit edilmesi, tedavinin en etkili olduğu erken aşamada kalp hastalığı belirtilerinin saptanmasını mümkün kılar.
2. Teşhis süresinin kısaltılması: Manuel uzman yorumlama yaklaşık 5-15 dakika sürerken, ONNX Runtime üzerinde çalışan model, tek bir 12 kanallı EKG için 50 ms altında tahmin üretmektedir; bu yaklaşık 6.000× hızlanmadır ve yığın işlemede yüksek hacimli klinik tarama için ölçeklenebilir.
3. Geliştirilmiş doğruluk: Beş kategorili sınıflandırmada %97,38 test doğruluğu ile 0,9744 makro-F1 düzeyine ulaşılmıştır. Önerideki literatür notu olan 'EKG yorumlarının %33'üne kadarı uzman referansına kıyasla hata içerir' bulgusunun ışığında, bu doğruluk düzeyi klinik karar destek aracı olarak anlamlı katkı sağlar; özellikle uzmanlık merkezleri dışındaki birinci basamak ortamlarında.
4. Sağlık hizmetlerine erişilebilirliğin iyileştirilmesi: ONNX modelinin sıradan CPU sunucularında çalışması ve INT8 nicemleme ile Raspberry Pi 4 üzerinde dağıtım olanağı, sistemin Orta Asya'nın uzak bölgelerindeki küçük klinik ve mobil sağlık birimlerinde konuşlandırılmasını mümkün kılmaktadır.
5. İyileştirilmiş tedavi sonuçları: Eşlik eden klinik karar destek web uygulaması (FastAPI + React + ONNX Runtime); kalp hastalığının daha hızlı ve doğru teşhisini, iki dilli (İngilizce/Rusça) PDF rapor üretimini ve incelemelerin zaman içinde karşılaştırılmasını sağlayarak klinik iş akışına doğrudan değer katmaktadır.

Eşlik Eden Klinik Karar Destek Web Uygulaması

Araştırma kodunun yanı sıra, bir klinik karar destek web uygulaması geliştirilmiş ve pilot kullanımdadır. Uygulama iki katmanlı bir sistemdir: ONNX'e dışa aktarılmış modeli (PyTorch kontrol noktasından training/export_onnx.py tarafından üretilen) tüketen bir FastAPI Python arka ucu ve doktorların incelemeleri yüklemek, tahminleri gözden geçirmek, sinyal görselleştirmelerini görüntülemek, incelemeleri zaman içinde karşılaştırmak ve iki dilli (İngilizce/Rusça) PDF raporları dışa aktarmak için kullandığı bir React + TypeScript tek-sayfalık uygulama.

Araştırma ve dağıtım arasındaki mimari ayrım kasıtlıdır. Araştırma kodu (eğit, değerlendir, ablate); PyTorch 2.4 ile tek bir GPU iş istasyonunda çalışır. Dağıtım kodu sıradan CPU sunucularında ONNX Runtime kullanarak çalışır; PyTorch bağımlılığı yoktur ve len=500 modeli için 50 ms altı çıkarım üretir. Bu ayrım üretim konteynerini

küçük tutar (CUDA yok, PyTorch wheel yok), saldırı yüzeyini azaltır ve araştırma ile dağıtım ekiplerinin bağımsız olarak yinelemesine olanak tanır.

Web uygulamasının öne çıkan özellikleri:

- Yönetici tarafından tohumlanan kullanıcı oluşturma ile JWT tabanlı kimlik doğrulama; doktor hesapları geçici şifreleri e-posta yoluyla alır.
- Hasta CRUD ve hasta başına inceleme geçmişi.
- Çok formatlı yükleme: WFDB (.hea + .mat), MIT-BIH (.dat), CSV, fotoğraflar ve kâğıt EKG çıktılarının PDF taramaları.
- Dijital EKG ekipmanı bulunmayan birinci basamak ortamlarında yararlı olan, kâğıt EKG fotoğrafından yaklaşık 12 kanallı sayısal sinyal kurtaran görüntüden sinyale dijitalleştirme hattı.
- api/services/ecg_inference.py aracılığıyla ONNX çıkarımı; tanı kategorisini, güveni, ilk 3 alternatifi ve kalibre edilmiş risk skorunu (Bölüm 5.6) döndürür.
- İngilizce veya Rusça LLM tarafından üretilen anlatı özeti; (tanı, dil, yaş aralığı, cinsiyet, model sürümü) anahtarıyla yedi gün boyunca Redis önbellekli.
- WeasyPrint aracılığıyla iki dilli PDF rapor dışa aktarımı; düzenleyici izlenebilirlik için her sayfaya model sürümü damgalanmış.
- Doktor ve klinik bazında demografik döküm ve inceleme hacmi eğilimleri içeren analitik panosu.
- FDA SaMD / EU MDR uyumluluğu için her tanı tahmini, model sürümü, kullanıcı, hasta ve zaman damgasının denetim günlüğü.

Model güncelleme iş akışı kasıtlı olarak basittir: (1) PyTorch'ta yeniden eğit; (2) training/export_onnx.py ile ONNX'e dışa aktar; (3) ONNX paketini (model + etiket kodlayıcı + metadata.json + version.txt) ecg-webapp/model/ konumuna kopyala; (4) API konteynerini yeniden başlat. Uygulama yeni ONNX dosyasını başlangıçta yükler; herhangi bir API kod değişikliği gerekmez ve geri alma tek bir ortam değişkeni değişikliğidir.

Mevcut hat, tek model sürümü olan tek-kiracılı bir klinik dağıtımı için uygundur. Özelleşmiş bir model sunucusuna (Triton, KServe veya BentoML) geçiş yolu proje BENİOKU'sunda belgelenmiştir ve şunlardan herhangi biri doğru hâle gelirse değerlendirilmelidir: üretimde aynı anda birden çok model sürümü (A/B veya kanarya), kiracıya özel model varyantları olan çok kiracılı dağıtım, > 10 RPS ile GPU çıkarımı veya 1 GiB'i aşan model artefaktı. Mevcut pilotta bu tetikleyicilerden hiçbiri uygulanmaz.

ÖZET

1. Çalışmanın Konusu ve Önemi

Kardiyovasküler hastalıklar, Dünya Sağlık Örgütü'nün en güncel verilerine göre yıllık tahmini 17,9 milyon ölüme yol açmakta ve dünya genelindeki tüm ölümlerin yaklaşık üçte birinden sorumlu görünmektedir. 12 kanallı elektrokardiyogram (EKG); aritmiler, ileti bozuklukları, miyokard infarktüsü, ventriküler hipertrofi ve repolarizasyon anormallikleri başta olmak üzere geniş bir kardiyak patoloji yelpazesini saptamak için kullanılan, en yaygın ve en yüksek hacimli non-invaziv tanı aracıdır. Standart bir 10 saniyelik 12 kanallı kayıt, 500 Hz örnekleme hızında 12 × 5000 boyutunda bir matris üretir ve kardiyak vektörün on iki geometrik izdüşümünden yaklaşık 8-12 kalp atımı yakalar.

EKG'nin doğru yorumlanması; uzun yıllar süren bir uzmanlık eğitimi gerektirmektedir. Kardiyologlar; atriyal flatter ile atriyoventriküler düğümsel re-entrant taşikardisini ayırt eden ya da ön duvar miyokard infarktüsünün milimetre altı ST segment yükselmelerini tanıyan morfolojik incelikleri yorumlama becerisini ancak yıllar içinde geliştirir. Uzmanlık merkezleri dışında — birinci basamak sağlık hizmetlerinde, ambulans hizmetlerinde, bu çalışmanın yürütüldüğü Orta Asya'nın uzak kırsal bölgelerinde — 12 kanallı bir EKG'yi gerçek zamanlı yorumlayabilecek yerel kardiyoloji uzmanlığı çoğu kez mevcut değildir. Bu durum, otomatik EKG yorumlama sistemlerine duyulan klinik ihtiyacı belirgin biçimde artırmaktadır.

2. Problem Tanımı ve Araştırma Soruları

Bu tezin bir önceki aşamasında, 12 kanallı EKG sınıflandırması için geleneksel giriş temsiliyle (12 kanal × 5000 örneklem, 500 Hz) Chapman-Shaoxing külliyatı üzerinde temel bir 1B evrişimli sinir ağı (1B-CNN) eğitilmiştir. Bu temel model 78 tanı kategorisinde yalnızca %88,43 test doğruluğu ve 0,8713 makro-F1 değerine ulaşabilmiş; on bir kategori F1 = 0,60 eşiğinin altına düşmüştür. Literatürdeki alışılagelmiş yanıt, bu açığı kapatmak için mimari sofistikasyon eklemek olmuştur: dikkat katmanları, yinelemeli kodlayıcılar, odak kayıplı eğitim, etiket-taksonomi

temizliği ve hibrit CNN-LSTM yığınları bu doğrultuda öne sürülen başlıca yaklaşımlardır.

Bu tezde, söz konusu doğal refleksin tersine, karşıt bir hipotez sınanmıştır: 5000 örneklemli giriş, tipik kapasiteli bir CNN'in yararlanabileceğinden çok daha fazla zamansal artıklık taşımakta; tek satırlık bir ön işleme değişikliği — SciPy'nin Chebyshev tip-I süzgeci ile 500 örnekleme anti-aliasing'li altörnekleme — tüm tanısal açıdan ilgili özellikleri korurken gradyan sinyalini bu özellikler üzerinde yoğunlaştırma potansiyeline sahiptir. Bu hipotez doğrulanırsa, düz 1B-CNN modelleri ile dikkat-hibrit modeller arasındaki yayımlanmış performans farkının önemli bir kısmının mimari yenilik değil, yetersiz eğitilmiş temel modelleri yansıttığı sonucuna varılır.

Tez kapsamında dört araştırma sorusu formüle edilmiştir. Birinci soru: mimari, kayıp, eniyileyici, veri artırma, tohum ve ayrım sabit tutulduğunda, giriş uzunluğu Chapman-Shaoxing üzerinde temel bir 1B-CNN'in test doğruluğunu ne ölçüde belirlemektedir? İkinci soru: anti-aliasing'li altörnekleme, sınıf bazında F1 geri kazanımıyla ölçüldüğü şekliyle EKG'nin tanısal içeriğini korumakta mıdır? Üçüncü soru: altörneklemenin duvar saati ve verimlilik maliyeti ne düzeydedir; modern GPU'larda DataLoader yapılandırmasıyla nasıl etkileşmektedir? Dördüncü soru: geometrik değişmezlik argümanı, sınıf bazında geri kazanım profilini açıklamakta yeterli midir, yoksa iyileşme yalnızca anti-aliasing süzgecinin düzenlileştirici etkisiyle mi açıklanır?

3. Veri Seti ve Ön İşleme

Çalışmada Chapman-Shaoxing 12 kanallı EKG veri tabanı (Zheng ve ark., 2020) kullanılmıştır. Bu külliyat; Shaoxing Halk Hastanesi'nde 2013-2019 yılları arasında elde edilen 10.646 hastaya ait 45.152 adet 10 saniyelik 12 kanallı EKG kaydını, WFDB formatında bir çift dosya (metin tabanlı .hea başlığı ve MATLAB .mat veri yükü) olarak içermektedir. Tanısal annotation, 78 farklı SNOMED CT kodu kullanılarak yapılmış; az sayıda kayıt birden fazla kod taşımaktadır.

Sınıflandırma hedefleri _read_header_metadata, _map_snomed_to_diagnosis ve _map_snomed_to_category_diagnosis yöntemleriyle üretilmiştir. SNOMED kodları beş üst-düzey klinik kategoriye eşlenmiştir: Normal (sinüs ritmi ve normal ileti), Miyokard

İnfarktüsü (MI), ST/T-Dalga Değişiklikleri (STTC), İleti Bozukluğu (CD) ve Hipertrofi (HYP). Birden fazla kod taşıyan kayıtlar için _select_primary_diagnosis yöntemi MI > CD > STTC > HYP > Normal klinik öncelik sırasını uygulamıştır. Sınıf dengesizliği belirgindir; load_local_records yönteminde max_samples_per_class = 5000 ile çoğunluk sınıfları kayıt düzeyinde altörneklenmiştir.

Tüm sinyaller dördüncü dereceden Butterworth bant geçiren süzgeçten (0,5 Hz yüksek geçiren ile taban kayma giderimi, 40 Hz alçak geçiren ile kas artefaktı giderimi) geçirilmiş; ardından her kanal bağımsız olarak sıfır ortalama ve birim varyansa normalleştirilmiştir (kanal başına z-skor). Çok küçük sınıflar için (n < 500) genlik ölçekleme [0,85; 1,15], Gaussian gürültü σ = 0,02, ±%5 dairesel zaman kaydırma ve %10 olasılıkla genlik ters çevirme içeren agresif veri artırma uygulanmıştır.

4. Yöntem ve Model Mimarisi

Model mimarisi, tek boyutlu bir artıklı (residual) evrişimli sinir ağıdır. Temel birim ResidualBlock olarak tanımlanmış olup iki adet 1B evrişim, ardından toplu normalleştirme ve atlama bağlantısı içermektedir. Atlama bağlantısının amacı, gradyanların evrişimsel yolu atlamasına izin vererek derin ağlardaki yok olan/patlayan gradyan sorununu önlemektir (He ve ark., 2016).

Tam ECGCNN modülü; bir başlangıç evrişimi (64 kanal, kernel 7) ve dört ResidualBlock'u yığar. Kanal programı 64 → 128 → 256 → 512 → 512 olarak büyür. Her artıklı aşamadan sonra 2 ile maks-havuzlama uygulanır. Global ortalama havuzlama (AdaptiveAvgPool1d) zamansal boyutu daraltır; ardından iki tam bağlantılı katman (256 → 128 → num_classes) logitleri üretir. Toplam parametre sayısı yaklaşık 3,72 milyondur; global ortalama havuzlama sayesinde model giriş uzunluğundan bağımsız olarak çalışır.

Eğitim kaybı; sınıf-ağırlıklı çapraz entropiyi Lin ve ark.'nın (2017) odak kayıp yeniden ağırlıklamasıyla (γ = 2,0) birleştirir. Ek olarak, etiket yumuşatma (Szegedy ve ark., 2016; smoothing = 0,1) ileri geçişte uygulanır. Eniyileyici Adam'dır (lr = 1e-4, weight_decay = 1e-2); ReduceLROnPlateau çizelgesi beş epoch'luk doğrulama-kayıp platosunda öğrenme oranını yarıya indirir. Erken durdurma on epoch boyunca iyileşme

olmazsa devreye girer. Karma duyarlıklı eğitim (torch.amp/FP16) kullanılarak RTX 5090 üzerinde 1,6-2× hızlanma elde edilmiştir.

Anti-aliasing'li altörnekleme adımı, bu tezin merkezi ablation değişkenidir. $q \in \{1, 5, 10\}$ katsayılarıyla scipy.signal.decimate fonksiyonu çağrılmakta; bu fonksiyon dahili olarak sekizinci dereceden Chebyshev tip-I IIR alt-geçiren süzgeci ileri-geri (sıfır-fazlı) modda uygulamaktadır. Sıfır-faz özelliği zorunludur: herhangi bir faz kayması, referans nokta konumlarını frekansa bağlı biçimde kaydırır ve geometrik değişmezlik argümanını ortadan kaldırır.

5. Geometrik Değişmezlik Argümanı

Tezin merkezi teorik katkısı, referans noktaların (fiducial points) geometrik değişmezliği argümanıdır. Bir EKG'nin tanısal içeriği; P, QRS ve T dalgalarının başlangıç, tepe ve bitiş noktaları olmak üzere yaklaşık 60 referans nokta içeren seyrek bir kümede yoğunlaşır. 500 Hz'te 10 saniyelik bir pencere için bu, 5000 örneklem arasına yayılan yaklaşık 60 referans nokta anlamına gelir; örneklerin yaklaşık %98'i, referans nokta grafının zaten kodladığının ötesinde bilgi taşımaz.

Sıfır-fazlı Chebyshev tip-I anti-aliasing süzgeci, bu noktaların geometrik konfigürasyonunu örnekleme çözünürlüğü hassasiyetinde korur. Her referans noktanın zamanı, yeni örnekleme periyodunun ±½'si dahilinde korunur; 10× altörnekleme sonrasında bu çözünürlük 10 ms'dir; QT aralığı gibi en sıkı klinik zamanlama ölçümü 10 ms hassasiyetinde raporlanmaktadır. EKG eğrisinin şekli — referans noktaları arasında bir kırık çizgi olarak görüldüğünde — altörnekleme altında değişmezdir.

6. Deneysel Sonuçlar

Tüm deneyler tek bir NVIDIA RTX 5090 GPU (34,19 GiB VRAM, CUDA 12.8) üzerinde PyTorch 2.4 ve SciPy 1.13 ile gerçekleştirilmiştir. Dört konfigürasyon her hiperparametreyi paylaşır; yalnızca giriş uzunluğu (ve son çalıştırmada DataLoader işçi sayısı) değişkendir.

Manşet sonuçlar şöyledir: len = 5000 (temel) konfigürasyonu %88,43 test doğruluğu, 0,8713 makro-F1 ve 89,88 ms tek-örnek çıkarım gecikmesi; len = 1000 konfigürasyonu %97,22 test doğruluğu, 0,9716 makro-F1 ve 26,14 ms gecikme; len =

500 konfigürasyonu %97,34 test doğruluğu, 0,9737 makro-F1 ve 27,20 ms gecikme; len = 500 + 4 DataLoader işçisi konfigürasyonu ise %97,38 test doğruluğu, 0,9744 makro-F1 ve 43,50 ms (yığın bazlı) gecikme üretmektedir. 5000 → 500 altörneklemesi 8,91 yüzde puan ve 0,1024 makro-F1 iyileşme sağlarken çıkarımı 3,3× hızlandırmaktadır. Ana etki 1000 örneklemde yakalanmıştır; 1000 → 500 yalnızca 0,12 puan ek iyileşme getirmektedir.

Sınıf bazında geri kazanım çözümlemesi, len = 5000 temelinde F1 = 0,60 altına düşen on bir sınıfın len = 500'de tek tip biçimde F1 ≥ 0,95 düzeyine geri kazandırıldığını ortaya koymaktadır. En çarpıcı iyileşme, temel modelde en kötü performans gösteren Sol Ventriküler Hipertrofi (F1 = 0,022 → ≥ 0,99; Δ = +0,97) sınıfında gözlenmiştir. Anormal Q dalgası (0,180 → ≥ 0,99), İç ileti farklılıkları (0,286 → ≥ 0,98) ve Atriyal flatter (0,581 → ≥ 0,99) gibi morfolojik sınıflar F1 ≥ 0,99 düzeyine ulaşmıştır.

7. Tartışma ve Çıkarımlar

Sonuç, üç bileşik kuvvetin ortak etkisi olarak yorumlanmaktadır. Birinci kuvvet, alıcı alan kapsamıdır: ağın yaklaşık 2048 örneklik etkin alıcı alanı, 5000 örneklemde pencerenin yalnızca %40'ını kapsarken 500 örneklemde tüm 10 saniyelik pencereyi aşmaktadır. Bu, ritime dayalı sınıfların geri kazanımını açıklamaktadır. İkinci kuvvet, referans nokta yoğunluğudur: gradyan sinyali geometrik olarak bilgilendirici örneklerde yoğunlaşır. Üçüncü kuvvet, parametre ekonomisidir: 3,7 milyon parametrelik kapasite, artıklı düşük frekans varyasyonunu modellemekten ince morfolojik ayrımları öğrenmeye yeniden tahsis edilir.

Anti-aliasing süzgeci belirleyicidir. Bir yan deneyde scipy.signal.decimate yerine numpy dilimleme [::10] kullanıldığında doğruluk %97,34'ten yaklaşık %84 düzeyine — len = 5000 temel modelinin de altına — düşmüştür. Bu bulgu, '+10 puan F1' ile 'temel modelden de kötü' arasındaki farkın anti-aliasing süzgeciyle açıklandığını ve geometrik değişmezlik argümanını uygulamada geçerli kılan adımın bu olduğunu kanıtlamaktadır.

Sonuç; dikkatin, yinelemeli katmanların veya odak kaybının yararsız olduğu anlamına gelmemektedir. Bu mekanizmaların, giriş boyutunda yetersiz eğitilmiş bir

temel modele karşı ölçüldüğünü; dolayısıyla bildirilen katkılarının daha düşük bir başlangıç noktasına göre üst sınır olduğunu ima etmektedir. Bu mekanizmaların decimate-500 temel modeline karşı yeniden değerlendirilmesi gelecek çalışmaların önemli bir parçasıdır.

8. Sınırlılıklar ve Gelecek Çalışmalar

Çalışma tek bir veri setine (Chapman-Shaoxing) dayanmaktadır. Giriş uzunluğu etkisinin farklı edinim ekipmanı, hasta demografisi veya etiket taksonomisine sahip diğer külliyatlara genelleşip genelleşmediği açık bir sorudur. PTB-XL çapraz veri seti doğrulaması en yakın testtir ve sürmektedir. Altörnekleme katsayısı 500 örneklemin altında ($q > 10$) karakterize edilmemiştir; geometrik değişmezlik argümanı, $q = 10$'da 25 Hz anti-aliasing kesimiyle tasarım gereği kaldırılan yüksek frekans içeriğine dayanan alt-tanılar (geç potansiyeller, mikro-alternanslar, fragmente QRS) için geçerli olmayabilir.

Her raporlanan sonuç tek tohumludur; çoklu-tohum varyans bantları içeren bir sağlamlık çalışması, takip çalışmaları için planlanmıştır. Klinik dağıtım için sınıf bazında duyarlılık/özgüllük eğrileri ve olasılık kalibrasyonu gibi metrikler eklenmelidir. Eşlik eden klinik karar destek web uygulaması (FastAPI + React + ONNX Runtime) zaten pilot kullanımdadır; INT8 nicemleme ile Raspberry Pi 4 üzerinde kenar dağıtımı doğal sonraki adımdır.

9. Sonuç

Bu tez; mimari, kayıp, eniyileyici, veri artırma ve tohum sabit tutulduğunda yalnızca giriş uzunluğunun değiştirilmesinin, temel bir 1B-CNN'in test doğruluğunu %88,43'ten %97,38'e yükselttiğini göstermiştir. Bu; on bir başarısız sınıfın $F1 \geq 0{,}95$ düzeyine geri kazandırılması ve 3,3-9,8× hızlanma ile birlikte gelmektedir. Tüm sonuç; sıfır-fazlı Chebyshev tip-I altörnekleme altında referans nokta grafının geometrik değişmezliği çerçevesinde açıklanmıştır. Tezde, giriş uzunluğunun yayımlanmış EKG kıyaslama çalışmalarında yetersiz raporlanan bir tasarım değişkeni olduğu ve düz CNN modelleri ile dikkat-hibrit modeller arasındaki farkın bir kısmının uzunluk-eniyilemesinden kaynaklanabileceği savunulmaktadır. PyTorch referans uygulamasının

tamamı (yaklaşık 1.800 satır), klinik karar destek web uygulaması ile birlikte bir bütün olarak sunulmuştur.

КЕҢИРИ КЫСКАЧА МАЗМУНУ (КЫРГЫЗЧА)

1. Изилдөөнүн темасы жана маанилүүлүгү

Дүйнөлүк ден соолук уюмунун эң акыркы маалыматы боюнча, жүрөк-кан тамыр оорулары жыл сайын болжол менен 17,9 миллион өлүмгө алып келип, дүйнөдөгү бардык өлүмдөрдүн үчтөн бирин түзөт. 12 каналдуу электрокардиограмма (ЭКГ); аритмияларды, өткөрүү бузулууларын, миокард инфарктын, желудоктук гипертрофияны жана реполяризация аномалияларын камтыган кеңири кардиалдык патологиялардын аныкталышы үчүн колдонулган, эң кеңири жайылган жана эң жогорку көлөмдөгү инвазивдик эмес диагностикалык каражат болуп саналат. Стандарттык 10 секунддук 12 каналдуу жазуу, 500 Гц жыштыкта 12 × 5000 матрицасын чыгарат жана жүрөк вектордун он эки геометриялык проекциясынан болжол менен 8-12 жүрөк уруусун тартат.

ЭКГнын туура чечмелениши; узак жылдар бою кесиптик билимди талап кылат. Кардиологдор; алдыртан флаттер менен атриовентрикулярдык түйүндүн ре-энтрант тахикардиясын ажыратуу же алдыңкы дубалдын миокард инфарктынын миллиметрден аз ST сегмент жогорулашын тааныган морфологиялык тыкыйыктарды чечмелөө жөндөмүн жылдар бою өнүктүрөт. Адистешкен борборлордон сырткары — биринчи деңгээлдеги саламаттык сактоодо, тез жардам кызматтарында, ушул иш жүргүзүлгөн Орто Азиянын алыскы айыл аймактарында — 12 каналдуу ЭКГны реалдуу убакта чечмелей ала турган жергиликтүү кардиолог адистиги көп учурда жок болуп турат. Бул жагдай, автоматташтырылган ЭКГ чечмелөө системаларына болгон клиникалык муктаждыкты олуттуу түрдө жогорулатат.

2. Көйгөйдүн аныктамасы жана изилдөө суроолору

Бул иштин мурдагы этабында, 12 каналдуу ЭКГ классификациясы үчүн салттуу кириш формасы (12 канал × 5000 үлгү, 500 Гц) колдонулуп Chapman-Shaoxing маалыматтар базасында жөнөкөй 1B-CNN окутулган. Бул базалык модель 78 диагноз категориясында жогору болсо %88,43 тест тактыгына жана 0,8713 макро-F1 көрсөткүчүнө жетүү менен чектелген; 11 категория F1 = 0,60

чегинин астына түшкөн. Адабияттагы адаттагы жооп, бул айырманы жабуу үчүн архитектуралык татаалдашууну кошуу болду: көңүл буруу катмарлары, кайра кайталануучу кодировщиктер, фокусу жоготуу окутуусу, белгилерди тазалоо жана гибрид CNN-LSTM топтомдору ушул багытта сунушталган негизги ыкмалардыр.

Бул иште, ушул табигый рефлекстин тескерисине, карама-каршы гипотеза сыналган: 5000 үлгүлүк кириш, кадимки сыйымдуулуктагы CNN пайдалана алгандан алда канча көп убакыттык кайталоону камтыйт; бир сап алдын ала иштетүү өзгөрүүсү — SciPy'нин Чебышев тип-I фильтри менен 500 үлгүгө анти-aliasing'дик кыскартуу — бардык диагностикалык маанилүү өзгөчөлүктөрдү сактоо менен бирге градиент сигналын ушул өзгөчөлүктөргө топтоштуруу мүмкүнчүлүгүнө ээ. Бул гипотеза ырасталса, жөнөкөй 1B-CNN моделдери менен көңүл буруу-гибрид моделдери ортосундагы жарыяланган айырмачылыктын бир бөлүгү архитектуралык жаңылануулардан эмес, кириш узундугун оптимизациялоодон келип чыкат деп жыйынтык чыгарууга болот.

3. Маалымат базасы жана алдын ала иштетүү

Изилдөөдө Chapman-Shaoxing 12 каналдуу ЭКГ маалымат базасы (Zheng жана башкалар, 2020) колдонулган. Бул база; Шаосин Эл Ооруканасында 2013-2019-жылдары алынган 10.646 пациентке тиешелүү 45.152 даана 10 секунддук 12 каналдуу ЭКГ жазууларын камтыйт. Диагностикалык annotation 78 түрдүү SNOMED CT коду колдонулуп жасалган.

Бардык сигналдар Баттерворт зоналык өткөргүч фильтринен (0,5 Гц жогорку өткөргүч менен таандык тегиздөө, 40 Гц төмөнкү өткөргүч менен булчуң артефактын алып салуу) өткөрүлгөн; андан кийин ар бир канал өзүнчө нөл орточо маниге жана бирдик дисперсияга нормалдаштырылган (канал боюнча z-skor). Эң кичинекей класстар үчүн ($n < 500$) амплитуда масштабдоо [0,85; 1,15], Гаусс ызы $\sigma = 0{,}02$, ±%5 тегерек убакыт жылыштыруу жана %10 ыктымалдуулук менен амплитуда тескери буруу камтылган агрессивдүү маалыматтарды көбөйтүү колдонулган.

4. Метод жана моделдин архитектурасы

Моделдин архитектурасы, бир өлчөмдүү калдыктуу (residual) свертка нейрондук тарабы. Негизги бирдик ResidualBlock деп аталат жана эки 1B свертка, андан кийин batch normalization жана skip connection камтыйт. Skip connection-дин максаты, градиенттердин свертка жолун айланып өтүшүнө жол берип, терең тармактардагы жоголгон/жарылган градиент маселесин алдын алуу (He жана башкалар, 2016).

Толук ECGCNN модулу; баштапкы свертка (64 канал, kernel 7) жана төрт ResidualBlock'ту үстөмдөтөт. Канал программасы 64 → 128 → 256 → 512 → 512 деп өсөт. Ар бир калдык этабынан кийин 2 менен max-pooling колдонулат. Глобалдык орточо pooling (AdaptiveAvgPool1d) убакыт өлчөмүн чектейт; андан кийин эки толук туташкан катмар (256 → 128 → num_classes) логиттерди чыгарат. Жалпы параметрлердин саны болжол менен 3,72 миллиондур.

5. Геометриялык өзгөрүүсүздүк аргументи

Тездин негизги теориялык салымы, референс чекиттеринин геометриялык өзгөрүүсүздүгү аргументиди. ЭКГнин диагностикалык мазмуну; P, QRS жана T толкундарынын башталышы, чокусу жана аякталышы болуп болжол менен 60 референс чекитти камтыган сейрек жыйнакта топтолот. 500 Гц жыштыкта 10 секунддук терезе үчүн бул, 5000 үлгү арасында жайгашкан болжол менен 60 референс чекит дегенди билдирет; үлгүлөрдүн болжол менен %98ы, референс чекиттер графынын кодуланган маалыматынан тышкары эч кандай маалымат ташыбайт.

6. Эксперименталдык натыйжалар

Бардык эксперименттер NVIDIA RTX 5090 GPU (34,19 GiB VRAM, CUDA 12.8) жалгыз өлчөмүндө PyTorch 2.4 жана SciPy 1.13 менен жүргүзүлгөн. Төрт конфигурация ар бир гиперпараметрди бөлүшөт; кириш узундугу гана (жана акыркы тестте DataLoader жумушчуларынын саны) өзгөрөт.

Башкы натыйжалар төмөнкүдөй: len = 5000 (базалык) конфигурациясы %88,43 тест тактыгы, 0,8713 макро-F1 жана 89,88 мс бир үлгүлүк божомолдоо

узактыгы; len = 1000 конфигурациясы %97,22 тест тактыгы, 0,9716 макро-F1 жана 26,14 мс узактык; len = 500 конфигурациясы %97,34 тест тактыгы, 0,9737 макро-F1 жана 27,20 мс узактык; len = 500 + 4 DataLoader жумушчусу конфигурациясы %97,38 тест тактыгы, 0,9744 макро-F1 жана 43,50 мс (кезек негизделген) узактык берет. 5000 → 500 кыскартуу 8,91 пайыздык пункт жакшыртуу жана 0,1024 макро-F1 жакшыруу менен бирге божомолду 3,3× ылдамдатат.

Класс боюнча кайтып келүү талдоосу, len = 5000 базасында F1 = 0,60 астына түшкөн он бир класстын len = 500'дө бирдей түрдө F1 ≥ 0,95 деңгээлине кайтып келгенин көрсөтүп турат. Эң таасирдүү жакшыруу, базалык моделде эң начар иштеген Сол Кардинелик Гипертрофия (F1 = 0,022 → ≥ 0,99; Δ = +0,97) классында байкалган.

7. Талкуу жана корутундулар

Натыйжа, үч айкалышкан күчтүн жалпы таасири катары чечмеленет. Биринчи күч; кабыл алуу талаасынын камтуусуду: тармактын болжол менен 2048 үлгүлүк натыйжалуу кабыл алуу талаасы, 5000 үлгүдө терезенин %40ын гана камтыса, 500 үлгүдө бүт 10 секунддук терезени ашат. Бул, ритмге негизделген класстардын кайтып келүүсүн түшүндүрөт. Экинчи күч; референс чекит тыгыздыгыдыр. Үчүнчү күч; параметр экономиясыдыр.

Анти-aliasing фильтри чечүүчү. Бир каптал эксперименттерде scipy.signal.decimate ордуна numpy тилимдеме [::10] колдонулганда тактык %97,34тен болжол менен %84 деңгээлине — len = 5000 базалык моделинен да төмөн — түшкөн. Бул табылга, '+10 пункт F1' менен 'базалык моделден да жаман' ортосундагы айырманын анти-aliasing фильтри менен түшүндүрүлгөнүн жана геометриялык өзгөрүүсүздүк аргументин практикада жарактуу кылган кадамдын ушул экендигин далилдейт.

8. Чектөөлөр жана келечек иштер

Изилдөө бир маалымат базасына (Chapman-Shaoxing) негизделген. Кириш узундугу таасиринин ар кайсы алуу шаймандары, пациенттердин демографиясы же этикеттер таксономиясы менен башка корпустарга жалпылана тургандыгы

ачык суроо. PTB-XL аралык маалымат базасы текшерилиши жакынкы тест болуп саналат жана аткарылып жатат.

Ар бир билдирилген натыйжа жалгыз уруктуу; көп-уруктуу дисперсия зоналарын камтыган бекемдик изилдөө, башкы изилдөөлөр үчүн пландалган. Клиникалык таркатуу үчүн класс боюнча сезгичтик/өзгөчөлүк ийри сызыктары жана ыктымалдуулук калибрлөө сыяктуу метрикалар кошулушу керек. ONNX менен экспорт кылынган моделди колдонгон клиникалык чечим колдоо веб тиркемеси (FastAPI + React + ONNX Runtime) пилот колдонууда; INT8 кванттоо менен Raspberry Pi 4 түзмөгүндө четке жайгаштыруу табигый кийинки кадамдыр.

9. Корутунду

Бул дисертация; архитектура, жоготуу, оптимизатор, маалыматтарды көбөйтүү жана урук туруктуу сактаганда гана кириш узундугунун өзгөртүлүшү, базалык 1B-CNN тест тактыгын %88,43тен %97,38ге жогорулатканын көрсөттү. Бул; он бир ийгиликсиз класстын F1 ≥ 0,95 деңгээлине кайтарылышы жана 3,3-9,8× ылдамдатуу менен бирге келет. Бардык натыйжа; нөл фазалуу Чебышев тип-I кыскартуу астында референс чекиттер графынын геометриялык өзгөрүүсүздүгү алкагында түшүндүрүлгөн. Дисертацияда, кириш узундугунун жарыяланган ЭКГ салыштыруу изилдөөлөрүндө жетишсиз билдирилген долбоор өзгөрмөсү экендиги жана жөнөкөй CNN моделдери менен көңүл буруу-гибрид моделдер ортосундагы айырмачылыктын бир бөлүгү узундукту оптимизациялоодон келип чыгышы мүмкүн экендиги колдоого алынат.

KAYNAKLAR

[1] P. Rajpurkar, A. Y. Hannun, M. Haghpanahi, C. Bourn, A. Y. Ng. Cardiologist-level arrhythmia detection with convolutional neural networks. arXiv:1707.01836, 2017.

[2] A. Y. Hannun, P. Rajpurkar, M. Haghpanahi, G. H. Tison, C. Bourn, M. P. Turakhia, A. Y. Ng. Cardiologist-level arrhythmia detection and classification in ambulatory electrocardiograms using a deep neural network. Nature Medicine, 25(1):65-69, 2019.

[3] N. Strodthoff, P. Wagner, T. Schaeffter, W. Samek. Deep learning for ECG analysis: Benchmarks and insights from PTB-XL. IEEE Journal of Biomedical and Health Informatics, 25(5):1519-1528, 2020.

[4] B. K. Iwana, S. Uchida. An empirical survey of data augmentation for time series classification with neural networks. PLoS ONE, 16(7):e0254841, 2021.

[5] Z. Wang, W. Yan, T. Oates. Time series classification from scratch with deep neural networks. International Joint Conference on Neural Networks (IJCNN), 1578-1585, 2017.

[6] X. Chen, Z. Wang, M. J. McKeown. Adaptive support-guided deep learning for physiological signal analysis. IEEE Transactions on Biomedical Engineering, 68(5):1573-1584, 2021.

[7] S. S. Xu, M.-W. Mak, C. C. Cheung. Support-guided augmentation for electrocardiogram signal classification. Biomedical Signal Processing and Control, 71:103213, 2022.

[8] S. L. Oh, E. Y. Ng, R. S. Tan, U. R. Acharya. Automated diagnosis of arrhythmia using combination of CNN and LSTM techniques with variable length heart beats. Computers in Biology and Medicine, 102:278-287, 2018.

[9] J. Zheng, J. Zhang, S. Danioko, H. Yao, H. Guo, C. Rakovski. A 12-lead electrocardiogram database for arrhythmia research covering more than 10,000 patients. Scientific Data, 7(1):48, 2020.

[10] T.-Y. Lin, P. Goyal, R. Girshick, K. He, P. Dollar. Focal loss for dense object detection. IEEE International Conference on Computer Vision (ICCV), 2980-2988, 2017.

[11] P. Wagner, N. Strodthoff, R.-D. Bousseljot, D. Kreiseler, F. I. Lunze, W. Samek, T. Schaeffter. PTB-XL, a large publicly available electrocardiography dataset. Scientific Data, 7(1):154, 2020.

[12] G. B. Moody, R. G. Mark. The impact of the MIT-BIH arrhythmia database. IEEE Engineering in Medicine and Biology Magazine, 20(3):45-50, 2001.

[13] H. Nyquist. Certain topics in telegraph transmission theory. Transactions of the AIEE, 47:617-644, 1928.

[14] A. V. Oppenheim, R. W. Schafer. Discrete-Time Signal Processing, 3rd ed. Pearson, 2009.

[15] P. Virtanen et al. SciPy 1.0: Fundamental algorithms for scientific computing in Python. Nature Methods, 17:261-272, 2020.

[16] K. He, X. Zhang, S. Ren, J. Sun. Deep residual learning for image recognition. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 770-778, 2016.

[17] S. Ioffe, C. Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. International Conference on Machine Learning (ICML), 448-456, 2015.

[18] D. P. Kingma, J. Ba. Adam: A method for stochastic optimization. International Conference on Learning Representations (ICLR), 2015.

[19] N. Srivastava, G. E. Hinton, A. Krizhevsky, I. Sutskever, R. Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. Journal of Machine Learning Research, 15:1929-1958, 2014.

[20] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, Z. Wojna. Rethinking the inception architecture for computer vision. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2818-2826, 2016.

[21] N. V. Chawla, K. W. Bowyer, L. O. Hall, W. P. Kegelmeyer. SMOTE: Synthetic minority over-sampling technique. Journal of Artificial Intelligence Research, 16:321-357, 2002.

[22] PhysioNet. WFDB software package: Tools for working with physiologic signal databases. Erişim: https://www.physionet.org/about/software/.

[23] A. Paszke et al. PyTorch: An imperative style, high-performance deep learning library. Advances in Neural Information Processing Systems (NeurIPS), 8024-8035, 2019.

[24] F. Pedregosa et al. Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12:2825-2830, 2011.

[25] Dünya Sağlık Örgütü. Cardiovascular diseases (CVDs) — Bilgi Notu. Cenevre, 2023. Erişim: https://www.who.int/news-room/fact-sheets/detail/cardiovascular-diseases-(cvds).

[26] A. Ayal, M. Elbashir, A. Mohammed. Classification of 27 heart abnormalities using 12-lead ECG signals with combined deep learning techniques. Bulletin of Electrical Engineering and Informatics, 12(4):2220-2235, 2023. https://doi.org/10.11591/eei.v12i4.4668

[27] C. J. Breen, G. Kelly, W. Kernohan. ECG interpretation skill acquisition: A review of learning, teaching and assessment. Journal of Electrocardiology, 73, 2019. https://doi.org/10.1016/j.jelectrocard.2019.03.010

[28] Z. Ebrahimi, M. Loni, M. Daneshtalab, A. Gharehbaghi. A review on deep learning methods for ECG arrhythmia classification. Expert Systems with Applications: X, 7:100033, 2020. https://doi.org/10.1016/j.eswax.2020.100033

[29] X. Liu, H. Wang, Z. Li, L. Qin. Deep learning in ECG diagnosis: A review. Knowledge-Based Systems, 227:107187, 2021. https://doi.org/10.1016/j.knosys.2021.107187

[30] N. Rafie, A. H. Kashou, P. A. Noseworthy. ECG Interpretation: Clinical Relevance, Challenges, and Advances. Hearts, 2(4), 2021. https://doi.org/10.3390/hearts2040039

[31] A. H. Ribeiro, M. H. Ribeiro, G. M. M. Paixão, D. M. Oliveira, P. R. Gomes, J. A. Canazart, M. P. S. Ferreira, C. R. Andersson, P. W. Macfarlane, W. Meira Jr., T. B. Schön, A. L. P. Ribeiro. Automatic diagnosis of the 12-lead ECG using a deep neural network. Nature Communications, 11(1):1760, 2020. https://doi.org/10.1038/s41467-020-15432-4

[32] Mount Sinai Health System. Electrocardiogram Information. New York, t.y. Erişim: https://www.mountsinai.org/health-library/tests/electrocardiogram

[33] Johns Hopkins Medicine. Electrocardiogram. Mart 2024. Erişim: https://www.hopkinsmedicine.org/health/treatment-tests-and-therapies/electrocardiogram

[34] Department of Health & Human Services. ECG test. Better Health Channel, Victoria, AU, t.y. Erişim: http://www.betterhealth.vic.gov.au/health/conditionsandtreatments/ecg-test

EK 1. TAM KOD LİSTELEMELERİ

Bu ek; Bölüm 4-5'te açıklanan sistemi uygulayan, training/ecg_cnn_pytorch.py adlı 1.767 satırlık modülün tamamını hafif biçimsel düzenlemelerle yeniden üretmektedir. Modül şöyle düzenlenmiştir:

- Modül içe aktarımları ve genel tohum (satırlar 1-47).
- FocalLoss ve label_smoothing_loss yardımcıları (satırlar 50-82).
- ResidualBlock ve ECGCNN modülleri (satırlar 84-192).
- Tüm veri, ön işleme, eğitim, değerlendirme ve çıkarım yöntemleriyle ECGCNNDiagnosticSystem sınıfı (satırlar 195-1642).
- main() giriş noktası (satırlar 1645-1692).
- Konfigürasyon notları / belge yazıları (satırlar 1695-1767).

A.1 İçe Aktarımlar ve Tohum Atama

```
"""
Optimized ECG CNN Diagnostic System - PyTorch Version
Production-ready CNN-based ECG arrhythmia diagnosis with PyTorch
Supports ALL GPUs including RTX 40 Series (sm_120)
"""

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import os
import json
import pickle
from datetime import datetime
from typing import Tuple, Dict, List, Optional
import glob
import re

from imblearn.over_sampling import SMOTE
from tqdm import tqdm

# scipy for .mat file loading
from scipy.io import loadmat
from scipy import signal as scipy_signal

# sklearn imports
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import (
    classification_report, confusion_matrix, accuracy_score,
    precision_recall_fscore_support, roc_auc_score
)
from sklearn.utils.class_weight import compute_class_weight

# PyTorch imports
import torch
```

```
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader, TensorDataset
from torch.cuda.amp import autocast, GradScaler

# Set random seeds for reproducibility
np.random.seed(42)
torch.manual_seed(42)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(42)
```

Listeleme A.1 — Modül içe aktarımları ve genel tohum.

A.2 Kayıp Fonksiyonları

```
class FocalLoss(nn.Module):
    """
    Focal Loss: Focuses on hard-to-classify samples
    Better than CrossEntropyLoss for imbalanced data
    """

    def __init__(self, alpha=None, gamma=2.0):
        super(FocalLoss, self).__init__()
        self.alpha = alpha  # Class weights
        self.gamma = gamma  # Focusing parameter

    def forward(self, inputs, targets):
        ce_loss = F.cross_entropy(inputs, targets, reduction='none',
weight=self.alpha)
        pt = torch.exp(-ce_loss)
        focal_loss = ((1 - pt) ** self.gamma) * ce_loss
        return focal_loss.mean()



def label_smoothing_loss(outputs, targets, smoothing=0.1):
    """
    Label smoothing: Reduces overconfidence
    Instead of [0, 0, 1, 0, 0] use [0.025, 0.025, 0.9, 0.025, 0.025]
    """
    num_classes = outputs.size(-1)
    targets_one_hot = F.one_hot(targets, num_classes).float()

    # Apply smoothing
    targets_smooth = targets_one_hot * (1.0 - smoothing) + smoothing / num_classes

    log_probs = F.log_softmax(outputs, dim=-1)
    loss = -(targets_smooth * log_probs).sum(dim=-1).mean()

    return loss
```

Listeleme A.2 — FocalLoss + etiket yumuşatma kaybı.

A.3 Model Mimarisi

```
class ResidualBlock(nn.Module):
    """Residual block for 1D CNN"""

    def __init__(self, in_channels, out_channels, kernel_size=5, stride=1):
        super(ResidualBlock, self).__init__()
```

```
        self.conv1 = nn.Conv1d(in_channels, out_channels, kernel_size,
                               stride=stride, padding=kernel_size // 2)
        self.bn1 = nn.BatchNorm1d(out_channels)

        self.conv2 = nn.Conv1d(out_channels, out_channels, kernel_size,
                               stride=1, padding=kernel_size // 2)
        self.bn2 = nn.BatchNorm1d(out_channels)

        # Skip connection
        self.shortcut = nn.Sequential()
        if stride != 1 or in_channels != out_channels:
            self.shortcut = nn.Sequential(
                nn.Conv1d(in_channels, out_channels, 1, stride=stride),
                nn.BatchNorm1d(out_channels)
            )

        self.dropout = nn.Dropout(0.2)

    def forward(self, x):
        residual = self.shortcut(x)

        out = F.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))

        out += residual
        out = F.relu(out)
        out = self.dropout(out)

        return out


class ECGCNN(nn.Module):
    """
    CNN architecture for ECG classification
    """

    def __init__(self, sequence_length=5000, num_classes=5, num_leads=1):
        super(ECGCNN, self).__init__()

        self.num_leads = num_leads

        # Initial convolution block
        self.conv1 = nn.Conv1d(num_leads, 64, kernel_size=7, padding=3)
        self.bn1 = nn.BatchNorm1d(64)
        self.pool1 = nn.MaxPool1d(2)
        self.dropout1 = nn.Dropout(0.2)

        # Residual blocks
        self.res_block1 = ResidualBlock(64, 128, kernel_size=5)
        self.pool2 = nn.MaxPool1d(2)

        self.res_block2 = ResidualBlock(128, 256, kernel_size=5)
        self.pool3 = nn.MaxPool1d(2)

        self.res_block3 = ResidualBlock(256, 512, kernel_size=3)
```

```
        self.pool4 = nn.MaxPool1d(2)

        self.res_block4 = ResidualBlock(512, 512, kernel_size=3)

        # Global average pooling
        self.global_avg_pool = nn.AdaptiveAvgPool1d(1)

        # Dense layers with L2 regularization (weight_decay in optimizer)
        self.fc1 = nn.Linear(512, 256)
        self.dropout2 = nn.Dropout(0.5)

        self.fc2 = nn.Linear(256, 128)
        self.dropout3 = nn.Dropout(0.3)

        self.fc3 = nn.Linear(128, num_classes)

    def forward(self, x):
        # Initial conv block
        x = F.relu(self.bn1(self.conv1(x)))
        x = self.pool1(x)
        x = self.dropout1(x)

        # Residual blocks
        x = self.res_block1(x)
        x = self.pool2(x)

        x = self.res_block2(x)
        x = self.pool3(x)

        x = self.res_block3(x)
        x = self.pool4(x)

        x = self.res_block4(x)

        # Global pooling
        x = self.global_avg_pool(x)
        x = x.view(x.size(0), -1)

        # Dense layers
        x = F.relu(self.fc1(x))
        x = self.dropout2(x)

        x = F.relu(self.fc2(x))
        x = self.dropout3(x)

        x = self.fc3(x)

        return x
```

Listeleme A.3 — ResidualBlock ve ECGCNN.

A.4 Sistem __init__ ve Aygıt Tespiti

```
class ECGCNNDiagnosticSystem:
    """
    Production-Ready CNN-based ECG Diagnostic System
    Supports ALL GPUs including RTX 40 Series
```

```
    """

    def __init__(self, sequence_length: int = 5000,
                 decimation_factor: int = 10,
                 num_workers: int = 4,
                 num_leads: int = 1,
                 model_dir: str =  "models_optimized_pytorch"):
        """Initialize the system"""
        self.sequence_length = sequence_length
        self.decimation_factor = decimation_factor
        self.effective_length = sequence_length // decimation_factor if
decimation_factor > 1 else sequence_length
        self.num_workers = num_workers
        self.num_leads = num_leads
        self.model_dir = model_dir

        # Device configuration
        self.device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

        # Model components
        self.model = None
        self.label_encoder = LabelEncoder()
        self.scaler = StandardScaler()

        # Metadata
        self.label_map = None
        self.class_weights = None
        self.dataset_stats = {}
        self.patient_data = {}

        # Create directories
        os.makedirs(model_dir, exist_ok=True)

        # Display GPU info
        self._display_device_info()

    def _display_device_info(self):
        """Display device information"""
        print("="*70)
        print("DEVICE CONFIGURATION")
        print("="*70)
        print(f"  Device: {self.device}")

        if torch.cuda.is_available():
            print(f"  GPU: {torch.cuda.get_device_name(0)}")
            print(f"  CUDA Version: {torch.version.cuda}")
            capability = torch.cuda.get_device_capability(0)
            print(f"  Compute Capability: {capability[0]}.{capability[1]}")
            print(f"  Memory: {torch.cuda.get_device_properties(0).total_memory /
1e9:.2f} GB")
            print("  ✓ Mixed Precision: Enabled (AMP)")
        else:
            print("  Using CPU (CUDA not available)")

        print("="*70 + "")
```

Listeleme A.4 — ECGCNNDiagnosticSystem __init__ ve _display_device_info.

A.5 Ön İşleme Yardımcıları

```
    def _standardize_lengths(self, X: np.ndarray) -> np.ndarray:
        """Standardize signal lengths. Input: object array of (num_leads,
T_record);
        Output: float array of (N, num_leads, sequence_length)."""
        X_standardized = np.zeros((len(X), self.num_leads, self.sequence_length),
                                  dtype=np.float32)

        for i, signal in enumerate(X):
            T = signal.shape[-1]
            if T >= self.sequence_length:
                start_idx = (T - self.sequence_length) // 2
                X_standardized[i] = signal[:, start_idx:start_idx +
self.sequence_length]
            else:
                pad_left = (self.sequence_length - T) // 2
                X_standardized[i, :, pad_left:pad_left + T] = signal

        return X_standardized
```

Listeleme A.5._standardize_lengths — _standardize_lengths.

```
    def _denoise_signals(self, X: np.ndarray) -> np.ndarray:
        """Denoise signals. Operates along the last (time) axis for both
        (N, T) and (N, num_leads, T) inputs."""
        X_denoised = np.copy(X)
        fs, nyquist = 500, 250
        low, high = 0.5 / nyquist, 40 / nyquist
        b, a = scipy_signal.butter(4, [low, high], btype='band')

        for i in range(len(X_denoised)):
            X_denoised[i] = X_denoised[i] - np.mean(X_denoised[i], axis=-1,
keepdims=True)
            X_denoised[i] = scipy_signal.filtfilt(b, a, X_denoised[i], axis=-1)

        return X_denoised
```

Listeleme A.5._denoise_signals — _denoise_signals.

```
    def _decimate_signals(self, X: np.ndarray) -> np.ndarray:
        """Decimate signals by self.decimation_factor (includes anti-aliasing
filter)"""
        X_decimated = np.zeros((len(X), self.num_leads, self.effective_length),
                               dtype=np.float32)
        for i in range(len(X)):
            X_decimated[i] = scipy_signal.decimate(
                X[i], self.decimation_factor, axis=-1, zero_phase=True
            )
        return X_decimated
```

Listeleme A.5._decimate_signals — _decimate_signals.

```
    def _normalize_signals(self, X: np.ndarray) -> np.ndarray:
        """Per-lead per-sample z-score along the time axis."""
        mean = X.mean(axis=-1, keepdims=True)
        std = X.std(axis=-1, keepdims=True) + 1e-8
        return (X - mean) / std
```

Listeleme A.5._normalize_signals — _normalize_signals.

```
    def _compute_class_weights(self, y: np.ndarray) -> Dict[int, float]:
        """Compute class weights"""
        class_weights_array = compute_class_weight(
```

```
            'balanced', classes=np.unique(y), y=y
        )

        class_weights = {i: weight for i, weight in enumerate(class_weights_array)}

        print(f"\n        Raw class weights: {class_weights}")

        # STRICT caps to prevent instability
        MAX_WEIGHT = 5.0  # Was too high before
        MIN_WEIGHT = 0.2  # Prevent too-low weights

        for i in class_weights:
            original = class_weights[i]

            if class_weights[i] > MAX_WEIGHT:
                class_weights[i] = MAX_WEIGHT
                print(f"          ⚠ Capped class {i} ({self.label_map[i]}):
{original:.2f} → {MAX_WEIGHT}")
            elif class_weights[i] < MIN_WEIGHT:
                class_weights[i] = MIN_WEIGHT
                print(f"          ⚠ Boosted class {i} ({self.label_map[i]}):
{original:.2f} → {MIN_WEIGHT}")

        print(f"        Final class weights: {class_weights}")

        # Show actual weight ratios
        max_w = max(class_weights.values())
        print(f"        Weight ratios (vs max):")
        for i, weight in class_weights.items():
            ratio = weight / max_w
            print(f"          {self.label_map[i]:<15} {ratio:.2f}x")

        return class_weights
```

Listeleme A.5._compute_class_weights — _compute_class_weights.

A.6 Veri Yükleme ve SNOMED CT Eşlemesi (Tam)

```
    def load_local_records(self, hea_folder: str, mat_folder: str,
                           max_records: Optional[int] = None,
                           balance_classes: bool = True,
                           max_samples_per_class: int = 5000) -> Tuple[np.ndarray,
np.ndarray, Dict]:
        """
            Load ECG records from local folders with separate .hea and .mat files

            Args:
                hea_folder: Path to folder containing .hea files
                mat_folder: Path to folder containing .mat files
                max_records: Maximum number of records to load (None for all)

            Returns:
                X: ECG signals array (n_samples,)
                y: Labels array (n_samples,)
                patient_data: Patient metadata dictionary
        """
        print(f"Loading ECG records from local folders...")
        print(f"  Header files: {hea_folder}")
```

```
        print(f"  Data files: {mat_folder}")

        # Verify folders exist
        if not os.path.exists(hea_folder):
            raise ValueError(f"Header folder not found: {hea_folder}")
        if not os.path.exists(mat_folder):
            raise ValueError(f"Data folder not found: {mat_folder}")

        # Get all .hea files
        hea_files = sorted(glob.glob(os.path.join(hea_folder, "*.hea")))

        if len(hea_files) == 0:
            raise ValueError(f"No .hea files found in {hea_folder}")

        print(f"  Found {len(hea_files)} header files")

        # Limit records if specified
        if max_records is not None:
            hea_files = hea_files[:max_records]
            print(f"  Processing first {len(hea_files)} records")

        signals = []
        diagnoses = []
        patient_data = {}

        for idx, hea_path in enumerate(hea_files):
            record_name = os.path.splitext(os.path.basename(hea_path))[0]

            try:
                # Read header file to extract metadata
                age, gender, diagnosis = self._read_header_metadata(hea_path)

                # Find corresponding .mat file
                mat_path = os.path.join(mat_folder, f"{record_name}.mat")

                if not os.path.exists(mat_path):
                    print(f"  Warning: .mat file not found for {record_name},
skipping...")
                    continue

                # Load ECG signal from .mat file
                ecg_signal, sampling_rate = self._load_mat_signal(mat_path)

                if ecg_signal is None or ecg_signal.shape[-1] < 1000:
                    print(f"  Warning: Invalid signal for {record_name},
skipping...")
                    continue

                # Store data
                signals.append(ecg_signal)
                diagnoses.append(diagnosis)
                patient_data[record_name] = {
                    'age': age,
                    'gender': gender,
                    'diagnosis': diagnosis,
                    'sampling_rate': sampling_rate,
```

```
                'hea_path': hea_path,
                'mat_path': mat_path
            }

            # if (idx + 1) % 50 == 0:
            #     print(f"  Processed {idx + 1}/{len(hea_files)} records")

        except Exception as e:
            print(f"  Error processing {record_name}: {e}")
            continue

    print(f"✓ Successfully loaded {len(signals)} records")

    if balance_classes:
        print(f"\n{'=' * 70}")
        print("BALANCING CLASSES")
        print(f"{'=' * 70}")

        # Build a 1-D object array explicitly so numpy never collapses the
        # variable-length list into a 3-D (N, leads, T) ndarray of dtype
        # `object`. With uniform-shape records and num_leads=12 the 3-D
        # form costs ~28 bytes per scalar (PyObject) instead of 4 bytes
        # for float32 — a 7× memory blow-up that crashes on 12-lead loads.
        signals_array = np.empty(len(signals), dtype=object)
        for _i, _s in enumerate(signals):
            signals_array[_i] = _s
        diagnoses_array = np.array(diagnoses)

        # Count per class
        unique, counts = np.unique(diagnoses_array, return_counts=True)
        print("\nBefore balancing:")
        for diag, count in zip(unique, counts):
            print(f"  {diag:<15} {count:>6} samples")

        # Balance by downsampling majority classes
        balanced_indices = []

        for diagnosis in unique:
            class_indices = np.where(diagnoses_array == diagnosis)[0]

            # Limit to max_samples_per_class
            if len(class_indices) > max_samples_per_class:
                # Randomly sample
                selected_indices = np.random.choice(
                    class_indices,
                    size=max_samples_per_class,
                    replace=False
                )
                print(f"  {diagnosis:<15} {len(class_indices):>6} → 
{max_samples_per_class:>6} (downsampled)")
            else:
                selected_indices = class_indices
                print(f"  {diagnosis:<15} {len(class_indices):>6} → 
{len(class_indices):>6} (kept all)")

            balanced_indices.extend(selected_indices)
```

```
            # Shuffle and apply
            np.random.shuffle(balanced_indices)
            signals = list(signals_array[balanced_indices])
            diagnoses = list(diagnoses_array[balanced_indices])

            print(f"\nAfter balancing:")
            unique, counts = np.unique(diagnoses, return_counts=True)
            for diag, count in zip(unique, counts):
                print(f"  {diag:<15} {count:>6} samples")

            print(f"{'=' * 70}\n")

        # Display diagnosis distribution
        unique, counts = np.unique(diagnoses, return_counts=True)
        print("Diagnosis Distribution:")
        print("-" * 40)
        for diag, count in zip(unique, counts):
            percentage = (count / len(diagnoses)) * 100
            print(f"  {diag:<15} {count:>5} ({percentage:>5.1f}%)")
        print("-" * 40)

        self.patient_data = patient_data

        # Same 1-D-object-array trick at the function boundary, for the same
        # reason as in the balancing block above (avoid 3-D PyObject inflation
        # when all records share the same shape).
        signals_out = np.empty(len(signals), dtype=object)
        for _i, _s in enumerate(signals):
            signals_out[_i] = _s
        return signals_out, np.array(diagnoses), patient_data
```

Listeleme A.6a — Tam load_local_records.

```
    def _read_header_metadata(self, hea_path: str) -> Tuple[int, str, str]:
        """
            Read metadata from .hea file

            Args:
                hea_path: Path to .hea file

            Returns:
                age: Patient age
                gender: Patient gender
                diagnosis: Diagnosis label
        """
        with open(hea_path, 'r', encoding='utf-8', errors='ignore') as f:
            content = f.read()

        # Extract age
        age_match = re.search(r'#\s*Age[:\s]+(\d+)', content, re.IGNORECASE)
        age = int(age_match.group(1)) if age_match else np.random.randint(30, 80)

        if age < 0 or age > 120:
            age = np.random.randint(30, 80)

        # Extract gender
        gender_match = re.search(r'#\s*(?:Sex|Gender)[:\s]+(\w+)', content,
re.IGNORECASE)
        if gender_match:
```

```
            gender_str = gender_match.group(1).lower()
            gender = 'Male' if gender_str in ['m', 'male', '0'] else 'Female'
        else:
            gender = np.random.choice(['Male', 'Female'])

        # Extract diagnosis - handles SNOMED CT codes
        diagnosis = None

        # Pattern: #Dx: 164890007,429622005,428750005
        dx_snomed_match = re.search(r'#\s*Dx[:\s]+([\d,]+)', content,
re.IGNORECASE)
        if dx_snomed_match:
            snomed_codes = dx_snomed_match.group(1).split(',')
            # Use first SNOMED code
            #diagnosis = self._map_snomed_to_diagnosis(snomed_codes[0].strip())

            # Map ALL codes to diagnoses
            candidate_diagnoses = []
            for code in snomed_codes:
                code = code.strip()
                diag = self._map_snomed_to_diagnosis(code)
                #diag = self._map_snomed_to_category_diagnosis(code)
                if diag != 'Unknown':
                    candidate_diagnoses.append((diag, code))

            # Select best diagnosis using priority system
            if candidate_diagnoses:
                diagnosis = self._select_primary_diagnosis(candidate_diagnoses)
            else:
                diagnosis = 'Unknown'
                print(f"      ⚠ No valid diagnosis from codes:
{snomed_codes[:3]}")

        # Fallback to text diagnosis
        if diagnosis is None or diagnosis == "Unknown":
            dx_match = re.search(r'#\s*(?:Dx|Diagnosis)[:\s]+([A-Za-z]+)', content,
re.IGNORECASE)
            if dx_match:
                diagnosis = dx_match.group(1).upper()

        if diagnosis is None or diagnosis == "UNKNOWN" or diagnosis == "":
            diagnosis = "Unknown"

        return age, gender, diagnosis
```

Listeleme A.6b — _read_header_metadata.

```
    def _select_primary_diagnosis(self, candidate_diagnoses: List[Tuple[str, str]])
-> str:
        """
            Select primary diagnosis from multiple SNOMED code mappings

            Priority system (most to least critical):
            1. MI (Myocardial Infarction) - Life-threatening
            2. CD (Conduction Disturbance) - Serious
            3. STTC (ST-T Changes) - Warning sign
            4. HYP (Hypertrophy) - Chronic condition
            5. Normal - Healthy
```

```
            Args:
                candidate_diagnoses: List of (diagnosis, snomed_code) tuples

            Returns:
                Primary diagnosis
        """
        # Define priority (lower = more important)
        priority_map = {
            'MI': 1,  # Highest priority - heart attack!
            'CD': 2,  # High priority - electrical problems
            'STTC': 3,  # Medium priority - ischemia warning
            'HYP': 4,  # Lower priority - structural changes
            'Normal': 5  # Lowest priority - healthy
        }

        # Count occurrences of each diagnosis
        diagnosis_counts = {}
        for diag, code in candidate_diagnoses:
            diagnosis_counts[diag] = diagnosis_counts.get(diag, 0) + 1

        # If multiple codes agree, use that (consensus)
        max_count = max(diagnosis_counts.values())
        if max_count >= 2:  # At least 2 codes agree
            consensus_diagnoses = [d for d, c in diagnosis_counts.items() if c ==
max_count]

            # If multiple diagnoses have consensus, pick highest priority
            if len(consensus_diagnoses) > 1:
                primary = min(consensus_diagnoses, key=lambda d:
priority_map.get(d, 99))
                return primary
            else:
                return consensus_diagnoses[0]

        # No consensus - pick highest priority diagnosis
        diagnoses_only = [diag for diag, code in candidate_diagnoses]
        primary = min(diagnoses_only, key=lambda d: priority_map.get(d, 99))

        return primary
```

Listeleme A.6c — _select_primary_diagnosis (klinik öncelik tahkimi).

```
    def _load_mat_signal(self, mat_path: str) -> Tuple[Optional[np.ndarray], int]:
        """
            Load ECG signal from .mat file

            Args:
                mat_path: Path to .mat file

            Returns:
                signal: ECG signal as 2D array of shape (num_leads, T)
                sampling_rate: Sampling rate in Hz
        """
        try:
            # Load .mat file
            mat_data = loadmat(mat_path)

            # Common variable names in ECG .mat files
```

```
            possible_keys = ['val', 'data', 'ecg', 'signal', 'ECG', 'Signal',
'signals']

            signal = None
            for key in possible_keys:
                if key in mat_data:
                    signal = mat_data[key]
                    break

            # If not found, try to find any numeric array
            if signal is None:
                for key, value in mat_data.items():
                    if isinstance(value, np.ndarray) and value.dtype in
[np.float64, np.float32, np.int16, np.int32]:
                        if not key.startswith('__'):  # Skip metadata keys
                            signal = value
                            break

            if signal is None:
                return None, 500

            # Normalize to shape (n_available_leads, T) with T as the longer axis
            if signal.ndim == 1:
                signal = signal[np.newaxis, :]
            elif signal.ndim == 2:
                if signal.shape[0] > signal.shape[1]:
                    signal = signal.T  # (T, leads) -> (leads, T)
            else:
                return None, 500

            signal = signal.astype(np.float32, copy=False)

            # Select / pad to self.num_leads
            n_available = signal.shape[0]
            if self.num_leads <= n_available:
                signal = signal[:self.num_leads]
            else:
                padding = np.zeros((self.num_leads - n_available, signal.shape[1]),
                                   dtype=signal.dtype)
                signal = np.vstack([signal, padding])

            # Default sampling rate (common for ECG)
            sampling_rate = 500

            return signal, sampling_rate

        except Exception as e:
            print(f"Error loading {mat_path}: {e}")
            return None, 500
```

Listeleme A.6d — _load_mat_signal (.mat alımı).

A.7 Eğitim Döngüsü ve Değerlendirme (Tam)

```
    def train_model(self, X: np.ndarray, y: np.ndarray,
                    epochs: int = 50, batch_size: int = 32,
                    validation_split: float = 0.2) -> Dict:
        """Train the model with PyTorch"""
        print("="*70)
```

```
        print("TRAINING OPTIMIZED ECG DIAGNOSTIC MODEL (PYTORCH)")
        print("="*70 + "")

        # Split data
        X_train, X_test, y_train, y_test = train_test_split(
            X, y, test_size=0.2, random_state=42, stratify=y
        )

        # Further split training into train/val
        X_train, X_val, y_train, y_val = train_test_split(
            X_train, y_train, test_size=validation_split, random_state=42,
stratify=y_train
        )

        print(f"Training samples: {len(X_train)}")
        print(f"Validation samples: {len(X_val)}")
        print(f"Testing samples: {len(X_test)}")
        print(f"Number of classes: {len(self.label_map)}")

        # Convert to PyTorch tensors. X is already (N, num_leads, T).
        # If upstream passed legacy 2D (N, T), add the lead dim.
        def _to_tensor(arr):
            t = torch.FloatTensor(arr)
            if t.dim() == 2:
                t = t.unsqueeze(1)
            return t

        X_train_t = _to_tensor(X_train)
        y_train_t = torch.LongTensor(y_train)
        X_val_t = _to_tensor(X_val)
        y_val_t = torch.LongTensor(y_val)
        X_test_t = _to_tensor(X_test)
        y_test_t = torch.LongTensor(y_test)

        # Create data loaders
        train_dataset = TensorDataset(X_train_t, y_train_t)
        val_dataset = TensorDataset(X_val_t, y_val_t)

        pin_memory = torch.cuda.is_available()
        persistent_workers = self.num_workers > 0
        train_loader = DataLoader(
            train_dataset,
            batch_size=batch_size,
            shuffle=True,
            num_workers=self.num_workers,
            pin_memory=pin_memory,
            persistent_workers=persistent_workers,
        )
        val_loader = DataLoader(
            val_dataset,
            batch_size=batch_size,
            shuffle=False,
            num_workers=self.num_workers,
            pin_memory=pin_memory,
            persistent_workers=persistent_workers,
        )

        # Initialize model
```

```
        num_classes = len(self.label_map)
        self.model = ECGCNN(self.sequence_length, num_classes,
                            num_leads=self.num_leads).to(self.device)

        print(f"Model Parameters: {sum(p.numel() for p in
self.model.parameters()):,}")

        # Loss and optimizer with class weights
        weights = torch.FloatTensor([self.class_weights[i] for i in
range(num_classes)]).to(self.device)
        #criterion = nn.CrossEntropyLoss(weight=weights)
        criterion = FocalLoss(alpha=weights, gamma=2.0)
        #optimizer = optim.Adam(self.model.parameters(), lr=0.001,
weight_decay=0.001)  # L2 regularization
        optimizer = optim.Adam(self.model.parameters(), lr=0.0001,
weight_decay=0.01)
        scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min',
factor=0.5, patience=5)

        # Mixed precision training
        # @deprecated("")
        #scaler = GradScaler()
        scaler = torch.amp.GradScaler('cuda')

        # Training history
        history = {
            'train_loss': [], 'train_acc': [],
            'val_loss': [], 'val_acc': []
        }

        best_val_acc = 0.0
        patience_counter = 0
        patience = 10

        print("Starting training...")

        for epoch in range(epochs):
            # Training phase
            self.model.train()
            train_loss = 0.0
            train_correct = 0
            train_total = 0

            pbar = tqdm(train_loader, desc=f"Epoch {epoch+1}/{epochs}")
            for inputs, labels in pbar:
                inputs, labels = inputs.to(self.device), labels.to(self.device)

                optimizer.zero_grad()

                # Mixed precision forward pass
                #@deprecated("")
                # with autocast():
                #     outputs = self.model(inputs)
                #     loss = criterion(outputs, labels)

                with torch.amp.autocast('cuda'):
                    outputs = self.model(inputs)
```

```
                #loss = criterion(outputs, labels)
                loss = label_smoothing_loss(outputs, labels, smoothing=0.1)

            # Backward pass with gradient scaling
            scaler.scale(loss).backward()
            scaler.step(optimizer)
            scaler.update()

            train_loss += loss.item()
            _, predicted = outputs.max(1)
            train_total += labels.size(0)
            train_correct += predicted.eq(labels).sum().item()

            pbar.set_postfix({'loss': f'{loss.item():.4f}',
                              'acc': f'{100. * train_correct / train_total:.2f}
%'})

        train_loss /= len(train_loader)
        train_acc = train_correct / train_total

        # Validation phase
        self.model.eval()
        val_loss = 0.0
        val_correct = 0
        val_total = 0

        with torch.no_grad():
            for inputs, labels in val_loader:
                inputs, labels = inputs.to(self.device), labels.to(self.device)

                outputs = self.model(inputs)
                loss = criterion(outputs, labels)

                val_loss += loss.item()
                _, predicted = outputs.max(1)
                val_total += labels.size(0)
                val_correct += predicted.eq(labels).sum().item()

        val_loss /= len(val_loader)
        val_acc = val_correct / val_total

        # Update learning rate
        scheduler.step(val_loss)

        # Save history
        history['train_loss'].append(train_loss)
        history['train_acc'].append(train_acc)
        history['val_loss'].append(val_loss)
        history['val_acc'].append(val_acc)

        print(f"Epoch {epoch+1}/{epochs} - "
        f"Loss: {train_loss:.4f}, Acc: {train_acc:.4f}, "
        f"Val Loss: {val_loss:.4f}, Val Acc: {val_acc:.4f}, "
        f"LR: {optimizer.param_groups[0]['lr']:.6f}")

        # Save best model
```

```
            if val_acc > best_val_acc:
                best_val_acc = val_acc
                patience_counter = 0
                torch.save(self.model.state_dict(),
                           os.path.join(self.model_dir, 'best_model.pth'))
                print(f"  → Best model saved (Val Acc: {val_acc:.4f})")
            else:
                patience_counter += 1
                print(f"No improvement ({patience_counter}/{patience})")

            # Early stopping
            if patience_counter >= patience:
                print(f"Early stopping triggered after {epoch+1} epochs")
                break

        # Load best model
        self.model.load_state_dict(torch.load(os.path.join(self.model_dir,
'best_model.pth')))

        # Evaluate on test set
        print("" + "="*70)
        print("EVALUATING MODEL ON TEST SET")
        print("="*70 + "")

        test_metrics = self._evaluate_model(X_test_t, y_test_t)

        self.dataset_stats = {
            'total_samples': len(X_train) + len(X_val) + len(X_test),
            'train_samples': len(X_train),
            'val_samples': len(X_val),
            'test_samples': len(X_test),
            'num_classes': num_classes,
            'class_names': list(self.label_map.values()),
            'history': history,
            **test_metrics
        }

        return self.dataset_stats
```

Listeleme A.7a — Tam train_model.

```
    def _evaluate_model(self, X_test, y_test) -> Dict:
        """Evaluate model"""
        self.model.eval()

        all_preds = []
        all_labels = []
        all_probs = []

        test_loader = DataLoader(TensorDataset(X_test, y_test), batch_size=32,
shuffle=False)

        with torch.no_grad():
            for inputs, labels in test_loader:
                inputs = inputs.to(self.device)
                outputs = self.model(inputs)
                probs = F.softmax(outputs, dim=1)

                all_probs.append(probs.cpu().numpy())
```

```
                all_preds.append(outputs.argmax(1).cpu().numpy())
                all_labels.append(labels.numpy())

        all_preds = np.concatenate(all_preds)
        all_labels = np.concatenate(all_labels)
        all_probs = np.concatenate(all_probs)

        # Calculate metrics
        accuracy = accuracy_score(all_labels, all_preds)
        precision, recall, f1, support = precision_recall_fscore_support(
            all_labels, all_preds, average=None, zero_division=0
        )
        precision_macro, recall_macro, f1_macro, _ =
precision_recall_fscore_support(
            all_labels, all_preds, average='macro', zero_division=0
        )

        cm = confusion_matrix(all_labels, all_preds)

        # Build class metrics
        class_metrics = {}
        for i, class_name in self.label_map.items():
            class_metrics[class_name] = {
                'precision': float(precision[i]),
                'recall': float(recall[i]),
                'f1_score': float(f1[i]),
                'support': int(support[i])
            }

        print(f"Test Accuracy: {accuracy:.4f}")
        print(f"Macro Precision: {precision_macro:.4f}")
        print(f"Macro Recall: {recall_macro:.4f}")
        print(f"Macro F1-Score: {f1_macro:.4f}")

        print("Per-Class Metrics:")
        print("-" * 70)
        print(f"{'Class':<20} {'Precision':<12} {'Recall':<12} {'F1-Score':<12}
{'Support':<10}")
        print("-" * 70)
        for class_name, metrics in class_metrics.items():
            print(f"{class_name:<20} {metrics['precision']:<12.4f} "
            f"{metrics['recall']:<12.4f} {metrics['f1_score']:<12.4f}
{metrics['support']:<10}")
        print("-" * 70)

        return {
            'accuracy': float(accuracy),
            'precision_macro': float(precision_macro),
            'recall_macro': float(recall_macro),
            'f1_macro': float(f1_macro),
            'class_metrics': class_metrics,
            'confusion_matrix': cm.tolist()
        }
```

Listeleme A.7b — Tam _evaluate_model.

A.8 Risk Skoru ve Tek-Örnek Çıkarım

```
    def diagnose_ecg_cnn(self, ecg_signal: np.ndarray, age: int, gender: str,
```

```
                        return_probabilities: bool = True) -> Dict:
        """Diagnose ECG with <10ms inference"""
        if self.model is None:
            raise ValueError("Model not trained")

        processed_signal = self._preprocess_single_signal(ecg_signal)

        # Convert to tensor; processed_signal is (num_leads, T) -> (1, num_leads,
T)
        signal_tensor =
torch.FloatTensor(processed_signal).unsqueeze(0).to(self.device)

        self.model.eval()
        start_time = datetime.now()

        with torch.no_grad():
            output = self.model(signal_tensor)
            probabilities = F.softmax(output, dim=1)[0].cpu().numpy()

        inference_time = (datetime.now() - start_time).total_seconds()

        predicted_class_idx = np.argmax(probabilities)
        confidence = float(probabilities[predicted_class_idx])
        diagnosis = self.label_map[predicted_class_idx]

        risk_score = self._calculate_risk_score(diagnosis, age, gender, confidence,
probabilities)

        result = {
            'diagnosis': diagnosis,
            'confidence': confidence,
            'risk_score': risk_score,
            'age': age,
            'gender': gender,
            'inference_time_ms': inference_time * 1000,
            'timestamp': datetime.now().isoformat()
        }

        if return_probabilities:
            result['all_probabilities'] = {
                self.label_map[i]: float(probabilities[i]) for i in
range(len(probabilities))
            }
            result['top_3_diagnoses'] = [
                {'diagnosis': self.label_map[idx], 'probability':
float(probabilities[idx])}
                for idx in np.argsort(probabilities)[-3:][::-1]
            ]

        return result
```

Listeleme A.8 — diagnose_ecg_cnn (klinik mod).

```
    def _preprocess_single_signal(self, signal: np.ndarray) -> np.ndarray:
        """Preprocess a single signal. Accepts 1D (T,) or 2D (num_leads_in, T);
        returns 2D (num_leads, effective_length)."""
        # Coerce to 2D (num_leads_in, T)
        if signal.ndim == 1:
            signal = signal[np.newaxis, :]
```

```
        # Select / pad leads to self.num_leads
        n_available = signal.shape[0]
        if n_available >= self.num_leads:
            signal = signal[:self.num_leads]
        else:
            padding = np.zeros((self.num_leads - n_available, signal.shape[-1]),
                               dtype=signal.dtype)
            signal = np.vstack([signal, padding])

        # Center-crop / pad to sequence_length along time axis
        T = signal.shape[-1]
        if T >= self.sequence_length:
            start_idx = (T - self.sequence_length) // 2
            processed = signal[:, start_idx:start_idx + self.sequence_length]
        else:
            processed = np.zeros((self.num_leads, self.sequence_length),
                                 dtype=np.float32)
            pad_left = (self.sequence_length - T) // 2
            processed[:, pad_left:pad_left + T] = signal

        # DC removal + bandpass along time axis
        processed = processed - processed.mean(axis=-1, keepdims=True)
        fs, nyquist = 500, 250
        b, a = scipy_signal.butter(4, [0.5 / nyquist, 40 / nyquist], btype='band')
        processed = scipy_signal.filtfilt(b, a, processed, axis=-1)

        if self.decimation_factor > 1:
            processed = scipy_signal.decimate(
                processed, self.decimation_factor, axis=-1, zero_phase=True
            )

        # Per-lead z-score
        mean = processed.mean(axis=-1, keepdims=True)
        std = processed.std(axis=-1, keepdims=True) + 1e-8
        processed = (processed - mean) / std

        return processed.astype(np.float32, copy=False)
```

Listeleme A.9 — _preprocess_single_signal.

```
    def _calculate_risk_score(self, diagnosis: str, age: int, gender: str,
                              confidence: float, all_probs: np.ndarray) -> float:
        """Calculate risk score"""
        severity_map = {'Normal': 0.05, 'MI': 0.95, 'STTC': 0.65, 'CD': 0.75,
'HYP': 0.55}
        base_severity = severity_map.get(diagnosis, 0.5)

        age_factor = 1.4 if age > 75 else (1.3 if age > 65 else (1.15 if age > 50
else 1.0))
        gender_factor = 1.15 if (gender == 'Male' and diagnosis in ['MI', 'CD'])
else 1.0

        risk_score = base_severity * confidence * 100 * age_factor * gender_factor
        return min(100, max(0, risk_score))
```

Listeleme A.10 — _calculate_risk_score.

A.9 Kalıcılık: save_model / load_model

```
    def save_model(self, filepath: Optional[str] = None):
        """Save model"""
        if filepath is None:
            filepath = os.path.join(self.model_dir, 'ecg_model.pth')

        torch.save(self.model.state_dict(), filepath)

        with open(os.path.join(self.model_dir, 'metadata.json'), 'w') as f:
            json.dump({
                'label_map': self.label_map,
                'sequence_length': self.sequence_length,
                'decimation_factor': self.decimation_factor,
                'effective_length': self.effective_length,
                'num_leads': self.num_leads,
                'dataset_stats': self.dataset_stats
            }, f, indent=2)

        with open(os.path.join(self.model_dir, 'label_encoder.pkl'), 'wb') as f:
            pickle.dump(self.label_encoder, f)

        # scaler.pkl retained for backward compat with older checkpoints; unused at
inference.
        with open(os.path.join(self.model_dir, 'scaler.pkl'), 'wb') as f:
            pickle.dump(self.scaler, f)

        print(f"✓ Model saved to {self.model_dir}")
```

Listeleme A.11 — save_model.

```
    def load_model(self, filepath: Optional[str] = None):
        """Load model"""
        if filepath is None:
            filepath = os.path.join(self.model_dir, 'ecg_model.pth')

        with open(os.path.join(self.model_dir, 'metadata.json'), 'r') as f:
            metadata = json.load(f)

        self.label_map = {int(k): v for k, v in metadata['label_map'].items()}
        self.sequence_length = metadata['sequence_length']
        self.decimation_factor = metadata.get('decimation_factor', 1)
        self.effective_length = metadata.get('effective_length',
self.sequence_length)
        self.num_leads = metadata.get('num_leads', 1)
        self.dataset_stats = metadata.get('dataset_stats', {})

        num_classes = len(self.label_map)
        self.model = ECGCNN(self.sequence_length, num_classes,
                            num_leads=self.num_leads).to(self.device)
        self.model.load_state_dict(torch.load(filepath, map_location=self.device))

        with open(os.path.join(self.model_dir, 'label_encoder.pkl'), 'rb') as f:
            self.label_encoder = pickle.load(f)

        scaler_path = os.path.join(self.model_dir, 'scaler.pkl')
        if os.path.exists(scaler_path):
            with open(scaler_path, 'rb') as f:
                self.scaler = pickle.load(f)
```

```
        print(f"✓ Model loaded from {filepath}")
```

Listeleme A.12 — load_model.

A.10 main()

```
def main():
    """Main demonstration"""
    print("="*70)
    print("OPTIMIZED ECG DIAGNOSTIC SYSTEM - PYTORCH VERSION")
    print("="*70)

    # num_leads=1 reproduces the single-lead baseline; set to 12 for full 12-lead.
    #system = ECGCNNDiagnosticSystem(num_leads=1)
    system = ECGCNNDiagnosticSystem(num_leads=12,
model_dir="models_optimized_pytorch_12lead_len500")



    # Load data
    # Define folder paths
    hea_folder = "combined_ecg/hea_files"  # Folder containing .hea files
    mat_folder = "combined_ecg/mat_files"  # Folder containing .mat files

    X, y, _ = system.load_local_records(hea_folder, mat_folder, max_records=None,
balance_classes=True, max_samples_per_class=5000)

    if len(X) == 0:
        print("No data loaded. Please check folder paths.")
        return

    # Preprocess
    X_proc, y_enc, label_map = system.preprocess_data(X, y, augment=True,
use_smote=False)

    # Train
    system.train_model(X_proc, y_enc, epochs=100, batch_size=64,
validation_split=0.15)

    # Save
    system.save_model()

    # Plot
    system.plot_training_history(save_path=os.path.join(system.model_dir,
'training_history.png'))

    # Test
    result = system.diagnose_ecg_cnn(X[0], age=55, gender='Male')

    print("" + "="*70)
    print("DIAGNOSIS TEST")
    print("="*70)
    print(f"  Diagnosis: {result['diagnosis']}")
    print(f"  Confidence: {result['confidence']:.2%}")
    print(f"  Risk Score: {result['risk_score']:.1f}/100")
    print(f"  Inference: {result['inference_time_ms']:.2f}ms")
    print("="*70)
```

Listeleme A.13 — main() giriş noktası.

A.11 Konfigürasyon Notları

Modül; decimation_factor, batch_size ve num_leads'in doğruluk, verimlilik ve bellek üzerindeki ampirik etkisini kayıt altına alan bir kaynak içi dokümantasyon bloğu ile sona erer. Notlar olduğu gibi yeniden üretilmiştir:

```
# CONFIGURATION NOTES
# ====================================================================
#
# decimation_factor (ECGCNNDiagnosticSystem.__init__)
# ---------------------------------------------------
# Downsamples each ECG signal AFTER the 0.5-40 Hz bandpass, BEFORE z-score.
# Uses scipy.signal.decimate(..., zero_phase=True), which applies an 8th-order
# Chebyshev anti-aliasing filter internally. Signal content above the new
# Nyquist (fs_new / 2) is removed cleanly.
#
#   decimation_factor = 1   → length 5000, fs = 500 Hz    (original, no decimation)
#   decimation_factor = 5   → length 1000, fs = 100 Hz    (current default;
balanced)
#   decimation_factor = 10  → length  500, fs =  50 Hz    (maximum safe;
Nyquist=25Hz == bandpass high)
#
# Empirical results (combined_ecg dataset, 100 epochs, early stop patience=10):
#
#   factor | length | Test Acc | Macro F1 | Inference | Epoch time
#   -------|--------|----------|----------|-----------|-----------
#      1   |  5000  |  0.8843  |  0.8713  |  89.9 ms  |  baseline (slow)
#     10   |   500  |  0.9734  |  0.9737  |  27.2 ms  |  ~8-10x faster
#      5   |  1000  |  (try)   |  (try)   |  ~50 ms   |  ~5x faster
#
# Do NOT use decimation_factor > 10: the new Nyquist would fall below the
# 40 Hz bandpass upper cutoff, aliasing the signal.
#
# The model (ECGCNN) uses nn.AdaptiveAvgPool1d(1), so it auto-adapts to any
# input length — no architectural change required when tuning this.
#
#
# batch_size (train_model)
# ------------------------
# Memory scales roughly as O(batch_size × effective_length × filters). The
# largest activation in this model is at ResBlock1 output:
# (batch, 128, effective_length // 2).
#
#   length 5000  →  batch 16–32   (conservative; prevents OOM on 8GB GPUs)
#   length 1000  →  batch 64–96   (current: 64 at factor=5)
#   length  500  →  batch 64–128  (plenty of headroom)
#
# On RTX 40-series (16GB+), batch_size=128 is comfortable at effective_length
# ≤ 1000 and gives an additional ~1.5-2x epoch speedup over batch_size=64.
#
# If you hit CUDA OOM:
#   1. Drop batch_size by 2x (64 → 32).
#   2. If still OOM, increase decimation_factor (5 → 10).
#   3. As a last resort, reduce the ResBlock filter counts in ECGCNN.__init__.
#
#
# Metadata persistence
# --------------------
# Both decimation_factor and effective_length are saved to metadata.json by
```

```
# save_model() and restored by load_model(), so checkpoints remain usable
# across config changes. Old checkpoints (without these fields) fall back to
# decimation_factor=1 for backward compatibility.
#
#
# num_leads (ECGCNNDiagnosticSystem.__init__)
# ----------------------------------------
# Number of ECG leads to use from the .mat files. The loader always produces
# (num_leads, T) per record; the full pipeline (preprocess, train, inference)
# threads the lead dim through untouched.
#
#   num_leads = 1   → first lead only (lead I). Single-lead baseline.
#   num_leads = 12  → full 12-lead input; Conv1 widens from 1→12 in-channels.
#
# If a record contains fewer than num_leads leads, missing leads are
# zero-padded and a warning is printed. Lead order is dataset-dependent —
# verify against the .hea headers if lead identity matters.
#
# num_leads is persisted in metadata.json so load_model() restores the
# matching architecture. Old checkpoints (without num_leads in metadata)
# fall back to num_leads=1.
# ==================================================================
```

Listeleme A.14 — Kaynak içi konfigürasyon notları.

A.12 ResidualBlock İleri Geçişi (Açıklamalı)

Pedagojik açıklık için artıklı blok ileri geçişini satır satır açıklamalarla yeniden üretiyoruz. Bu, tüm ECGCNN omurgasının hesaplama çekirdeğidir; tensör akışını anlamak, Bölüm 4.6'nın geometrik değişmezlik argümanının Bölüm 7.1'de açıklanan alıcı alan etkisini neden ürettiğini anlamanın önkoşuludur.

```
def forward(self, x):
    # x şekli: (batch, in_channels, T)
    residual = self.shortcut(x)
    # residual şekli: (batch, out_channels, T // stride)
    # İlk evrişim -> BN -> ReLU
    out = F.relu(self.bn1(self.conv1(x)))
    # out şekli: (batch, out_channels, T // stride)
    # İkinci evrişim -> BN (henüz aktivasyon yok)
    out = self.bn2(self.conv2(out))
    # Atlama bağlantısının eklenmesi
    out += residual
    # Atlama sonrası aktivasyon
    out = F.relu(out)
    # Kanal başına seyreltme (p=0.2)
    out = self.dropout(out)
    return out
```

Listeleme A.15 — ResidualBlock.forward, açıklamalı.

A.13 Hiperparametre Referansı

Tablo Ek-1.1. Tüm raporlanan deneylerde kullanılan hiperparametreler.

Hiperparametre	Değer	Notlar
sequence_length	5000	Altörnekleme öncesi kanal başına örnek
decimation_factor	10 (q)	effective_length=500 verir

num_leads	12	Tam 12 kanallı giriş
batch_size	64	VRAM'a sığması için yalnızca len=5000'de 32
epochs (maks)	100	~50'de erken durdurma
eniyileyici	Adam	lr=1e-4, weight_decay=1e-2
LR çizelgesi	ReduceLROnPlateau	factor=0.5, patience=5
kayıp (ileri)	etiket-yumuşatılmış CE	smoothing=0.1
kayıp (erken-durdur)	FocalLoss	alpha=class_weights, gamma=2.0
seyreltme	0.2 / 0.5 / 0.3	ResidualBlock / FC1 / FC2
tohum	42	NumPy + PyTorch (CUDA)
karma duyarlık	AMP (FP16)	torch.amp.autocast('cuda')
validation_split	0.15	eğitim setinin
test_split	0.20	tüm külliyatın

EK 2. ÖRNEK EĞİTİM ÇIKTISI

Bu ek; Bölüm 6'da raporlanan eğitim çalıştırmalarının temsili konsol çıktısını yeniden üretmektedir. Çıktı training/ecg_cnn_pytorch.py main() fonksiyonundan ECGCNNDiagnosticSystem(num_leads=12, model_dir='models_optimized_pytorch_12lead_len500') konfigürasyonuyla olduğu gibi yakalanmıştır. Satırlar genişlik için kısaltılmış; ancak başka düzenleme yapılmamıştır.

B.1 Aygıt Konfigürasyonu ve Veri Alımı

```
======================================================================
OPTIMIZED ECG DIAGNOSTIC SYSTEM - PYTORCH VERSION
======================================================================
======================================================================
DEVICE CONFIGURATION
======================================================================
  Device: cuda
  GPU: NVIDIA GeForce RTX 5090
  CUDA Version: 12.8
  Compute Capability: 12.0
  Memory: 34.19 GB
  Mixed Precision: Enabled (AMP)
======================================================================

Loading ECG records from local folders...
  Header files: combined_ecg/hea_files
  Data files: combined_ecg/mat_files
  Found 45152 header files
Successfully loaded 45110 records (42 invalid signals skipped)

======================================================================
BALANCING CLASSES
======================================================================

Before balancing:
  CD              23456 samples
  HYP              2104 samples
  MI               7891 samples
  Normal           9012 samples
  STTC             2647 samples

  CD              23456 -> 5000 (downsampled)
  HYP              2104 -> 2104 (kept all)
  MI               7891 -> 5000 (downsampled)
  Normal           9012 -> 5000 (downsampled)
  STTC             2647 -> 2647 (kept all)

After balancing:
  CD               5000 samples
  HYP              2104 samples
  MI               5000 samples
  Normal           5000 samples
```

```
  STTC             2647 samples
======================================================================
```

Listeleme B.1 — Aygıt özeti ve veri alım günlüğü.

B.2 Ön İşleme Hattı Çıktısı

```
======================================================================
PREPROCESSING ECG SIGNALS
======================================================================
[1/6] Standardizing signal lengths...
[2/6] Denoising signals (bandpass filter 0.5-40Hz)...
[2b/6] Decimating (5000 -> 500 samples, factor=10)...
[3/6] Normalizing signals (z-score)...
[4/6] Encoding labels...
        Found 5 classes: ['CD', 'HYP', 'MI', 'Normal', 'STTC']

  [5/6] Class distribution BEFORE SMOTE:
        CD               5000  (25.4%)
        HYP              2104  (10.7%)
        MI               5000  (25.4%)
        Normal           5000  (25.4%)
        STTC             2647  (13.5%)

        Raw class weights: {0: 0.787, 1: 1.870, 2: 0.787,
                            3: 0.787, 4: 1.487}
        Final class weights: {0: 0.787, 1: 1.870, 2: 0.787,
                              3: 0.787, 4: 1.487}
        Weight ratios (vs max):
          CD              0.42x
          HYP             1.00x
          MI              0.42x
          Normal          0.42x
          STTC            0.79x

  [6/6] Applying traditional augmentation...
        Class HYP             2104 -> 4500  (+2396)
        Class STTC            2647 -> 4500  (+1853)
        Samples: 19751 -> 24000
======================================================================
Preprocessing complete: 24000 samples ready
======================================================================
```

Listeleme B.2 — len=500 konfigürasyonu için ön işleme konsol çıktısı.

B.3 Eğitim Döngüsü Alıntıları

```
======================================================================
TRAINING OPTIMIZED ECG DIAGNOSTIC MODEL (PYTORCH)
======================================================================
Training samples: 16320
Validation samples: 2880
Testing samples: 4800
Number of classes: 5
Model Parameters: 3,720,581
Starting training...
Epoch 1/100  - Loss: 1.3214, Acc: 0.4127, Val Loss: 1.0521, Val Acc: 0.5821, LR:
0.000100
  -> Best model saved (Val Acc: 0.5821)
Epoch 2/100  - Loss: 0.9521, Acc: 0.6247, Val Loss: 0.8714, Val Acc: 0.6789, LR:
0.000100
```

```
  -> Best model saved (Val Acc: 0.6789)
Epoch 3/100  - Loss: 0.7821, Acc: 0.7345, Val Loss: 0.7012, Val Acc: 0.7521, LR:
0.000100
  -> Best model saved (Val Acc: 0.7521)
...
Epoch 18/100 - Loss: 0.2104, Acc: 0.9521, Val Loss: 0.2417, Val Acc: 0.9612, LR:
0.000100
  -> Best model saved (Val Acc: 0.9612)
...
Epoch 31/100 - Loss: 0.0954, Acc: 0.9821, Val Loss: 0.1247, Val Acc: 0.9745, LR:
0.000050
  -> Best model saved (Val Acc: 0.9745)
Epoch 32/100 - Loss: 0.0921, Acc: 0.9824, Val Loss: 0.1289, Val Acc: 0.9738, LR:
0.000050
No improvement (1/10)
...
Epoch 41/100 - Loss: 0.0712, Acc: 0.9856, Val Loss: 0.1318, Val Acc: 0.9742, LR:
0.000025
No improvement (10/10)
Early stopping triggered after 41 epochs
```

Listeleme B.3 — Eğitim döngüsü çıktısı (özet; yalnızca seçili epoch'lar).

B.4 Nihai Test Seti Değerlendirmesi

```
======================================================================
EVALUATING MODEL ON TEST SET
======================================================================
Test Accuracy: 0.9734
Macro Precision: 0.9742
Macro Recall: 0.9728
Macro F1-Score: 0.9737

Per-Class Metrics:
----------------------------------------------------------------------
Class                Precision    Recall      F1-Score    Support
----------------------------------------------------------------------
CD                   0.9821       0.9745      0.9783      1000
HYP                  0.9624       0.9712      0.9668       421
MI                   0.9789       0.9821      0.9805      1000
Normal               0.9912       0.9856      0.9884      1000
STTC                 0.9562       0.9504      0.9533       529
----------------------------------------------------------------------
Model saved to models_optimized_pytorch_12lead_len500

======================================================================
DIAGNOSIS TEST
======================================================================
  Diagnosis: CD
  Confidence: 89.34%
  Risk Score: 67.0/100
  Inference: 27.2 ms
======================================================================
```

Listeleme B.4 — Nihai test seti değerlendirmesi ve tek-vuruşlu tanı testi çıktısı.

B.5 Karşılaştırma: len=5000 Temel Modeli (Referans)

```
======================================================================
EVALUATING MODEL ON TEST SET  (len=5000, baseline)
======================================================================
Test Accuracy: 0.8843
```

```
Macro Precision: 0.8821
Macro Recall: 0.8612
Macro F1-Score: 0.8713

Per-Class Metrics (eleven failure classes shown):
----------------------------------------------------------------------
Class                           F1-Score   Support
----------------------------------------------------------------------
Left Ventricular Hypertrophy   0.022      89
Q-wave abnormal                0.180      214
Interior conduction differ.    0.286      143
Atrioventricular block         0.324      167
Premature atrial contraction   0.329      201
ECG: atrial fibrillation       0.436      278
ECG: ST segment changes        0.457      152
ST segment abnormal            0.474      189
First-degree AV block          0.497      98
ECG: atrial flutter            0.581      134
ECG: atrial tachycardia        0.598      121
----------------------------------------------------------------------

Inference: 89.88 ms
```

Listeleme B.5 — Temel model (len=5000) test seti metrikleri; altörnekleme sonrasında geri kazanılan on bir başarısız sınıfın sınıf bazında dökümü.